Subsymbolic AI

Ibrahim Nasser*

Last updated on May 26, 2026

Contents

1 Introduction

Def 1.1. Artificial intelligence (AI): The capability of computational systems to perform tasks typically associated with human intelligence, such as learning, reasoning, problem-solving, perception, and decision-making

Def 1.2. Symbolic AI: A subfield of Artificial Intelligence (AI) based on the assumption that many aspects of intelligence can be achieved by the manipulation of symbols, combining them into meaningful structures (expressions) and manipulating them (using processes) to produce new expressions.

Def 1.3. Statisitical AI: Remedies the two shortcomings of symbolic AI approaches: that all concepts represented by symbols are crisply defined, and that all aspects of the world are knowable/representable in principle. Statistical AI adopts sophisticated mathematical models of uncertainty and uses them to create more accurate world models and reason about them.

Def 1.4. Subsymbolic AI (a.k.a connectionism/neural AI): A subfield of AI that posits that intelligence is inherently tied to brains, where information is represented by a simple sequence pulses that are processed in parallel via simple calculations realized by neurons, and thus concentrates on neural computing.

Def 1.5. Embodied AI: Posits that intelligence cannot be achieved by reasoning about the state of the world (symbolically, statistically, or sub-symbolically), but must be embodied i.e. situated in the world, equipped with a "body" that can interact with it via sensors and actuators. Here, the main method for realizing intelligent behavior is by learning from the world.

Def 1.6. Reasoning: The process of producing valid arguments and predictive world models. There are three forms of reasoning:

- **deductive reasoning** to produce new knowledge from existing knowledge. *Example:* All humans are mortal. Socrates is a human. Therefore, Socrates is mortal.
- **inductive reasoning** to produce knowledge from perception. *Example:* The sun has risen every morning in recorded history. Therefore, the sun will rise tomorrow.
- **abductive reasoning** to produce explanations for observations and given knowledge. *Example:* The grass is wet this morning. If it rained last night, that would explain it. Therefore, it probably rained.

Def 1.7. Inference: The act or process of reaching a **conclusion** about something from known facts or evidence (jointly called **premises**)

Def 1.8. Formal Logic: The science of deductively valid inferences, meaning arguments whose conclusions necessarily follow from their premises by virtue of their form (their structure), regardless of the topic.

Def 1.9. Agent: An agent is a structure $A := \langle \mathcal{P}, \mathcal{A}, f \rangle$ where:

- $\mathcal{P}$ is a set of percepts
- $\mathcal{A}$ is a set of actions
- f is a function $f : \mathcal{P}^* \to \mathcal{A}$ that maps from percepts to actions.

In other words, an agent is anything that perceives its environment via sensors and acts on it with actuators.

Def 1.10. Performance Measure: A function that evaluates a sequence of environments.

Def 1.11. Rational: An agent is called **rational**, if it chooses whichever action maximizes the expected value of the performance measure given the percept sequence to date.

Def 1.12. PEAS: To design rational agents, we must specify the PEAS components: Performance Measure, Environment, Actuators, and Sensors.

Def 1.13. Environment Types: For an agent a we classify the environment e of a by its **type**, which is one of the following. We call e

1. *fully observable*, iff the a's sensors give it access to the complete state of the environment at any point in time; otherwise, we call it *partially observable*.
2. *deterministic*, iff the next state of the environment is completely determined by the current state and a's actions; otherwise, *stochastic*.
3. *episodic*, iff a's experience is divided into atomic episodes, where it perceives and then performs a single action, and the next episode does not depend on previous ones. Otherwise, *sequential*.
4. *dynamic*, iff the environment can change without an action performed by a; otherwise, *static*. If the environment does not change but a's performance measure does, we call e *semidynamic*.

5. *discrete*, iff the sets of e's state and a's actions are couuntable; otherwise, *continuous*.
6. *single-agent*, iff only a acts on e; otherwise *multi-agent*.

Def 1.14. Reflex: An agent $\langle \mathcal{P}, \mathcal{A}, f \rangle$ is called a **reflex agent**, iff it only takes the last percept into account when choosing an action, i.e. $f(p_1, \cdots, p_k) = f(p_k)\ \forall p_1 \cdots p_k \in \mathcal{P}$

Def 1.15. Model-Based Agent: A model-based agent $\langle \mathcal{P}, \mathcal{A}, \mathcal{S}, \mathcal{T}, s_0, S, a \rangle$ is an agent $\langle \mathcal{P}, \mathcal{A}, f \rangle$ whose actions depend on:

- a **world model**: a set $\mathcal{S}$ of possible *states*, and a start state $s_0 \in \mathcal{S}$.
- a **transition model** $\mathcal{T}$ that predicts a new state $\mathcal{T}(s, a)$ from a state s and an action a.
- a **sensor model** S that given a state s and a percept p determines a new state $S(s, p)$.
- an **action function** $a : \mathcal{S} \to \mathcal{A}$ that given a state $s \in \mathcal{S}$ selects the next action $a \in \mathcal{A}$.

> **Note**
>
> If the agent is in state s then it took action a and now perceives p, then the agent state will become $s' = S(p, \mathcal{T}(s, a))$ and accordingly take action $a' = a(s')$.

Def 1.16. Goal Based Agent: A goal based agent $\langle \mathcal{P}, \mathcal{A}, \mathcal{S}, \mathcal{T}, s_0, S, a, \mathcal{G} \rangle$ is a model based agent with an explicit set of goals. It consists of:

- a set of internal states $\mathcal{S}$ and an initial state $s_0 \in \mathcal{S}$,
- a transition model $\mathcal{T} : \mathcal{S} \times \mathcal{A} \to \mathcal{S}$,
- a state update function $S : \mathcal{P} \times \mathcal{S} \to \mathcal{S}$,
- a set of goals $\mathcal{G}$,
- a goal conditioned action function $a : \mathcal{S} \times \mathcal{G} \to \mathcal{A}$ selecting an action given a state and the goals.

Def 1.17. Utility-Based Agent: A utility-based agent uses a world model along with a utility function that models its preferences among the states of that world. It chooses the action that maximizes the expected utility.

Def 1.18. Learning Agent: A learning agent is an agent that can improve its own behavior through experience. It is composed of four main components:

- the performance element, which selects actions based on the agent's current percepts and represents its existing knowledge or behavior,
- the learning element, which improves the performance element over time by analyzing feedback and identifying how to make better decisions,
- the critic, which evaluates the agent's performance according to a given performance standard and provides feedback to the learning element,
- the problem generator, which suggests new and informative actions or experiences that help the agent explore and learn more effectively.

Def 1.19. State Representation: We call a state representation **atomic**, iff it has no internal structure (black box). However, iff each state is characterized by attributes and their values then the representation is **factored**. A **structured** state representation is when include representations of objects, their properties and relationships.

2 Probability Theory

Def 2.1. Sample Space: We say that a set Ω is a **sample space** of a random experiment, iff Ω contains all possible outcomes of that experiment. Each individual outcome $\omega \in \Omega$ is called a **sample point** or an **elementary outcome**.

Example 2.1. Throwing two dice $\rightsquigarrow \Omega = \{\langle 1,1\rangle, \langle 1,2\rangle, \cdots, \langle 6,6\rangle\}$ with $\Gamma(\Omega) = 36$

Example 2.2. Number of tosses until the first head $\rightsquigarrow \Omega = \{1,2,3,\cdots\} = \mathbb{N}$ with $\Gamma(\Omega) = \aleph_0$

Example 2.3. A random point in the interval $[0,1] \rightsquigarrow \Omega = [0,1]$ with $\Gamma(\Omega) = \mathfrak{c}$

Def 2.2. Event: Given a sample space Ω, we say that E is an **event** iff $E \subseteq \Omega$. That is, an event is a subset of the sample space. We say an event E occurs iff the outcome ω of the experiment satisfies $\omega \in E$.

Note

Because events are sets, we use Set Theory to describe relations between them:

- Complement $\bar{E}$: the event E that does not occur
- Union $A \cup B$: the event that A or B or both occurs
- Intersection $A \cap B$: the event that both A and B occur
- Disjoint / Mutually Exclusive: two events A and B are disjoint iff $A \cap B = \emptyset$

Note

We usually borrow propositional logic syntax to express union and intersection:

$$P(A \cup B) \rightsquigarrow P(A \vee B) \qquad P(A \cap B) \rightsquigarrow P(A \wedge B)$$

We also use "," to express joint probability

$$P(A \cap B) \rightsquigarrow P(A \wedge B) \rightsquigarrow P(A, B)$$

Example 2.4. The sample space for rolling one die is $\Omega = \{1,2,\cdots,6\}$, table 1 shows some events of interest.

Event Description	**Set Representation**
A: Even Number	$\{2,4,6\}$
B: Prime Number	$\{2,3,5\}$
$A \cap B$ (Even and Prime)	$\{2\}$
$A \cup B$ (Even or Prime)	$\{2,3,4,5,6\}$
$\bar{A}$ (Not Even)	$\{1,3,5\}$

Table 1: Examples of Events

Def 2.3. σ-algebra: Given a sample space Ω, a collection of subsets $\mathcal{F} \subseteq \mathcal{P}(\Omega)$ is called a **σ-algebra** iff it satisfies the following three axioms:

1. **Non-emptiness:** The sample space is included: $\Omega \in \mathcal{F}$. (which implies $\emptyset \in \mathcal{F}$ via complementation)
2. **Closure under complementation:** If $A \in \mathcal{F}$, then $\bar{A} \in \mathcal{F}$.
3. **Closure under countable unions:** If $A_1, A_2, \cdots \in \mathcal{F}$ is a sequence of events, then $\bigcup_{i=1}^{\infty} A_i \in \mathcal{F}$.

The pair $(\Omega, \mathcal{F})$ is called a **measurable space**.

Example 2.5. Let the sample space be $\Omega = \{1,2,3\}$.

1. Consider the collection $\mathcal{F} = \{\emptyset, \{1\}, \{2,3\}, \Omega\}$. This is a σ-algebra because:

- $\Omega \in \mathcal{F}$ (and $\emptyset \in \mathcal{F}$).
- It is closed under complements: $\bar{\{1\}} = \{2,3\} \in \mathcal{F}$ and $\bar{\emptyset} = \Omega \in \mathcal{F}$.
- It is closed under unions: $\{1\} \cup \{2,3\} = \Omega \in \mathcal{F}$.

2. Consider the collection $\mathcal{G} = \{\emptyset, \Omega, \{1\}, \{2\}\}$. This fails to be a σ-algebra for two reasons:

- No closure under Complements: $\{1\} \in \mathcal{G}$, but its complement $\{\bar{1}\} = \{2, 3\}$ is not in $\mathcal{G}$.
- No closure under Unions: $\{1\} \in \mathcal{G}$ and $\{2\} \in \mathcal{G}$, but their union $\{1, 2\}$ is not in $\mathcal{G}$.

Def 2.4. Probability Measure: A **probability measure** P is a function $P : \mathcal{F} \to [0, 1]$ that assigns a real number to every event $A \in \mathcal{F}$, satisfying the following **Kolmogorov Axioms**:

1. **Non-negativity:** $P(A) \geq 0$ for all $A \in \mathcal{F}$.
2. **Normalization:** $P(\Omega) = 1$.
3. **Countable Additivity:** For any sequence of disjoint events $A_1, A_2, \cdots \in \mathcal{F}$:

$$P\left(\bigcup_{i=1}^{\infty} A_i\right) = \sum_{i=1}^{\infty} P(A_i)$$

Def 2.5. Probability Space: A **probability space** is the triplet $\langle \Omega, \mathcal{F}, P \rangle$, where:

- Ω is the sample space.
- $\mathcal{F}$ is a σ-algebra on Ω.
- P is a probability measure defined on $(\Omega, \mathcal{F})$.

Example 2.6. For a fair 6-sided die roll, the probability space is defined as follows:

- $\Omega = \{1, 2, 3, 4, 5, 6\}$
- $\mathcal{F} = \mathcal{P}(\Omega)$ (the power set)
- $P(A) = \frac{\Gamma(A)}{\Gamma(\Omega)} = \frac{\Gamma(A)}{6}$

Note

From the Kolmogorov Axioms, we can derive the following fundamental properties of probability measures:

- **Complement Rule:** For any event A, $P(\bar{A}) = 1 - P(A)$.
- **Monotonicity:** If $A \subseteq B$, then $P(A) \leq P(B)$.
- **Inclusion-Exclusion Principle:** For any two events A and B:

$$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

- **Set Partition Identity:** For any two events A and B, B can be split into parts that are in A and parts that are not:

$$P(B) = P(A \cap B) + P(\bar{A} \cap B)$$

- **Impossible Event:** $P(\emptyset) = 0$.

Def 2.6. Random Variable: Given a probability space $\langle \Omega, \mathcal{F}, P \rangle$, a **random variable** X is a function $X : \Omega \to D$ that maps each sample point $\omega \in \Omega$ to a numerical value in a set $D \subseteq \mathbb{R}$. We call D the **domain** of X, denoted by $\text{dom}(X)$.

Example 2.7. In a coin flip experiment where $\Omega = \{\text{Head}, \text{Tail}\}$, we can define a random variable X such that:

- $X(\text{Head}) = 1$
- $X(\text{Tail}) = 0$

In this case, $\text{dom}(X) = \{0, 1\}$.

Def 2.7. Probability of a Random Variable: For a random variable X and a specific value $x \in \text{dom}(X)$, the probability that X takes the value x is defined as the measure of the event consisting of all outcomes ω that map to x:

$$P(X = x) := P\left(\{\omega \in \Omega \mid X(\omega) = x\}\right)$$

Example 2.8. Consider rolling a fair die where $\Omega = \{1, 2, \ldots, 6\}$. If I define a random variable Y as the square of the outcome, then the probability of the event $Y = 4$ is:

$$P(Y = 4) = P\left(\{\omega \in \Omega \mid \omega^2 = 4\}\right) = P\left(\{2\}\right) = \frac{1}{6}$$

Def 2.8. Probability of Negation: For a random variable X and $x \in \text{dom}(X)$, we define the probability of the logical negation as:

$$P(X \neq x) := P(\{\omega \in \Omega \mid X(\omega) \neq x\}) = 1 - P(X = x)$$

Def 2.9. Probability of Disjunction: For random variables X_1, X_2, we define the probability of the logical disjunction ("or") as:

$$P(X_1 = x_1 \vee X_2 = x_2) := P(\{\omega \in \Omega \mid X_1(\omega) = x_1\} \cup \{\omega \in \Omega \mid X_2(\omega) = x_2\})$$

Def 2.10. Joint Probability: Given two random variables X and Y defined on the same sample space Ω, the **joint probability** of $x \in \text{dom}(X)$ and $y \in \text{dom}(Y)$ is defined as the probability of the intersection of their individual events:

$$P(X = x, Y = y) := P(\{\omega \in \Omega \mid X(\omega) = x\} \cap \{\omega \in \Omega \mid Y(\omega) = y\})$$

Example 2.9. In a roll of two dice where X is the first die and Y is the second die: The joint probability $P(X = 3, Y = 4)$ is the probability of the sample point $(3, 4) \in \Omega$:

$$P(X = 3, Y = 4) := P(\{(3, 4)\}) := \frac{1}{36}$$

Example 2.10. To find the probability that the sum of two dice is 7 but neither is 3, we define the following random variables on the sample space $\Omega = \{1, \ldots, 6\}^2$:

- X_1, X_2: outcomes of the first and second die respectively.
- S: the sum, where $S(\omega) = X_1(\omega) + X_2(\omega)$.

We are interested in the event E:

$$E = \{\omega \in \Omega \mid S(\omega) = 7\} \cap \{\omega \in \Omega \mid X_1(\omega) \neq 3\} \cap \{\omega \in \Omega \mid X_2(\omega) \neq 3\}$$

Step 1: Identify outcomes where $S = 7$:

$$A = \{(1, 6), (2, 5), (3, 4), (4, 3), (5, 2), (6, 1)\}$$

Step 2: Apply the constraints $X_1 \neq 3$ and $X_2 \neq 3$:

- $\omega \in A$ and $X_1(\omega) = 3 \implies \omega = (3, 4)$
- $\omega \in A$ and $X_2(\omega) = 3 \implies \omega = (4, 3)$

Step 3: Calculate the cardinality of the filtered set:

$$E = A \setminus \{(3, 4), (4, 3)\} = \{(1, 6), (2, 5), (5, 2), (6, 1)\} \implies \Gamma(E) = 4$$

Step 4: Use the probability measure:

$$P(E) = \frac{\Gamma(E)}{\Gamma(\Omega)} = \frac{4}{36} = \frac{1}{9}$$

Def 2.11. Marginalization: For two random variables X and Y, the **marginal probability** of X is calculated by summing the joint probabilities over all $y \in \text{dom}(Y)$:

$$P(X = x) := \sum_{y \in \text{dom}(Y)} P(X = x, Y = y)$$

This is a direct application of the axiom of countable additivity to the partition of the event $\{X = x\}$ into smaller, mutually exclusive joint events.

Example 2.11. Suppose I have a joint probability table for two random variables X and Y representing a simplified weather/activity model:

	$Y=0$ (Stay)	$Y=1$ (Walk)	**Total**
$X=0$ (Rain)	0.3	0.1	$P(X=0)=\mathbf{0.4}$
$X=1$ (Sun)	0.1	0.5	$P(X=1)=\mathbf{0.6}$

Table 2: Marginalization Example

To find $P(X=0)$, I marginalize over Y:

$$P(X=0)=P(X=0,Y=0)+P(X=0,Y=1)=0.3+0.1=0.4$$

Def 2.12. Conditional Probability: Let A and B be events in a probability space where $P(B)>0$. The **conditional probability** of A given B is defined as:

$$P(A\mid B):=\frac{P(A\cap B)}{P(B)}$$

In this context, we refer to:

- $P(A)$ as the **prior** probability.
- $P(B)$ as the **evidence**.
- $P(A\mid B)$ as the **posterior** probability.

Example 2.12. Suppose I roll a fair 6-sided die, so $\Omega=\{1,2,3,4,5,6\}$. Let A be the event that the outcome is 2, and B be the event that the outcome is even.

- $P(A)=P(\{2\})=1/6$
- $P(B)=P(\{2,4,6\})=3/6=1/2$
- $P(A\cap B)=P(\{2\})=1/6$

The probability that I roll a 2 **given** I know the result is even is:

$$P(A\mid B)=\frac{P(A\cap B)}{P(B)}=\frac{1/6}{1/2}=\frac{1}{3}$$

Theorem 2.1. $P(A\mid C)=1-P(\bar{A}\mid C)$

Proof.

1. Partition: $C=(A\cap C)\cup(\bar{A}\cap C)$.
2. Note that $(A\cap C)$ and $(\bar{A}\cap C)$ are disjoint because $A\cap\bar{A}=\emptyset$.
3. Apply the Axiom of Additivity: $P(C)=P(A\cap C)+P(\bar{A}\cap C)$.
4. Rearrange: $P(\bar{A}\cap C)=P(C)-P(A\cap C)$.
5. Conditional probability:
$$P(\bar{A}\mid C)=\frac{P(\bar{A}\cap C)}{P(C)}$$
6. Substitute:
$$P(\bar{A}\mid C)=\frac{P(C)-P(A\cap C)}{P(C)}$$
7. Simplify:
$$P(\bar{A}\mid C)=\frac{P(C)}{P(C)}-\frac{P(A\cap C)}{P(C)}=1-P(A\mid C)$$
8. Rearrange the terms:
$$P(A\mid C)=1-P(\bar{A}\mid C)$$

□

Theorem 2.2. $P(A \mid C) = P(A \cap B \mid C) + P(A \cap \bar{B} \mid C)$

Proof.

1. Partition: $(A \cap C) = (A \cap C \cap B) \cup (A \cap C \cap \bar{B})$
2. Note that $(A \cap C \cap B)$ and $(A \cap C \cap \bar{B})$ are disjoint because $B \cap \bar{B} = \emptyset$.
3. Axiom of Additivity: $P(A \cap C) = P(A \cap B \cap C) + P(A \cap \bar{B} \cap C)$
4. Conditional probability:
$$P(A \mid C) = \frac{P(A \cap C)}{P(C)}$$
5. Substitute:
$$P(A \mid C) = \frac{P(A \cap B \cap C) + P(A \cap \bar{B} \cap C)}{P(C)}$$
6. Distribute:
$$P(A \mid C) = \frac{P(A \cap B \cap C)}{P(C)} + \frac{P(A \cap \bar{B} \cap C)}{P(C)}$$
7. Finally:
$$P(A \mid C) = P(A \cap B \mid C) + P(A \cap \bar{B} \mid C)$$

□

Theorem 2.3. $P(A \cup B \mid C) = P(A \mid C) + P(B \mid C) - P(A \cap B \mid C)$

Proof.

1. Conditional probability
$$P(A \cup B \mid C) = \frac{P((A \cup B) \cap C)}{P(C)}$$
2. Distribute: $(A \cup B) \cap C = (A \cap C) \cup (B \cap C)$
3. Inclusion-Exclusion:
$$P((A \cap C) \cup (B \cap C)) = P(A \cap C) + P(B \cap C) - P((A \cap C) \cap (B \cap C))$$
4. Simplify: $(A \cap C) \cap (B \cap C) = A \cap B \cap C$
5. Substitute:
$$P(A \cup B \mid C) = \frac{P(A \cap C) + P(B \cap C) - P(A \cap B \cap C)}{P(C)}$$
6. Simplify:
$$P(A \cup B \mid C) = \frac{P(A \cap C)}{P(C)} + \frac{P(B \cap C)}{P(C)} - \frac{P(A \cap B \cap C)}{P(C)}$$
7. Finally:
$$P(A \cup B \mid C) = P(A \mid C) + P(B \mid C) - P(A \cap B \mid C)$$

□

Theorem 2.4.
$$P(A \mid B) = \frac{P(B \mid A) \cdot P(A)}{P(B)}$$ **(Bayes' Theorem)**

Proof.

1. conditional probability:
$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}$$

2. Multiply both sides by $P(B)$ to express the joint probability:

$$P(A \cap B) = P(A \mid B) \cdot P(B)$$

3. conditional probability:

$$P(B \mid A) = \frac{P(B \cap A)}{P(A)}$$

4. Multiply both sides by $P(A)$ to get a second expression for the joint probability:

$$P(B \cap A) = P(B \mid A) \cdot P(A)$$

5. intersection is commutative $\rightsquigarrow P(A \cap B) = P(B \cap A)$
6. Equate the right-hand sides of the expressions from Step 2 and Step 4:

$$P(A \mid B) \cdot P(B) = P(B \mid A) \cdot P(A)$$

7. Divide both sides by $P(B)$ (assuming $P(B) > 0$) to isolate the **posterior** probability:

$$P(A \mid B) = \frac{P(B \mid A) \cdot P(A)}{P(B)}$$

□

Def 2.13. Bayesian Terminology: Within the context of Bayes' Theorem, we define the following nomenclature:

- **Posterior:** $P(A \mid B)$ is the probability of the hypothesis A after observing the evidence B.
- **Likelihood:** $P(B \mid A)$ is the probability that the evidence B would be observed if the hypothesis A were true.
- **Prior:** $P(A)$ is the probability of the hypothesis A before the evidence B is taken into account.
- **Evidence:** $P(B)$ is the total probability of observing the evidence under all possible hypotheses.

Def 2.14. The Product Rule: For any two events A and B where the conditional probabilities are well-defined ($P(A) > 0, P(B) > 0$), the probability of their joint occurrence is:

$$P(A \cap B) = P(A \mid B) \cdot P(B)$$
$$P(A \cap B) = P(B \mid A) \cdot P(A)$$

This rule establishes that the probability of "A and B" is the probability of one event, scaled by the probability of the other event occurring given that the first has occurred.

Example 2.13. Suppose I draw two cards from a deck without replacement. Let A be the event the first card is an Ace, and B be the event the second card is an Ace.

- $P(A) = 4/52$
- $P(B \mid A) = 3/51$ (since one Ace is gone)

The probability of drawing two Aces is:

$$P(A \cap B) = P(B \mid A) \cdot P(A) = \frac{3}{51} \cdot \frac{4}{52}$$

Def 2.15. Independence: Two events A and B are **independent** iff $P(A \cap B) = P(A) \cdot P(B)$

Theorem 2.5. If A and B are independent, then the posterior equals the prior:

$$P(A \mid B) = P(A) \quad \text{and} \quad P(B \mid A) = P(B)$$

Proof.

1. Conditional probability:
$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}$$

2. Independence:
$$P(A \mid B) = \frac{P(A) \cdot P(B)}{P(B)} = P(A)$$

3. The proof for $P(B \mid A) = P(B)$ follows identical logic by symmetry.

□

Def 2.16. Pairwise Independence: A collection of events $A_1, A_2, \ldots, A_n$ is **pairwise independent** iff every pair of distinct events is independent. That is, for all $i \neq j$:
$$P(A_i \cap A_j) = P(A_i) \cdot P(A_j)$$

Def 2.17. Mutual Independence: A collection of events $A_1, A_2, \ldots, A_n$ is **mutually independent** iff for every finite subset of indices $I \subseteq \{1, 2, \ldots, n\}$, the probability of their intersection is the product of their individual probabilities:
$$P\left(\bigcap_{i \in I} A_i\right) = \prod_{i \in I} P(A_i)$$

> Note
>
> Mutual independence strictly implies pairwise independence, but pairwise independence does not guarantee mutual independence.

Def 2.18. Conditional Independence: Given events A, B, C with $P(C) \neq 0$, then A and B are **conditionally independent** given C iff $P(A \mid B \cap C) := P(A \mid C)$, or $P(B \mid A \cap C) := P(B \mid C)$:

Theorem 2.6. If A and B are conditionally independent given C, then:
$$P(A \cap B \mid C) := P(A \mid C) \cdot P(B \mid C)$$

Proof.

1. By definition:
$$P(A \cap B \mid C) = \frac{P(A \cap B \cap C)}{P(C)}$$

2. By the product rule:
$$P(A \cap B \cap C) = P(A \mid B \cap C) \cdot P(B \cap C)$$

3. Expanding $P(B \cap C)$ further:
$$P(A \cap B \cap C) = P(A \mid B \cap C) \cdot P(B \mid C) \cdot P(C)$$

4. Substituting Step 3 into Step 1:
$$P(A \cap B \mid C) = \frac{P(A \mid B \cap C) \cdot P(B \mid C) \cdot P(C)}{P(C)}$$

5. Simplifying:
$$P(A \cap B \mid C) = P(A \mid B \cap C) \cdot P(B \mid C)$$

6. Since A and B are conditionally independent given C, $P(A \mid B \cap C) = P(A \mid C)$, yielding:
$$P(A \cap B \mid C) = P(A \mid C) \cdot P(B \mid C)$$

□

Def 2.19. Probability Distribution: A **probability distribution** is a vector v with values in $[0, 1]$ such that $\sum_i v_i = 1$.

> Note
>
> The **probability distribution** of a random variable X (written as $\mathbf{P}(X)$) is a complete description of the probabilities $P(X = x)$ for all values $x \in \text{dom}(X)$. It completely characterizes the behavior of the random variable by mapping each possible outcome to its corresponding probability, effectively describing how the total probability of 1 (from the Normalization axiom) is distributed across the domain of X.

Def 2.20. Full Joint Probability Distribution (JPD): Given a set of random variables $X_1, X_2, \ldots, X_n$, a **full joint probability distribution** $\mathbf{P}(X_1, X_2, \ldots, X_n)$ is a probability distribution that assigns a probability to every possible, mutually exclusive joint probability assignment for all variables. It can be represented as an n-dimensional tensor where each entry $P(X_1 = x_1, X_2 = x_2, \ldots, X_n = x_n) \in [0, 1]$ corresponds to a single atomic event, and the sum over all possible combinations of values is 1:

$$\sum_{x_1 \in \text{dom}(X_1)} \cdots \sum_{x_n \in \text{dom}(X_n)} P(X_1 = x_1, \ldots, X_n = x_n) = 1$$

Example 2.14. Consider two boolean random variables: W (Weather: Sunny=1, Rainy=0) and C (Commute: Walk=1, Drive=0). The full joint probability distribution is given by the table:

	$C = \text{Walk}$	$C = \text{Drive}$	**Total**
$W = \text{Sunny}$	0.6	0.1	0.7
$W = \text{Rainy}$	0.05	0.25	0.3
Total	0.65	0.35	**1.0**

Note that the sum of all internal cells $(0.6 + 0.1 + 0.05 + 0.25 = 1.0)$ satisfies the normalization requirement.

Def 2.21. Conditional Probability Distribution (CPD): Given random variables X and Y, the **conditional probability distribution** $\mathbf{P}(X \mid Y)$ represents a complete table or matrix of probabilities, providing the distribution of X for every possible observed value $y \in \text{dom}(Y)$. It consists of the conditional probabilities $P(X = x \mid Y = y)$ for all $x \in \text{dom}(X)$ and all $y \in \text{dom}(Y)$. For any fixed y, the distribution over X satisfies the normalization constraint:

$$\sum_{x \in \text{dom}(X)} P(X = x \mid Y = y) = 1$$

Example 2.15. Continuing from the previous example, we can form the conditional probability distribution $\mathbf{P}(C \mid W)$ by dividing each joint probability by the marginal probability of W. Note that for any fixed value of W (each column), the probabilities sum to 1:

$\mathbf{P}(C \mid W)$	$W = \text{Sunny}$	$W = \text{Rainy}$
$C = \text{Walk}$	$0.6/0.7 = 6/7$	$0.05/0.3 = 1/6$
$C = \text{Drive}$	$0.1/0.7 = 1/7$	$0.25/0.3 = 5/6$
Total	**1.0**	**1.0**

Def 2.22. A **Naive Bayes Model** (Bayesian Classifier) consists of:

- Random variables $E_1 \cdots E_n$ (effects)
- Random variable C (the cause)
- The probability distribution $\mathbf{P}(C)$
- The CPD $\mathbf{P}(E_i \mid C)$

Where all $E_1, \cdots E_n$ are conditionally independent given C.

Lemma 2.7. Chain Rule:

$$\mathbf{P}(X_1, \cdots, X_n) = \mathbf{P}(X_1 \mid X_2, \cdots X_n) \cdot \mathbf{P}(X_2 \mid X_3 \cdots X_n) \cdots \mathbf{P}(X_{n-1} \mid X_n) \cdot \mathbf{P}(X_n)$$

Proof. via generalization of the product rule for e.g. $X_1, \cdots X_4$:

- $\mathbf{P}(X_1, X_2, X_3, X_4) = \mathbf{P}(X_1 \mid X_2, X_3, X_4) \cdot \mathbf{P}(X_2, X_3, X_4)$
- $\mathbf{P}(X_2, X_3, X_4) = \mathbf{P}(X_2 \mid X_3, X_4) \cdot \mathbf{P}(X_3, X_4)$
- $\mathbf{P}(X_3, X_4) = \mathbf{P}(X_3 \mid X_4) \cdot \mathbf{P}(X_4)$
- Hence, $\mathbf{P}(X_1, X_2, X_3, X_4) = \mathbf{P}(X_1 \mid X_2, X_3, X_4) \cdot \mathbf{P}(X_2 \mid X_3, X_4) \cdot \mathbf{P}(X_3 \mid X_4) \cdot \mathbf{P}(X_4)$

□

Question

Given a Naive Bayes Model, can we compute the Full JPD $\mathbf{P}(C, E_1, \cdots, E_n)$ using only $\mathbf{P}(C)$ and $\mathbf{P}(E_i \mid C)$?

Answer

Yes. Because of the conditional independence assumption in a Naive Bayes Model, the JPD simplifies to:

$$\mathbf{P}(C, E_1, \ldots, E_n) = \mathbf{P}(C) \prod_{i=1}^{n} \mathbf{P}(E_i \mid C)$$

Proof.

- Chain rule: $\mathbf{P}(E_1, \cdots, E_n, C) = \mathbf{P}(E_1 \mid E_2, \cdots E_n, C) \cdots \mathbf{P}(E_n \mid C) \cdot \mathbf{P}(C)$
- All $E_1, \cdots E_n$ are conditionally independent given C
- $\mathbf{P}(E_1 \mid E_2, \cdots E_n, C) = \mathbf{P}(E_1 \mid C)$ and $\mathbf{P}(E_2 \mid E_3, \cdots E_n, C) = \mathbf{P}(E_2 \mid C)$ and so on $\cdots$
- Hence, $\mathbf{P}(C, E_1, \ldots, E_n) = \mathbf{P}(C) \prod_{i=1}^{n} \mathbf{P}(E_i \mid C)$

□

Def 2.23. Marginalization Generalized: Given random variables $X_1, \cdots X_n$ and $Y_1, \cdots, Y_m$, we have

$$\mathbf{P}(X_1, \cdots, X_n) = \sum_{y_1 \in \text{dom}(Y_1), \cdots, y_m \in \text{dom}(Y_m)} \mathbf{P}(X_1, \cdots, X_n, Y_1 = y_1, \cdots Y_m = y_m)$$

Def 2.24. Given a Naive Bayes Model with a set of effects $E = \{E_1, \cdots, E_n\}$, we refer to the **observable** effect as $O = \{O_1, \cdots, O_{no}\}$ and the set of **unobservable** (unknown) effects as $U = \{U_1, \cdots, U_{nu}\}$ where $E = O \cup U$

Notation Understanding

Assume we have two unknown effects ($nu = 2$): *Headache* (U_1) where $D_H = \{\text{yes}, \text{no}\}$ and *Fever* (U_2) where $D_F = \{\text{high}, \text{low}\}$
We define $D_u = \text{dom}(U_1) \times \cdots \times \text{dom}(U_{nu}) = \{(y,h),(y,l),(n,h),(n,l)\}$
We can sum all joint probabilities of the unknown effects as :

$$\sum_{u \in D_u} P(U_1 = u_1, \cdots, U_{nu} = u_{nu}) = 1$$

- For $u = (y,h)$, we have $P(U_1 = y, U_2 = h)$
- For $u = (y,l)$, we have $P(U_1 = y, U_2 = l)$
- For $u = (n,h)$, we have $P(U_1 = n, U_2 = h)$
- For $u = (n,l)$, we have $P(U_1 = n, U_2 = l)$

For a specific tuple $u = \langle u_1, u_2 \rangle \in D_u$, assuming *Headache* and *Fever* are independent, the joint probability of $P(H = u_1, F = u_2)$ can be also computed from the product:

$$\prod_{j=1}^{nu} P(U_j = u_{u_j}) \qquad \text{where } u_{u_j} \text{ means the value little u at index j given the tuple u}$$

Let's take the tuple $\langle n, h \rangle$ and we are iterating over $U = \{H(U_1), F(U_2)\}$

$$\prod_{j=1}^{nu} P(U_j = u_{u_j}) = P(H = n) \cdot P(F = h)$$

If we want to sum all the joint probabilities for all values, we write:

$$\sum_{u \in D_u} \prod_{j=1}^{nu} P(U_j = u_{u_j})$$

Which will evaluate to 1 too (assuming independence).

Let $D_u = \text{dom}(U_1) \times \cdots \times \text{dom}(U_{nu})$, then:

$$\mathbf{P}(C \mid O_1, \cdots, O_{no}) = \frac{\mathbf{P}(C, O_1, \cdots, O_{no})}{\mathbf{P}(O_1, \cdots, O_{no})}$$

We marginalize over the unknowns to compute the numerator:

$$\mathbf{P}(C, O_1, \cdots, O_{no}) = \sum_{u \in D_u} \mathbf{P}(C, O_1, \cdots, O_{no}, U_1 = u_1, \cdots, U_{nu} = u_{nu})$$

$$\mathbf{P}(C, O_1, \cdots, O_{no}) = \sum_{u \in D_u} \mathbf{P}(C) \prod_{i=1}^{no} \mathbf{P}(O_i \mid C) \prod_{j=1}^{nu} \mathbf{P}(U_j = u_{u_j} \mid C)$$

$$\mathbf{P}(C, O_1, \cdots, O_{no}) = \mathbf{P}(C) \prod_{i=1}^{no} \mathbf{P}(O_i \mid C) \sum_{u \in D_u} \prod_{j=1}^{nu} \mathbf{P}(U_j = u_{u_j} \mid C)$$

Also, for the denominator, we have:

$$\mathbf{P}(O_1, \cdots, O_{no}) = \sum_{c \in \text{dom}(C)} \sum_{u \in D_u} \mathbf{P}(O_1, \cdots, O_{no}, C = c, U_1 = u_1, \cdots, U_{nu} = u_{nu})$$

$$\mathbf{P}(O_1, \cdots, O_{no}) = \sum_{c \in \text{dom}(C)} \sum_{u \in D_u} P(C = c) \prod_{i=1}^{no} \mathbf{P}(O_i \mid C = c) \prod_{j=1}^{nu} P(U_j = u_{u_j} \mid C = c)$$

$$\mathbf{P}(O_1, \cdots, O_{no}) = \sum_{c \in \text{dom}(C)} P(C = c) \prod_{i=1}^{no} \mathbf{P}(O_i \mid C = c) \sum_{u \in D_u} \prod_{j=1}^{nu} P(U_j = u_{u_j} \mid C = c)$$

Therefore,

$$\mathbf{P}(C \mid O_1, \cdots, O_{no}) = \frac{\mathbf{P}(C)\prod_{i=1}^{no}\mathbf{P}(O_i \mid C)\sum_{u\in D_u}\prod_{j=1}^{nu}\mathbf{P}(U_j = u_{u_j} \mid C)}{\sum_{c\in \text{dom}(C)} P(C=c)\prod_{i=1}^{no}\mathbf{P}(O_i \mid C=c)\sum_{u\in D_u}\prod_{j=1}^{nu}P(U_j = u_{u_j} \mid C=c)}$$

Note

$\sum_{u\in D_u}\prod_{j=1}^{nu}P(U_j = u_{u_j} \mid C=c) = 1$

Hence,

$$\mathbf{P}(C \mid O_1, \cdots, O_{no}) = \frac{\mathbf{P}(C)\prod_{i=1}^{no}\mathbf{P}(O_i \mid C)}{\sum_{c\in \text{dom}(C)} P(C=c)\prod_{i=1}^{no}\mathbf{P}(O_i \mid C=c)}$$

Note

The denominator is
- the same for any given observations $O_1, \cdots, O_{no}$ independent of the value of C
- the sum over all numerators

The denominator acts as a **normalizing constant**. It scales the product in the numerator so that the resulting values for $\mathbf{P}(C \mid O_1, \ldots, O_{no})$ sum to 1. This process is called **Normalization**.

Def 2.25. Normalization: Given a vector $w := \langle w_1, \ldots, w_k\rangle$ where each entry $w_i \in [0,1]$ and $\sum_{i=1}^{k} w_i \leq 1$, we define a function $\alpha(w)$ that normalizes the vector by dividing each entry by the sum of all its components.

$$(\alpha(w))_i = \frac{w_i}{\sum_{j=1}^{k} w_j}$$

By construction, $\sum_{i=1}^{k}(\alpha(w))_i = 1$, which ensures that $\alpha(w)$ is a valid probability distribution.

Inference Theorem in a Naive Bayes Model

$$\mathbf{P}(C \mid O_1, \cdots, O_{no}) = \alpha\left(\mathbf{P}(C)\prod_{i=1}^{no}\mathbf{P}(O_i \mid C)\right)$$

Example 2.16. Consider we are modeling *cavity* (C) and *toothache* (T) with Boolean domains $\{T, F\}$ and we know the following probabilities:

- $P(C=T) = 0.1 \quad P(T=T \mid C=T) = 0.8 \quad P(T=T \mid C=F) = 0.05$

What is the probability distribution for a patient having *cavity* given they have *toothache*?

$$\begin{aligned}
\mathbf{P}(C \mid t=T) &:= \alpha(\mathbf{P}(C)\cdot\mathbf{P}(T=T \mid C)) \\
\mathbf{P}(C) &:= \langle P(C=T), P(C=F)\rangle \\
\mathbf{P}(T=T \mid C) &:= \langle P(T=T \mid C=T), P(T=T \mid C=F)\rangle \\
\mathbf{P}(C \mid t=T) &:= \alpha(\langle P(C=T), P(C=F)\rangle \cdot \langle P(T=T \mid C=T), P(T=T \mid C=F)\rangle) \\
&:= \alpha\left(\langle P(C=T)\cdot P(T=T \mid C=T), P(C=F)\cdot P(T=T \mid C=F)\rangle\right) \\
&:= \alpha\left(\langle 0.1\cdot 0.8, 1 - P(C=T)\cdot 0.05\rangle\right) \\
&:= \alpha\left(\langle 0.1\cdot 0.8, 0.9\cdot 0.05\rangle\right) \\
&:= \alpha\left(\langle 0.08, 0.045\rangle\right) \\
&:= \langle 0.64, 0.36\rangle
\end{aligned}$$

Theorem 2.8. Let $Q, E_1, \cdots, E_{ne}, U_1, \cdots, U_{nu}$ be random variables and $D_u = \text{dom}(U_1)\times\cdots\times\text{dom}(U_{nu})$. Then:

$$\mathbf{P}(Q \mid E_1 = e_1, \cdots E_{ne} = e_{ne}) = \alpha\left(\sum_{u\in D_u}\mathbf{P}(Q, E_1 = e_1, \cdots E_{ne} = e_{ne}, U_1 = u_1, \cdots, U_{nu} = u_{n_u})\right)$$

We call Q the **query variable**, $E_1, \cdots, E_{ne}$ the **evidence**, and $U_1, \cdots, U_{nu}$ the **unknown** (or **hidden**) **variables**, and computing a conditional probability this way **enumeration**

Note

General Law of Inference by Enumeration: "To find out what you care about (Q), you must look at every possible world (U) that could exist alongside what you already saw (E)." concretely:

- sum over all relevant entries of the JPD of the variables, and
- normalize the results to yield a probability distribution

3 Bayesian Networks

Def 3.1. A **Bayesian Network** consists of

1. a DAG $\langle \mathcal{X}, E \rangle$ of random variables $\mathcal{X} = \{X_1, \cdots, X_n\}$, and
2. a CPD $\mathbf{P}(X_i \mid \text{Parents}(X_i))$ for every $X_i \in \mathcal{X}$,

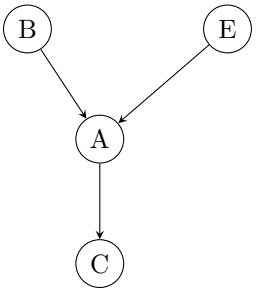
such that every X_i is conditionally independent of any conjunctions of non-descendants of X_i given Parents(X_i)

Note

Roots of the DAG are conditionally independent given the empty set $\emptyset$; i.e. they are independent. This follows directly from the definition:

- By definition, a root node X_i has no parents, meaning Parents$(X_i) = \emptyset$.
- Any other root node X_j is a non-descendant of X_i (since there are no directed paths between roots).
- By the definition of the Bayesian Network, X_i is conditionally independent of its non-descendant X_j given its parents ($\emptyset$).
- Conditional independence given the empty set is exactly equivalent to marginal independence.

Example 3.1. Consider a simple Bayesian Network modeling a security system with variables: B (Burglar), E (Earthquake), A (Alarm rings), and C (Call Police).



- Variables $\mathcal{X} = \{B, E, A, C\}$
- Edges $E = \{(B, A), (E, A), (A, C)\}$
- CPDs: $\mathbf{P}(B)$, $\mathbf{P}(E)$, $\mathbf{P}(A \mid B, E)$, and $\mathbf{P}(C \mid A)$

By definition, each node is conditionally independent of its non-descendants given its parents. Let's analyze each node:

- **Node B:** Parents$(B) = \emptyset$, NonDesc$(B) = \{E\}$.
  Property: B is independent of E given $\emptyset$ $\implies \mathbf{P}(B \mid E) = \mathbf{P}(B)$.
- **Node E:** Parents$(E) = \emptyset$, NonDesc$(E) = \{B\}$.
  Property: E is independent of B given $\emptyset$ $\implies \mathbf{P}(E \mid B) = \mathbf{P}(E)$.
- **Node A:** Parents$(A) = \{B, E\}$, NonDesc$(A) = \emptyset$.
  Property: Vacuously true (no non-descendants exists to be independent from).
- **Node C:** Parents$(C) = \{A\}$, NonDesc$(C) = \{B, E\}$.
  Property: C is independent of $\{B, E\}$ given A $\implies \mathbf{P}(C \mid A, B, E) = \mathbf{P}(C \mid A)$.
  *(Intuition: Once we know the alarm rang, knowing exactly **why** it rang doesn't give us any extra information about whether the police will be called).*

Theorem 3.1. The JPD of a Bayesian Network $\langle \mathcal{X}, E \rangle$ is given by:

$$\mathbf{P}(X_1, \cdots, X_n) = \prod_{X_i \in \mathcal{X}} \mathbf{P}(X_i \mid \text{Parents}(X_i))$$

Proof.

1. Let $X_1, X_2, \ldots, X_n$ be a **topological sort** (an ordering) of the variables in the DAG. This ordering guarantees that if $X_i \to X_j$ is an edge (meaning X_i is a parent of X_j), then $i < j$.
2. Because joint probability is commutative, we can examine the variables in the JPD as $\mathbf{P}(X_n, \ldots, X_1)$. Applying the Chain Rule, we get:

$$\begin{aligned}\mathbf{P}(X_n, \ldots, X_1) &= \mathbf{P}(X_n \mid X_{n-1}, \ldots, X_1) \cdot \mathbf{P}(X_{n-1} \mid X_{n-2}, \ldots, X_1) \cdots \mathbf{P}(X_2 \mid X_1) \cdot \mathbf{P}(X_1) \\ &= \prod_{i=1}^{n} \mathbf{P}(X_i \mid X_{i-1}, \ldots, X_1)\end{aligned}$$

3. Because we used a topological sort, all variables $X_{i-1}, \ldots, X_1$ appear before X_i in the ordering. This guarantees they must be non-descendants of X_i (since descendants can only appear *after* X_i).
4. Thus, the conditioning set $\{X_{i-1}, \ldots, X_1\}$ consists exactly of Parents(X_i) plus some other non-descendants of X_i.
5. By the definition of Bayesian Networks, X_i is conditionally independent of its non-descendants given its parents. Therefore, the extra non-descendants drop out of the probability:

$$\mathbf{P}(X_i \mid X_{i-1}, \ldots, X_1) = \mathbf{P}(X_i \mid \text{Parents}(X_i))$$

6. Substituting this back into the Chain Rule from Step 2 yields:

$$\mathbf{P}(X_n, \ldots, X_1) = \prod_{i=1}^{n} \mathbf{P}(X_i \mid \text{Parents}(X_i))$$

By commutativity of the joint probability, this is equivalent to $\mathbf{P}(X_1, \ldots, X_n)$. Validating the theorem.

□

Def 3.2. Given random variables $X_1, \cdots X_n$ with finite domains $D_1, \cdots, D_n$, the **size** of the Bayesian Network $\mathcal{B} = \langle \{X_1, \cdots, X_n\}, E \rangle$ is defined as:

$$\text{size}(\mathcal{B}) := \sum_{i=1}^{n} |D_i| \cdot \left(\prod_{X_j \in \text{Parents}(X_i)} |D_j| \right)$$

Note

- size$(\mathcal{B})\hat{=}$ The total number of entries in the CPD.
- Smaller Bayesian Networks need to estimate less probabilities $\rightsquigarrow$ more efficient inference.
- Explicit Full JPD has size $\prod_{i=1}^{n} |D_i|$
- If $|\text{Parents}(X_i)| \leq k$ for every X_i, and $D_{\max}$ is the largest random variable domain, then $\text{size}(\mathcal{B}) \leq n\,|D_{\max}|^{k+1}$
- In the worst case, $\text{size}(\mathcal{B}) = \sum_{i=1}^{n} \prod_{j=1}^{i} |D_j|$, namely if every variable depends on all its predecessors in the chosen variable ordering.
- Bayesian Networks are compact if each variable is directly influenced only by few of its predecessor variables.

Example 3.2. Consider a Bayesian Network $\mathcal{B}$ with 4 boolean random variables (meaning their domain sizes are $|D_i| = 2$ for all i) arranged in a simple chain: $X_1 \to X_2 \to X_3 \to X_4$.

We execute the formula term by term:

- **Node** X_1: No parents (Parents$(X_1) = \emptyset \implies$ independent product is 1).
  Term expands to: $|D_1| \cdot 1 = 2 \cdot 1 = 2$
- **Node** X_2: One parent (Parents$(X_2) = \{X_1\}$).
  Term expands to: $|D_2| \cdot |D_1| = 2 \cdot 2 = 4$
- **Node** X_3: One parent (Parents$(X_3) = \{X_2\}$).
  Term expands to: $|D_3| \cdot |D_2| = 2 \cdot 2 = 4$
- **Node** X_4: One parent (Parents$(X_4) = \{X_3\}$).
  Term expands to: $|D_4| \cdot |D_3| = 2 \cdot 2 = 4$

Summing these up gives the total size of the probability tables to be stored:

$$\text{size}(\mathcal{B}) = 2 + 4 + 4 + 4 = 14 \text{ probability entries}$$

We can verify the efficiency by comparing it to an explicit Full JPD:

$$\text{Full JPD Size} = \prod_{i=1}^{4} |D_i| = 2 \cdot 2 \cdot 2 \cdot 2 = 16 \text{ entries}$$

Even on just 4 variables, the conditional independencies in the Bayesian Network make it tangibly more compact.

Upper Bound Analysis

In our chain, no node has more than 1 parent, so the maximum in-degree is $k = 1$. The largest domain is $D_{\max} = 2$ (because all variables are boolean). For our $n = 4$ nodes, the sparsity upper bound formula yields:

$$\text{size}(\mathcal{B}) \leq n \cdot |D_{\max}|^{k+1} = 4 \cdot 2^{1+1} = 4 \cdot 4 = 16$$

Our actual calculated size of 14 is indeed strictly bounded by 16.
To see the real power of this bound, imagine extending this exact chain to $n = 100$ boolean variables. The Full JPD would require 2^{100} entries (a computational impossibility). In contrast, our Bayesian Network upper bound guarantees the memory footprint is simply $\leq 100 \cdot 2^2 = 400$ entries. This demonstrates why bounding the maximum number of parents k is critical for scalability!

Question

How to keep our Bayesian Network small?

Answer

- Reduce the number of edges $\rightsquigarrow$ Order the variables to allow for exploiting conditional independence (causes before effects), or
- Represent the CPD efficiently:
  - For a Boolean random variable X, we only need to store $P(X = T \mid \text{Parents}(X))$
  - Introduce different kinds of nodes exploiting domain knowledge.

Def 3.3. BN Construction Algorithm:

1. Initialize $BN = \langle \{X_1, \cdots, X_n\}, E \rangle$ where $E = \emptyset$
2. Fix any variable ordering, $X_1, \cdots, X_n$
3. **for** $i := 1, \cdots, n$ **do**:
   (a) Choose a minimal set $\text{Parents}(X_i) \subseteq \{X_1, \cdots, X_{i-1}\}$ such that:
   $$\mathbf{P}(X_i \mid X_{i-1}, \cdots, X_1) = \mathbf{P}(X_i \mid \text{Parents}(X_i))$$
   (b) For each $X_j \in \text{Parents}(X_i)$, insert $\langle X_j, X_i \rangle$ into E
   (c) Associate X_i with $\mathbf{P}(X_i \mid \text{Parents}(X_i))$

Note

The size of the resulting Bayesian Network depends on the chosen **variable ordering**. The size of a Bayesian Network is not a fixed property (it depends on the skill of the designer)

Rule of Thumb

Order *causes* before *effects*

Def 3.4. A node X in a Bayesian Network is called **deterministic**, if its value is completely determined by the values of $\text{Parents}(X)$

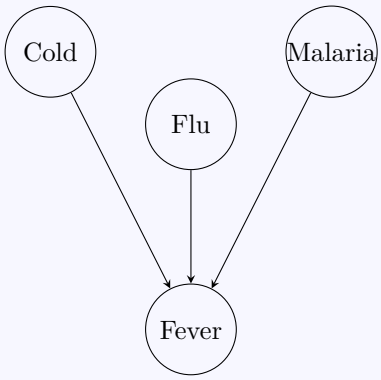
Example 3.3. The sum of two dice throws S is entirely determined by the values of the two dice `First` and `Second`

Note

- Deterministic nodes model direct, causal relationships.
- If X is deterministic, then $P(X \mid \text{Parents}(X)) \in \{0, 1\}$
- We can replace the CPD $\mathbf{P}(X \mid \text{Parents}(X))$ by a boolean function

Noisy Nodes

Sometimes, values of nodes are *almost* deterministic. For example, assume we have a network that model all causes of fever:



If there is a fever, then one of them (at least) must be the cause, but none of them necessarily cause a fever. We say that the causal relation between parent and child is **inhibited**. We can model **inhibition** by individual inhibition factors q_d

Def 3.5. The CPD of a **noisy disjunction node** X with $\text{Parents}(X) = \{X_1, \cdots, X_n\}$ in a Bayesian Network is given by:

$$P(X \mid X_1, \cdots, X_n) = 1 - \left(\prod_{\{j \mid X_j = T\}} q_j \right)$$

where the q_i are the **inhibition factors** of $X_i \in \text{Parents}(X)$, defined as:

$$q_i := P(\neg X \mid \neg X_1, \cdots, \neg X_{i-1}, X_i, \neg X_{i+1}, \cdots, \neg X_n)$$

Note

Instead of a distribution with 2^k parameters, we only need k parameters.

Example 3.4. Assume the following inhibition factors:

$$\begin{aligned} q_{\text{cold}} &= P(\neg\text{fever} \mid \text{cold}, \neg\text{flu}, \neg\text{malaria}) = 0.6 \\ q_{\text{flu}} &= P(\neg\text{fever} \mid \neg\text{cold}, \text{flu}, \neg\text{malaria}) = 0.2 \\ q_{\text{malaria}} &= P(\neg\text{fever} \mid \neg\text{cold}, \neg\text{flu}, \text{malaria}) = 0.1 \end{aligned}$$

If we model *fever* as a noisy disjunction node, then the general rule for the CPD

$$P(X_i \mid \text{Parents}(X_i)) = \prod_{\{j \mid X_j = T\}} q_j$$

gives the following table:

Cold	Flu	Malaria	P(Fever)	P(¬Fever)
F	F	F	0.0	1.0
F	F	T	0.9	**0.1**
F	T	F	0.8	**0.2**
F	T	T	0.98	$0.02 = 0.2 \cdot 0.1$
T	F	F	0.4	**0.6**
T	F	T	0.94	$0.06 = 0.6 \cdot 0.1$
T	T	F	0.88	$0.12 = 0.6 \cdot 0.2$
T	T	T	0.988	$0.012 = 0.6 \cdot 0.2 \cdot 0.1$

Def 3.6. John, Mary and the Alarm: This is the well-known Bayesian Network example found in textbooks. It goes as follows:

- I got very valuable stuff at home. So I bought an alarm. Unfortunately, the alarm just rings at home, doesn't call me on my mobile.
- I've got two neighbors, Mary and John, who'll call me if they hear the alarm.
- The problem is that, sometimes, the alarm is caused by an earthquake.
- Also, John might confuse the alarm with his telephone, and Mary might miss the alarm altogether because she typically listens to loud music.

$\rightsquigarrow$ Random variables: B, E, A, J, M for Burglary, Earthquake, Alarm, John, and Mary, respectively.

Given that both John and Mary call me, what is the probability of burglary?

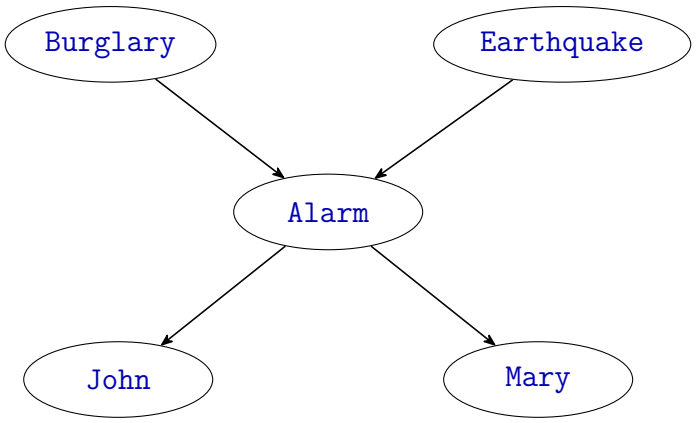


Figure 1: Burglary-Earthquake Example

Question

How to calculate $\mathbf{P}(B \mid J, M)$? (How inference in Bayesian Networks works?)

Let's use Enumeration (Theorem 2.8):

$$\mathbf{P}(Q \mid E_1 = e_1, \cdots E_{ne} = e_{ne}) = \alpha \left(\sum_{u \in D_u} \mathbf{P}(Q, E_1 = e_1, \cdots E_{ne} = e_{ne}, U_1 = u_1, \cdots, U_{nu} = u_{n_u}) \right)$$

with $Q = B, E = \{J, M\}, U = \{A, E\}$

$$\mathbf{P}(B \mid J, M) = \alpha \left(\sum_{b_a, b_e \in \{T,F\}} \mathbf{P}(B, J, M, A = b_a, E = b_e) \right)$$

Let's exploit Theorem 3.1

$$\mathbf{P}(X_1, \cdots, X_n) = \prod_{X_i \in \mathcal{X}} \mathbf{P}(X_i \mid \text{Parents}(X_i))$$

$$\mathbf{P}(B \mid J, M) = \alpha \left(\sum_{b_a, b_e \in \{T,F\}} P(J \mid A = b_a) \cdot P(M \mid A = b_a) \cdot \mathbf{P}(A = b_a \mid E = b_e, B) \cdot P(E = b_e) \cdot \mathbf{P}(B) \right)$$

These are 5 factors in 4 summands over two cases for Burglary $\rightsquigarrow$ 38 arithmetic operations $+3$ for α $\rightsquigarrow$ General worst case: $\mathcal{O}(n \cdot 2^n)$

Question

Can we do better?

Answer

We can optimize a little bit:

$$\mathbf{P}(B \mid J, M) = \alpha \left(\mathbf{P}(B) \cdot \left(\sum_{b_a, b_e \in \{T,F\}} P(J \mid A = b_a) \cdot P(M \mid A = b_a) \cdot \mathbf{P}(A = b_a \mid E = b_e, B) \cdot P(E = b_e) \right) \right)$$

$$\mathbf{P}(B \mid J, M) = \alpha \left(\mathbf{P}(B) \cdot \left(\sum_{b_e \in \{T,F\}} P(E = b_e) \cdot \sum_{b_a \in \{T,F\}} \mathbf{P}(A = b_a \mid E = b_e, B) \cdot P(J \mid A = b_a) \cdot P(M \mid A = b_a) \right) \right)$$

This way, we have 3 factors in 2 summands + two factors in the outer product, over two cases $\rightsquigarrow$ 28 arithmetic operations $+3$ for α

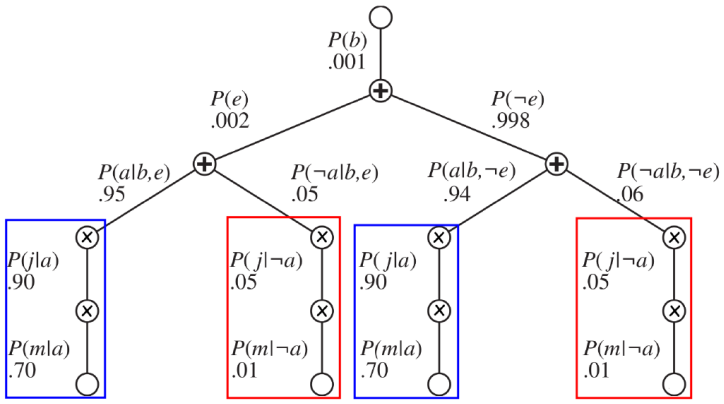
Question

Can we do better?

Answer

Yes! $\rightsquigarrow$ **Variable Elimination**

Notice that enumeration corresponds to a depth-first traversal of an arithmetic expression tree, here is an example with some example probabilities:



Notice the redundancies!

- $P(e)$ and $P(\neg e)$ (No need for both!)
- $P(j \mid \neg a), P(j \mid a), P(m \mid \neg a), P(m \mid a)$ each is being computed twice! That is because $P(J \mid A = b_a)$ and $P(M \mid A = b_a)$ only depends on A, but they are trapped behind the summation over E, hence computed twice.

We can precompute these and store them (trading time complexity by space complexity) $\rightsquigarrow$ Variable Elimination.
The idea is, instead of left-to-right (top-down DFS), we operate right-to-left (bottom-up) and store intermediate *factors* (*functions*) along with their *dependencies*

Lemma 3.2. Given a query $\mathbf{P}(Q_1, \cdots, Q_{n_Q} \mid E_1 = e_1, \cdots, E_{n_E} = e_{n_e})$, we can ignore and remove all hidden vavriables that are not ancestors of any of the $Q_1, \cdots, Q_{n_Q}$ or $E_1, \cdots, E_{n_E}$

4 Decision Networks

We now know how to update our world model, represented as a set of random variables, given observations. Now we need to talk about **taking actions**.

Question

Given a world model and a set of actions, what are the likely consequences of action? How good are these consequences?

Idea

(1) Represent actions as *special* random variables.
- Given disjoint actions $a_1, \cdots, a_n$, we introduce a random variable A with domain $\{a_1, \cdots, a_n\}$. Then we can query $\mathbf{P}(X \mid A = a_i)$

(2) Assign numerical values to the outcomes of actions, i.e. a function $u : \text{dom}(X) \to \mathbb{R}$

(3) Choose the action that **maximizes** the **expected value** if u

Def 4.1. **Decision Theory** investigates **decision problems**, i.e. how a utility-based agent a deals with choosing among actions based on the desirability of their outcomes given by a real-valued **utility function** U on states $s \in S$, i.e. $U : S \to \mathbb{R}$

Note

If our states are random variables, then we obtain a random variable for the utility function.
Let $X_i : \Omega \to D_i$ random variables on a probability space $\langle \Omega, \mathcal{F}, P \rangle$ and $f : D_1 \times \cdots \times D_n \to \mathbb{R}$. Then $F(x) := f(X_0(x), \cdots, X_n(x))$ is a random variable $\Omega \to \mathbb{R}$. Meaning, any function on the domains of random variables is itself a random variable.

Example 4.1. Consider commuting to work as en example experiment where we have $\Omega = \{$accident on the road, minor crash on the

We define a random variable X representing travel time in minutes:

- $X(\text{accident on the road}) = 90$
- $X(\text{minor crash on the road}) = 45$
- $X(\text{smooth commute}) = 20$

These are the states our world can be in. We want to define a utility function F representing our happiness. Because X is a **random**, F becomes **random** too, we do not get one happiness level, we get a distribution of possible hapiness levels based on the underlying reality Ω.

Say that $f(t) = 100 - t$ (the longer the trip, the less happy I am). Then we have:

Reality (ω)	Travel Time ($X(\omega)$)	Happiness ($f(X(\omega))$)
accident on the road	90	10
minor crash on the road	45	55
smooth commute	20	80

Def 4.2. Given a probability space $\langle \Omega, \mathcal{F}, P \rangle$ and a random variable $X : \Omega \to D$ with $D \subseteq \mathbb{R}$, then

$$\mathbb{E}(X) := \sum_{x \in D} P(X = x) \cdot x$$

is called the **Expected Value** of X. Analogously, let $e_1, \cdots, e_n$ be a sequence of events. Then the **Expected Value** of X given $e_1, \cdots, e_n$ is defined as

$$\mathbb{E}(X \mid e_1, \cdots, e_n) := \sum_{x \in D} P(X = x \mid e_1, \cdots, e_n) \cdot x$$

Example 4.2. Continuing with the Commute Time example, assume we have the following probabilities:

- $P(X = 20) = 0.7$
- $P(X = 45) = 0.2$
- $P(X = 90) = 0.10$

$$\begin{aligned}\mathbb{E}(X) &:= \sum_{x\in D} P(X=x)\cdot x \\ &:= P(X=20)\cdot 20 + P(X=45)\cdot 45 + P(X=90)\cdot 90 \\ &:= 0.7\cdot 20 + 0.2\cdot 45 + 0.10\cdot 90 \\ \mathbb{E}(X) &:= 32 \text{ minutes}\end{aligned}$$

This means, (while I might never actually spend exatly 32 minutes on any single day), if I commute for, e.g. a year, my average travel time will be 32 minutes.

Def 4.3. Given:

- $D = \{a_1, \cdots, a_n\}$: a set of possible action, where we have a special random variable $A : \Omega \to D$ representing the action.
- $X_i : \Omega \to D_i$: random variables (the states)
- $U : D \times D_i \to \mathbb{R}$: a utility function

Then, the **Expected Utility** of the action $a \in D$ is the expected value of U (interpreted as a random variable) given $A = a$:

$$\text{EU}(a) := \sum_{\langle x_1,\cdots,x_n\rangle \in D_1\times\cdots\times D_n} P(X_1 = x_1, \cdots, X_n = x_n \mid A = a)\cdot U(x_1, \cdots, x_n, a)$$

Example 4.3.

- $D = \{\text{take}, \text{leave}\}$ for taking or leaving the umbrella.
- X: random variable for the state of the weather outside with $D_x = \{\text{rain}, \text{sum}\}$ with priors: $P(X = \text{rain}) = 0.3, P(X = \text{sun}) = 0.7$
- $U : D \times D_x \to R$ the utility function defined as:
  - $U(\text{sun}, \text{leave}) = 100$
  - $U(\text{sun}, \text{take}) = 80$
  - $U(\text{rain}, \text{leave}) = 0$
  - $U(\text{rain}, \text{take}) = 60$

$$\begin{aligned}\text{EU}(\text{take}) &:= \sum_{x_i\in\{\text{rain},\text{sun}\}} P(X = x_i \mid A = \text{take})\cdot U(x_i, \text{take}) \\ &:= P(X = \text{rain} \mid A = \text{take})\cdot U(\text{rain}, \text{take}) + P(X = \text{sun} \mid A = \text{take})\cdot U(\text{sun}, \text{take}) \\ &:= P(X = \text{rain})\cdot U(\text{rain}, \text{take}) + P(X = \text{sun})\cdot U(\text{sun}, \text{take}) \\ &:= 0.3\cdot 60 + 0.7\cdot 80 = 74\end{aligned}$$

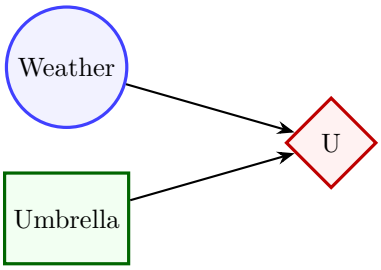
$$\begin{aligned}\text{EU}(\text{leave}) &:= \sum_{x_i\in\{\text{rain},\text{sun}\}} P(X = x_i \mid A = \text{leave})\cdot U(x_i, \text{leave}) \\ &:= P(X = \text{rain} \mid A = \text{leave})\cdot U(\text{rain}, \text{leave}) + P(X = \text{sun} \mid A = \text{leave})\cdot U(\text{sun}, \text{leave}) \\ &:= P(X = \text{rain})\cdot U(\text{rain}, \text{leave}) + P(X = \text{sun})\cdot U(\text{sun}, \text{leave}) \\ &:= 0.3\cdot 0 + 0.7\cdot 100 = 70\end{aligned}$$

Def 4.4. MEU Principle for Rationality: We call an action **rational** if it **Maximizes the Expected Utility (MEU)**. A utility-based agent is called rational, iff it always chooses a rational action.

Def 4.5. Decision Network: A **decision network** is a Bayesian Network with two additional kinds of nodes:

1. **Action Nodes**: representing a set of possible actions (square)
2. A single **Utility Node** (also called value node) (diamond)

Simple Example (no dependencies):



General Algorithm

Given evidence $E_j = e_j$ and action nodes $A_1, \cdots, A_k$, compute the expected utility of each action, given evidence, for each action we have:

$$\text{EU}(a \mid E = e_j) := \sum_{\langle x_1, \cdots, x_n \rangle \in D_1 \times \cdots \times D_n} P(X_1 = x_1, \cdots, X_n = x_n \mid A = a, E_j = e_j) \cdot U(x_1, \cdots, x_n, a)$$

And we are interested in the single action that MEU:

$$A_{\text{MEU}} = \arg\max_{a_1, \cdots, a_k} \text{EU}(a_i \mid E = e_j)$$

And we say $\text{MEU} = \text{EU}(A_{\text{MEU}} \mid E = e_j)$ (maximum expected utility is the expected utility we get if we would choose the action that maximizes the expected utility)

Note

Decision Theorists are mostly *economists*, not statisticians. Economists usually talk about *prizes* (which we call states).

Def 4.6. Given prizes A and B, an agent can express preferences of the form:

- $A \succ B$: A is preferred over B
- $A \sim B$: indifference between A and B
- $A \succcurlyeq B$: B is not preferred over A (the agent wants A at least as much as $B \rightsquigarrow$ *weak preference*)

Note

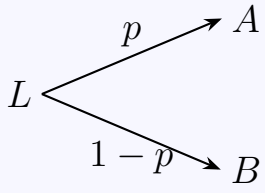
In computer science, we define relations $\succ, \sim$ on our set of states $\mathcal{S}$

But that works only for deterministic environments. In a non-deterministic one, the agent simply does not know the consequences of the actions. Now we look at what Decision Theorists call *lotteries*

Def 4.7. Let $\mathcal{S}$ be a set of states. We call a random variable X with domain $\{A_1, \cdots, A_n\} \subseteq \mathcal{S}$ a **lottery** and write $[p_1, A_1; p_2, A_2; \cdots; p_n, A_n]$ where $p_i = P(X = A_i)$

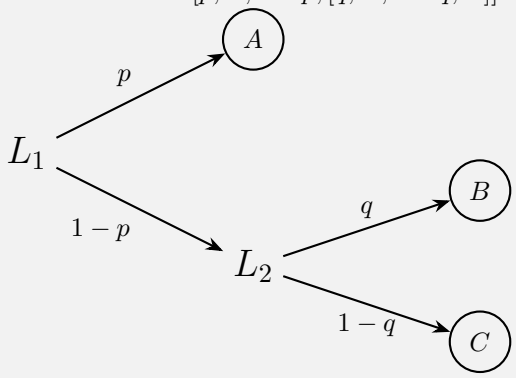
Idea

A lottery represents the result of a nondeterministic action that can have outcomes A_i with prior p_i. For the binary case, we use $[p, A; 1-p, B]$. We can then extend preferences to include lotteries, as a measure of how *strongly* we prefer one prize over another.



Note

We assume the set of states $\mathcal{S}$ to be closed under lotteries, ,i.e. lotteries are also states. That allows us to consider lotteries such as $[p, A; 1-p, [q, B; 1-q, C]]$



Preference Relations

- $X \succcurlyeq Y \land Y \succcurlyeq X \Rightarrow X \sim Y$
- $X \succcurlyeq Y \land \neg(Y \succcurlyeq X) \Rightarrow X \succ Y$

Def 4.8. We call a set $\succ$ of preferences **rational**, iff the following constraints hold:

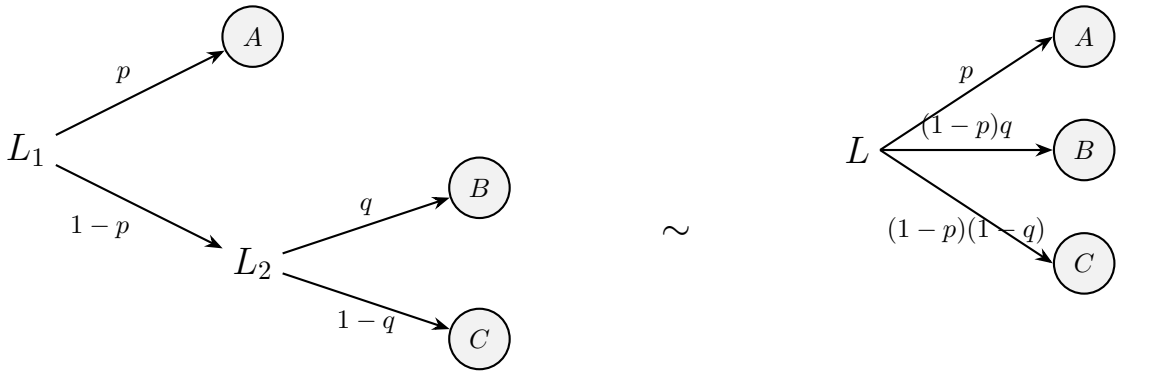
- **Orderability**: $A \succ B \lor B \succ A \lor A \sim B$
- **Transitivity**: $A \succ B \land B \succ C \Rightarrow A \succ C$
- **Continuity**: $A \succ B \succ C \Rightarrow (\exists p.[p, A; 1-p, C] \sim B)$
- **Substitutability**: $A \sim B \Rightarrow [p, A; 1-p, C] \sim [p, B; 1-p, C]$
- **Monotonicity**: $A \succ B \Rightarrow ((p > q) \Leftrightarrow [p, A; 1-p, B] \succ [q, A; 1-q, B])$
- **Decomposability**: $[p, A; 1-p, [q, B; 1-q, C]] \sim [p, A; ((1-p)q), B; ((1-p)(1-q)), C]$

From a set of rational preferences, we can obtain a meaningful utility function.

In plain English, this definition says when a person's preferences are considered *rational* in decision theory.

- **Orderability**: for any two options, you must be able to compare them. You either prefer one, prefer the other, or are indifferent.
- **Transitivity**: if you prefer A to B, and B to C, then you should also prefer A to C.
- **Continuity**: if A is better than B, and B is better than C, then there is some probability p such that you are indifferent between B for sure and the lottery $[p, A; 1-p, C]$.
- **Substitutability**: if you are indifferent between A and B, then replacing A by B inside any lottery should not change your preference.
- **Monotonicity**: if A is better than B, then a lottery with a higher probability of A should be preferred to one with a lower probability of A.

- **Decomposability**: a nested lottery should be equivalent to a single flat lottery with the same overall probabilities.



$[p, A; 1-p, [q, B; 1-q, C]]$ $\qquad$ $[p, A; (1-p)q, B; (1-p)(1-q), C]$

Theorem 4.1. Ramsey's Theorem: Given rational preferences, there exists a real valued function U such that $U(A) \geq U(B)$, iff $A \succcurlyeq B$ and $U([p, S_1; \cdots; p_n, S_n]) = \sum_i p_i U(S_i)$

Note

This theorem states that if an agent's / person's preferences are rational, then we can assign numbers to outcomes in a way that matches those preferences. i.e. there exists a utility function U that gives each outcome a number. If $A \succcurlyeq B$ then $U(A) \geq U(B)$. The higher the utility number, the more preferred the outcome. For a lottery $[p, S_1; \cdots; p_n, S_n]$, its utility is just the probability-weighted average of the utilities of its outcomes.

Def 4.9. We call a linear (total) ordering on states a **value function** or **ordinal utility function**. With deterministic prizes only (no lottery choices), only a total ordering on prizes can be determined.

Note

If we don't need to care about *relative* utilities of states, e.g. to compute non-trivial expected utilities, that's all we need anyway! i.e. when there are no probabilities to compare, "utility" is just a way to sort outcomes from best to worst, not a precise measurement.

Def 4.10. Standard approach to assessment of human utilities : Compare a given state A to a **standard lottery** L_p that has:

- best possible prize $u_\top$ with probability p
- worst possible catastrophe $u_\bot$ with probability $1-p$

adjust lottery probability p until $A \sim L_p$. Then $U(A) = p$

Example 4.4. Choose $u_\top \hat{=}$current state, $u_\bot \hat{=}$ instant death

pay \$30 $\sim L$ $\begin{cases} 0.999999 \rightarrow \text{continue as before} \\ 0.000001 \rightarrow \text{instant death} \end{cases}$

Popular Utility Functions

Def 4.11. Normalized utilities: $u_\top = 1, u_\bot = 0$ (not meaningful)
Def 4.12. Money: very intuitive, often easy to determine, but actually not well-suited as utility function
Def 4.13. Micromorts: one millionth chance of instant death. (useful for paying to reduce product risk, etc.). But not necessarily a good measure of risk, if the risk is only severe injury or illness
Def 4.14. QALYs: quality adjusted life years $\rightsquigarrow$ very better, more informative, useful for medical decisions involving substantial risk

Def 4.15. Value of Perfect Information (VPI): Let A be the set of available actions and F is a random variable. Given

evidence $E_i = e_i$, let α be the action that MEU a priori, and α_f the action that MEU given $F = f$, i.e.

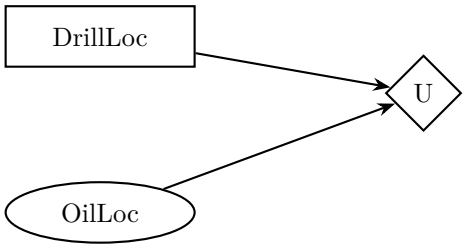
$$\alpha = \arg\max_{a \in A} \text{EU}(a \mid E_i = e_i)$$
$$\alpha_f = \arg\max_{a \in A} \text{EU}(a \mid E_i = e_i, F = f)$$

The **value of perfect information (VPI)** on F given evidence $E_i = e_i$ is defined as:

$$\text{VPI}_{E_i=e_i}(F) := \left(\sum_{f \in \text{dom}(F)} P(F = f \mid E_i = e_i) \cdot \text{EU}(\alpha_f \mid E_i = e_i, F = f) \right) - \text{EU}(\alpha \mid E_i = e_i)$$

Example 4.5. consider two blocks A, B and exactly one has oil which worth $\$K$. You can drill in one location. You know the priors: $P(\text{oil at A}) = P(\text{oil at B}) = 0.5$.

We can model this a Decision Network with one chance node (oil location), one action node (drilling location), and one utility node:



D	O	$U(D,O)$
a	a	$K\$$
a	b	0
b	b	$K\$$
b	a	0

	$P(O)$
a	0.5
b	0.5

Question

Without any information, what is the action that MEU?

$$\begin{aligned}
\text{EU}(D = a) &= P(O = a) \cdot U(D = a, O = a) + P(O = b) \cdot U(D = a, O = b) \\
&= 0.5 \cdot K + 0.5 \cdot 0 = \frac{K}{2} \\
\text{EU}(D = b) &= P(O = a) \cdot U(D = b, O = a) + P(O = b) \cdot U(D = b, O = b) \\
&= 0.5 \cdot 0 + 0.5 \cdot K = \frac{K}{2}
\end{aligned}$$

Answer

Which means, without any additional information, both actions have the same expected utility. We write $\text{MEU}(\emptyset)$ for the maximum expected utility given no extra information:

$$\text{MEU}(\emptyset) = \arg\max_{a} \text{EU}(a) = \frac{K}{2}$$

Question

What if someone offered to give us information about where the oil is. What is the value of that information? i.e. How much we are willing to pay to get such information?

Answer

Assume we have information that says *the oil location is b* (our evidence), then we get EU of 0 for $D = a$ and we get EU of K for $D = b$ and we will choose to dig in b immediately.
The value of this perfect information is simply what we get with information (K) minus what we would get without it ($\frac{K}{2}$).

$$\text{VPI}(E) = \text{MEU}(A \mid E) - \text{MEU}(\emptyset)$$

5 Hidden Markov Models

Motivation

- The world changes in *stochastically predictable ways*
- For example, the weather changes, but the weather tomorrow is somewhat predictable given today's weather and other factors.
- The stock market changes, but the stock price tomorrow is probably related to today's price
- A patient's blood sugar changes, but their blood sugar is related to their blood sugar 10 minutes ago

Question

How can we model this?

Def 5.1. Given a probability space $\langle \Omega, \mathcal{F}, P \rangle$, and a time structure S with an order $\langle S, \preceq \rangle$. Then a sequence of random variables $(X_t)t \in S$ with $\text{dom}(X_t) = D$ is called **stochastic process** over the **time strcuture** S

Note

X_t models the outcome of the random variable X at a given time step t. The sample sapce Ω corresponds to the set of all possible sequences of outcomes.

We will almost exclusively use $\langle S, \preceq \rangle = \langle \mathbb{N}, \leq \rangle$

Notation

Given a stochastic process X_t over S and $a, b \in S$ with $a \preceq b$, we write $X_{a:b}$ for the sequence $X_a, X_{a+1}, \cdots, X_{b-1}, X_b$ and $X_{a:b}^{=x}$ for $X_a = x_a, \cdots, X_b = x_b$

Idea

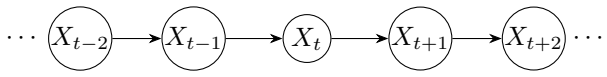
Construct a Bayesian Network from these variables

Def 5.2. Let $(X_t)_{t \in S}$ be a stochastic process. We say that X has **n-th order Markov property** iff X_t only depends on a bounded subset of $X_{0:t-1}$, i.e. for all $t \in S$ we have $\mathbf{P}(X_t \mid X_0, \cdots, X_{t-1}) = \mathbf{P}(X_t \mid X_{t-n}, \cdots, X_{t-1})$ for some $n \in S$.

A stochastic process with the **Markov** property for some n is called an **n-th order Markov process**

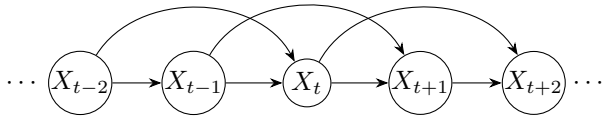
Def 5.3. First-Order Markov Property (Markov Chain):

$$\mathbf{P}(X_t \mid X_{0:t-1}) = \mathbf{P}(X_t \mid X_{t-1})$$



Def 5.4. Second-Order Markov Property:

$$\mathbf{P}(X_t \mid X_{0:t-1}) = \mathbf{P}(X_t \mid X_{t-2}, X_{t-1})$$



Note

Given a Markov chain, and we are now at time t, then the future is conditionally independent of the past given the present:

$$\mathbf{P}(X_{t+1} \mid X_{0:t}) = \mathbf{P}(X_{t+1} \mid X_t)$$

Def 5.5. Let $(X_t)_{t\in S}$ be a stochastic process where X has the Markov property (for any n). We call the probability distribution $\mathbf{P}(X_t \mid X_{0:t-1})$ the **transition model** of the stochastic process $(X_t)_{t\in S}$

Def 5.6. Stationary Assumption: A Markov chain is called **stationary** if the transition model is independent of time, i.e. $\mathbf{P}(X_t \mid X_{t-1})$ is the same for all t

Example 5.1. Consider we represent the weather as a Markov chain, we introduce a variable X with domain $\{\text{sun}, \text{rain}\}$ and we fix the initial distribution $\mathbf{P}(X) = \langle 1, 0\rangle$ (sunny), and we have a stationary transition model that might be:

X_{t-1}	X_t	$P(X_t \mid X_{t-1})$
sun	sun	0.9
sun	rain	0.1
rain	sun	0.3
rain	rain	0.7

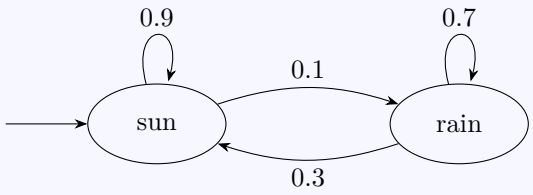
Different ways of representing transition models

If a Markov chain X is stationary and with finite domain, we can represent the transition model as a matrix T where $T_{ij} := P(X_{t=j} \mid X_{t-1=i})$

$$T = \begin{pmatrix} P(\text{sun} \mid \text{sun}) & P(\text{rain} \mid \text{sun}) \\ P(\text{sun} \mid \text{rain}) & P(\text{rain} \mid \text{rain}) \end{pmatrix}$$

$$\begin{pmatrix} 0.9 & 0.1 \\ 0.3 & 0.7 \end{pmatrix}.$$

We can also represent it to look like a finite state machine :



Question

Given our weather example (example 5.1), what is the probability distribution of X after one time step (knowing it is initially sunny)?

Answer

- Trivially from the transition model $P(X_2 = sun \mid X_1 = sun) = 0.9$. Hence, $\mathbf{P}(X_2) = \langle 0.9, 0.1\rangle$ But let's try to reason about it.
- We know that for $t = 1$, $P(\text{sun}) = 1$ and $P(\text{rain}) = 0$, i.e. the initial distribution is $\mathbf{P}(X_1) = \langle 1, 0\rangle$. We also have a stationary transition model. We are interested in knowing $\mathbf{P}(X_2)$.

We compute it for *sun* by marginalization over the different values of $X_{t=1}$:

$$\begin{aligned} P(X_2 = \text{sun}) &= \sum_{x\in\{\text{sun},\text{rain}\}} P(X_2 = \text{sun}, X_1 = x) \\ &= \sum_{x\in\{\text{sun},\text{rain}\}} P(X_2 = \text{sun} \mid X_1 = x) \cdot P(X_1 = x) \\ P(X_2 = \text{sun}) &= P(X_2 = \text{sun} \mid X_1 = \text{sun}) \cdot P(X_1 = \text{sun}) + P(X_2 = \text{sun} \mid X_1 = \text{rain}) \cdot P(X_1 = \text{rain}) \\ &= 0.9 \cdot 1.0 + 0.1 \cdot 0 = 0.9 \end{aligned}$$

What we did is the simplest algorithm for Markov chains, it is called **mini forward**

Def 5.7. Mini Forward Algorithm: Given a stationary Markov chain X with initial distribution $\mathbf{P}(X_1)$, we the distribution at any time step t by recursively applying:

$$\mathbf{P}(X_n) = \sum_{x_{n-1}} P(X_n \mid X_{n-1} = x_{n-1}) \cdot P(X_{n-1} = x_{n-1})$$

for $n = 2, 3, \ldots, t$. At each step, we marginalize out the previous time step by weighting each possible transition by its prior probability.

Conversion

If we were to do this for infinitely many time steps, the probability distribution $\mathbf{P}(X)$ of the Markov chain X converges.

$$\mathbf{P}(X_1) = \langle 1.0, 0.0 \rangle \rightsquigarrow \mathbf{P}(X_2) = \langle 0.9, 0.1 \rangle \rightsquigarrow \mathbf{P}(X_3) = \langle 0.84, 0.16 \rangle \rightsquigarrow$$

$$\mathbf{P}(X_4) = \langle 0.804, 0.196 \rangle \rightsquigarrow \cdots \rightsquigarrow \mathbf{P}(X_\infty) = \langle 0.75, 0.25 \rangle$$

This applies to any initial distribution $\mathbf{P}(X) = \langle p, 1-p \rangle$!

Def 5.8. As $t \to \infty$, $\mathbf{P}(X)$ converges towards a fixed point called the **stationary distribution** of the Markov chain X

In other words, the Markov chain essentially "forgets" its initial state; the long-term probabilities become completely independent of the starting conditions.

Computing the Stationary Distribution

To compute the stationary distribution S, we can use the matrix representation of the transition model T. Because the stationary distribution is a fixed point, passing it through the transition model returns the same distribution.
If we treat the distribution S as a column vector and T is the transition matrix (as defined earlier, where rows sum to 1), then taking the next step involves the transpose of T. The stationary distribution satisfies:

$$S = T^T \cdot S$$

Note: If you were to represent S as a row vector instead, the equation would simply be $S = S \cdot T$.
To compute S (as a column vector), we solve the system of linear equations:

$$(T^T - I) \cdot S = 0$$

along with the necessary constraint that all probabilities must sum to 1 ($\sum_i S_i = 1$). This means S is an eigenvector of T^T corresponding to the eigenvalue $\lambda = 1$.

Note

Markov chains are not so useful for most agents, usually we need another ingredient, observations.
We begin to speak of **hidden Markov models** (HMMs) when we have an underlying Markov chain X that is **hidden** to us and we **observe** some evidence E that is affected by that Markov chain.

Def 5.9. We call a stochastic process of *hidden* variables a **state variable**

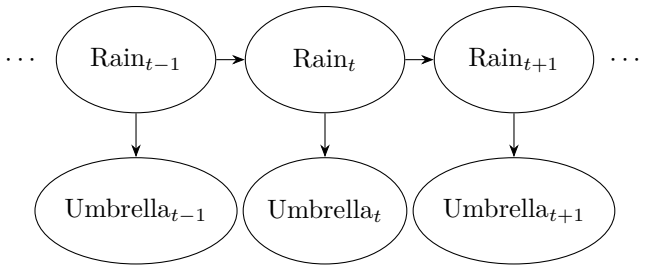
Example 5.2. You are a security guard in a secret underground facility, want to know it if is raining outside. Your only source of information is whether the director comes in with an umbrella.

- we have a stochastic process $\text{Rain}_0, \text{Rain}_1, \text{Rain}_2, \cdots$ of **hidden** variables, and
- a related stochastic process $\text{Umbrella}_0, \text{Umbrella}_1, \text{Umbrella}_2, \cdots$ of evidence variables.

And a combined stochastic process $\langle \text{Rain}_0, \text{Umbrella}_0 \rangle, \langle \text{Rain}_1, \text{Umbrella}_1 \rangle, \cdots$

Note: Umbrella_t only depends on Rain_t, not on e.g. Umbrella_{t-1}

We model this as a Bayesian Network:



Def 5.10. We call the conditional probability distribution $\mathbf{P}(E_t \mid X_{0:t}, E_{1:t-1})$ a **sensor model**

Def 5.11. We say that a sensor model has the **sensor Markov property**, iff $\mathbf{P}(E_t \mid X_{0:t}, E_{1:t-1}) = \mathbf{P}(E_t \mid X_t)$, i.e. the sensor model depends only the current state.

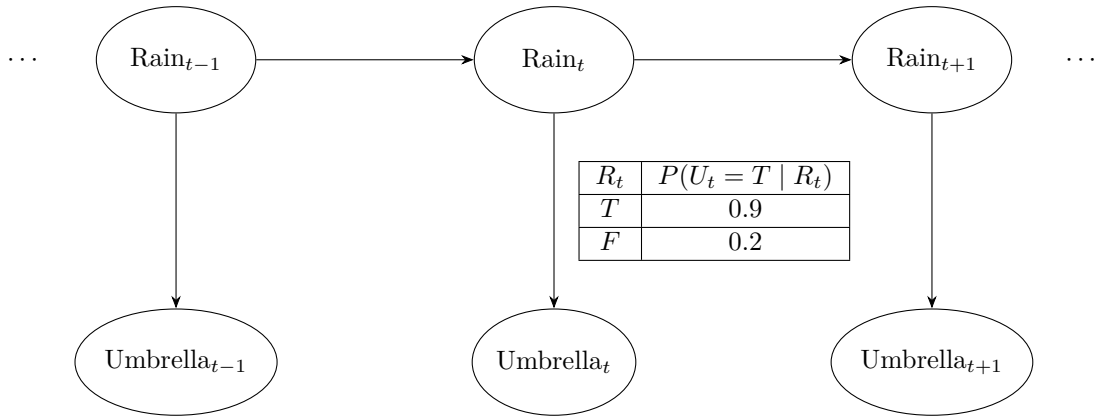
Observation Matrix

If a sensor model has the sensor Markov property, we can represent each observation $E_t = e_t$ at time t as the *diagonal* matrix O_t with $O_{t_{ii}} := P(E_t = e_t \mid X_t = i)$

Def 5.12. A pair $\langle X, E \rangle$ where X is a stationary Markov chain (hidden state variable), and E has the sensor markov property (evidence) is called a (stationary) **Hidden Markov Model (HMM)**

Example 5.3. Our umbrella example (example 5.2) is a HMM with the following example transition model and sensor model:

R_{t-1}	$P(R_t = T \mid R_{t-1})$
T	0.7
F	0.3



R_t	$P(U_t = T \mid R_t)$
T	0.9
F	0.2

Umbrella Matrices

$$O_T = \begin{pmatrix} 0.9 & 0 \\ 0 & 0.2 \end{pmatrix} \qquad O_F = \begin{pmatrix} 0.1 & 0 \\ 0 & 0.8 \end{pmatrix} \qquad T = \begin{pmatrix} 0.7 & 0.3 \\ 0.3 & 0.7 \end{pmatrix}$$

Def 5.13. Markov Inference Tasks: Gievn a Markov chain with state variables X_t and evidence variables E_t, we are interested in the following **Markov inference tasks**:

- **Filtering (Monitoring)**: The task of tracking / updating the probability distribution $B_t(X) = \mathbf{P}(X_t \mid E_{1:t}^{=e})$ (**belief state**) over time. i.e. given sequence $e_1, \cdots, e_t$ of observations up until time t, we compute the likely state of the world at current time t.
- **Prediction (State Estimation)**: Given sequence $e_1, \cdots, e_t$ of observations up until time t, compute the likely *future* state of the world at time $t + k$ for $k > 0$. i.e. calculating the probability distribution $\mathbf{P}(X_{t+k} \mid E_{1:t}^{=e})$.
- **Smoothing (Hindsight)**: Given sequence $e_1, \cdots, e_t$ of observations up until time t, compute the likely *past* state of the world at time $t - k$ for $0 < k < t$. i.e. calculate the probability distribution $\mathbf{P}(X_{t-k} \mid E_{1:t}^{=e})$

- **Most Likely Explanation (MLE)**: Given sequence $e_1, \cdots, e_t$ of observations up until time t, copmute the most likely sequence of states that led to these observations, i.e. calculate:

$$\arg\max_x \; P(X_{1:t}^{=x} \mid E_{1:t}^{=e})$$

Note

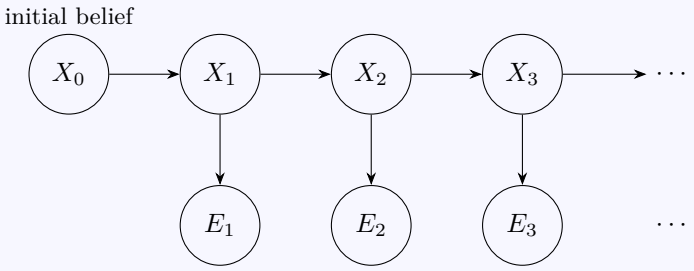
The most likely sequence of states is *not* (necessarily) the sequence of most likely states

Def 5.14. Given a Markov chain X and evidence variables E, let $\mathbf{P}(X_0)$ be the initial prior (initial belief), then we can compute the full JPD as:

$$\mathbf{P}(X_{0:t}, E_{1:t}) = \mathbf{P}(X_0) \cdot \left(\prod_{i=1}^{t} \mathbf{P}(X_i \mid X_{i-1}) \cdot \mathbf{P}(E_i \mid X_i) \right)$$

Explanation

This is a Bayesian Network (recall theorem 3.1):



Filtering Base Case

We want to compute the belief state at $t = 1$, which is $B(X_1) = \mathbf{P}(X_1 \mid E_1)$. We can derive this in two steps as follows:
By Bayes' theorem, we incorporate the new evidence to update our belief:

$$\begin{aligned} \mathbf{P}(X_1 \mid E_1) &= \frac{\mathbf{P}(E_1 \mid X_1) \cdot \mathbf{P}(X_1)}{\mathbf{P}(E_1)} \\ &= \alpha(\mathbf{P}(E_1 \mid X_1) \cdot \mathbf{P}(X_1)) \end{aligned}$$

Here $\mathbf{P}(E_1 \mid X_1)$ is our sensor model.
Next, we expand the prior prediction $\mathbf{P}(X_1)$ by marginalizing over the initial state X_0:

$$\begin{aligned} \mathbf{P}(X_1) &= \sum_{x_0} \mathbf{P}(X_1, X_0 = x_0) \\ &= \sum_{x_0} \mathbf{P}(X_1 \mid X_0 = x_0) \cdot \mathbf{P}(X_0 = x_0) \end{aligned}$$

Here, $\mathbf{P}(X_1 \mid X_0)$ is the transition model, and $\mathbf{P}(X_0)$ is the initial belief prior.
Combining these gives the complete filtering formula for $t = 1$:

$$\mathbf{P}(X_1 \mid E_1) = \alpha \left(\mathbf{P}(E_1 \mid X_1) \sum_{x_0} \mathbf{P}(X_1 \mid x_0) \cdot \mathbf{P}(x_0) \right)$$

Prediction Base Case

If we know the initial belief distribution $\mathbf{P}(X_0)$ and want to compute the distribution of the next state $\mathbf{P}(X_1)$ prior to any observations, we perform a 1-step prediction.
We can compute this by marginalizing over all possible values of the initial state X_0:

$$\mathbf{P}(X_1) = \sum_{x_0 \in \text{dom}(X_0)} \mathbf{P}(X_1, X_0 = x_0)$$

Using the chain rule of probability, we split the joint probability into the transition model and the initial prior:

$$\mathbf{P}(X_1) = \sum_{x_0} \mathbf{P}(X_1 \mid X_0 = x_0) \cdot \mathbf{P}(X_0 = x_0)$$

This equation tells us that the probability of being in a state at $t = 1$ is the sum of the probabilities of transitioning to it from every possible previous state, weighted by how likely those previous states were.

Filtering Derivation

To compute the belief state at time t given all evidence up to time t, we want to find $\mathbf{P}(X_t \mid E_{1:t})$. We split the evidence into the current observation E_t and all past observations $E_{1:t-1}$.

Step 1: Update (Bayes' Rule / Product Rule)
We want to find $P(X_t \mid E_t, E_{1:t-1})$. By definition of conditional probability:

$$\begin{aligned}\mathbf{P}(X_t \mid E_{1:t}) &= \mathbf{P}(X_t \mid E_t, E_{1:t-1}) \\ &= \frac{\mathbf{P}(X_t, E_t, E_{1:t-1})}{\mathbf{P}(E_t, E_{1:t-1})}\end{aligned}$$

We apply the product rule to the numerator ($\mathbf{P}(A, B, C) = \mathbf{P}(A \mid B, C) \cdot \mathbf{P}(B, C)$):

$$\mathbf{P}(X_t, E_t, E_{1:t-1}) = \mathbf{P}(E_t \mid X_t, E_{1:t-1}) \cdot \mathbf{P}(X_t, E_{1:t-1})$$

And apply the product rule once more to $\mathbf{P}(X_t, E_{1:t-1})$:

$$\mathbf{P}(X_t, E_t, E_{1:t-1}) = \mathbf{P}(E_t \mid X_t, E_{1:t-1}) \cdot \mathbf{P}(X_t \mid E_{1:t-1}) \cdot \mathbf{P}(E_{1:t-1})$$

We also apply the product rule to the denominator:

$$\mathbf{P}(E_t, E_{1:t-1}) = \mathbf{P}(E_t \mid E_{1:t-1}) \cdot \mathbf{P}(E_{1:t-1})$$

Now substitute both back into the fraction:

$$\mathbf{P}(X_t \mid E_{1:t}) = \frac{\mathbf{P}(E_t \mid X_t, E_{1:t-1}) \cdot \mathbf{P}(X_t \mid E_{1:t-1}) \cdot \mathbf{P}(E_{1:t-1})}{\mathbf{P}(E_t \mid E_{1:t-1}) \cdot \mathbf{P}(E_{1:t-1})}$$

The term $\mathbf{P}(E_{1:t-1})$ cancels out:

$$\mathbf{P}(X_t \mid E_{1:t}) = \frac{\mathbf{P}(E_t \mid X_t, E_{1:t-1}) \cdot \mathbf{P}(X_t \mid E_{1:t-1})}{\mathbf{P}(E_t \mid E_{1:t-1})}$$

Since $\frac{1}{\mathbf{P}(E_t \mid E_{1:t-1})}$ is just a normalizing constant, we can simplify our equation by replacing it with α:

$$\mathbf{P}(X_t \mid E_{1:t}) = \alpha\Big(\mathbf{P}(E_t \mid X_t, E_{1:t-1}) \cdot \mathbf{P}(X_t \mid E_{1:t-1})\Big)$$

By the Sensor Markov property, E_t only depends on X_t, simplifying the first term (the sensor model):

$$\mathbf{P}(X_t \mid E_{1:t}) = \alpha\Big(\mathbf{P}(E_t \mid X_t) \cdot \mathbf{P}(X_t \mid E_{1:t-1})\Big)$$

Step 2: Prediction (Law of Total Probability)
Now we need to compute the 1-step prediction $\mathbf{P}(X_t \mid E_{1:t-1})$. We do this by marginalizing over the previous state X_{t-1}:

$$\mathbf{P}(X_t \mid E_{1:t-1}) = \sum_x \mathbf{P}(X_t, X_{t-1} = x \mid E_{1:t-1})$$

To show how the terms split, we expand the conditional joint probability using the definition of conditional probability:

$$\mathbf{P}(X_t, X_{t-1} \mid E_{1:t-1}) = \frac{\mathbf{P}(X_t, X_{t-1}, E_{1:t-1})}{\mathbf{P}(E_{1:t-1})}$$

Applying the product rule to the numerator gives $\mathbf{P}(X_t, X_{t-1}, E_{1:t-1}) = \mathbf{P}(X_t \mid X_{t-1}, E_{1:t-1}) \cdot \mathbf{P}(X_{t-1}, E_{1:t-1})$:

$$\mathbf{P}(X_t, X_{t-1} \mid E_{1:t-1}) = \frac{\mathbf{P}(X_t \mid X_{t-1}, E_{1:t-1}) \cdot \mathbf{P}(X_{t-1}, E_{1:t-1})}{\mathbf{P}(E_{1:t-1})}$$

Notice that $\frac{\mathbf{P}(X_{t-1}, E_{1:t-1})}{\mathbf{P}(E_{1:t-1})}$ is simply the definition of conditional probability for $\mathbf{P}(X_{t-1} \mid E_{1:t-1})$. Therefore:

$$\mathbf{P}(X_t, X_{t-1} \mid E_{1:t-1}) = \mathbf{P}(X_t \mid X_{t-1}, E_{1:t-1}) \cdot \mathbf{P}(X_{t-1} \mid E_{1:t-1})$$

Substituting this conditionalized product rule back into our original marginalization sum gives:

$$\mathbf{P}(X_t \mid E_{1:t-1}) = \sum_x \mathbf{P}(X_t \mid X_{t-1} = x, E_{1:t-1}) \cdot \mathbf{P}(X_{t-1} = x \mid E_{1:t-1})$$

By the First-Order Markov property, the transition to X_t only depends on X_{t-1}, simplifying the first term (the transition model):

$$\mathbf{P}(X_t \mid E_{1:t-1}) = \sum_x \mathbf{P}(X_t \mid X_{t-1} = x) \cdot \mathbf{P}(X_{t-1} = x \mid E_{1:t-1})$$

Step 3: Combine
Substituting the prediction back into the update equation gives the recursive filtering formula.

$$\mathbf{P}(X_t \mid E_{1:t}^{=e}) = \alpha \left(\mathbf{P}(E_t = e_t \mid X_t) \cdot \left(\sum_x \mathbf{P}(X_t \mid X_{t-1} = x) \cdot P(X_{t-1} = x \mid E_{1:t-1}^{=e}) \right) \right)$$

Note

Notice that $P(X_{t-1} = x \mid E_{1:t-1}^{=e})$ is simply the filtering result from the *previous* time step. This means we can compute the belief state recursively!

Def 5.15. We call the inner part of the above expression the **forward** algorithm:

$$\mathbf{P}(X_t \mid E_{1:t}^{=e}) = \alpha \left(\text{FORWARD}(e_t, \mathbf{P}(X_{t-1} \mid E_{1:t-1}^{=e})) \right) = f_{1:t}$$

Filtering in Matrix Notation

$$\mathbf{P}(X_t \mid E_{1:t}^{=e}) = \alpha \left(O_t \cdot T^T \cdot \mathbf{P}(X_{t-1} \mid E_{1:t-1}^{=e}) \right) = f_{1:t}$$

Filtering the Umbrellas

Let's assume we were given the following information:

- $\mathbf{P}(R_0) = \langle 0.5, 0.5 \rangle$
- $P(R_t \mid R_{t-1}) = 0.6, P(\neg R_t \mid \neg R_{t-1}) = 0.8$

- $P(U_t \mid R_t) = 0.9, P(\neg U_t \mid \neg R_t) = 0.85$

From that, we can infer:

- $P(\neg R_t \mid R_{t-1}) = 0.4$
- $P(R_t \mid \neg R_{t-1}) = 0.2$
- $P(\neg U_t \mid R_t) = 0.1$
- $P(U_t \mid \neg R_t) = 0.15$

Hence, we construct the matrices:

$$O_T = \begin{pmatrix} 0.9 & 0 \\ 0 & 0.15 \end{pmatrix} \qquad O_F = \begin{pmatrix} 0.1 & 0 \\ 0 & 0.85 \end{pmatrix} \qquad T = \begin{pmatrix} 0.6 & 0.4 \\ 0.2 & 0.8 \end{pmatrix}$$

Assume that the director carries an umbrella on days 1 and 2, and not on day 3, hence:

$$O_1 = O_2 = O_T = \begin{pmatrix} 0.9 & 0 \\ 0 & 0.15 \end{pmatrix} \qquad O_3 = O_F = \begin{pmatrix} 0.1 & 0 \\ 0 & 0.85 \end{pmatrix}$$

Remember:

$$\mathbf{P}(X_t \mid E_{1:t}^{=e}) = \alpha \left(O_t \cdot T^T \cdot \mathbf{P}(X_{t-1} \mid E_{1:t-1}^{=e}) \right) = f_{1:t}$$

> **Note**
>
> Throughout these calculations, our state vector order is fixed: the top value represents the probability of Rain (R_t) and the bottom value represents the probability of No Rain ($\neg R_t$). i.e. $\begin{pmatrix} P(R_t \mid E_{1:t}) \\ P(\neg R_t \mid E_{1:t}) \end{pmatrix}$. This is because of how we initially constructed the transition and observation matrices.

For $t = 1$, we have:

$$\begin{aligned}
f_{1:1} &= \alpha \left(O_1 \cdot T^T \cdot \mathbf{P}(R_0) \right) \\
&= \alpha \left(\begin{pmatrix} 0.9 & 0 \\ 0 & 0.15 \end{pmatrix} \begin{pmatrix} 0.6 & 0.2 \\ 0.4 & 0.8 \end{pmatrix} \begin{pmatrix} 0.5 \\ 0.5 \end{pmatrix} \right) \\
&= \alpha \left(\begin{pmatrix} 0.9 & 0 \\ 0 & 0.15 \end{pmatrix} \begin{pmatrix} (0.6 \cdot 0.5) + (0.2 \cdot 0.5) \\ (0.4 \cdot 0.5) + (0.8 \cdot 0.5) \end{pmatrix} \right) \\
&= \alpha \left(\begin{pmatrix} 0.9 & 0 \\ 0 & 0.15 \end{pmatrix} \begin{pmatrix} 0.4 \\ 0.6 \end{pmatrix} \right) \\
&= \alpha \begin{pmatrix} 0.9 \cdot 0.4 \\ 0.15 \cdot 0.6 \end{pmatrix} \\
&= \alpha \begin{pmatrix} 0.36 \\ 0.09 \end{pmatrix} \\
&= \frac{1}{0.36 + 0.09} \begin{pmatrix} 0.36 \\ 0.09 \end{pmatrix} = \begin{pmatrix} \frac{0.36}{0.45} \\ \frac{0.09}{0.45} \end{pmatrix} \\
&= \begin{pmatrix} 0.8 \\ 0.2 \end{pmatrix}
\end{aligned}$$

For $t = 2$, we have:

$$\begin{aligned}
f_{1:2} &= \alpha\left(O_2 \cdot T^T \cdot f_{1:1}\right) \\
&= \alpha\left(\begin{pmatrix} 0.9 & 0 \\ 0 & 0.15 \end{pmatrix}\begin{pmatrix} 0.6 & 0.2 \\ 0.4 & 0.8 \end{pmatrix}\begin{pmatrix} 0.8 \\ 0.2 \end{pmatrix}\right) \\
&= \alpha\left(\begin{pmatrix} 0.9 & 0 \\ 0 & 0.15 \end{pmatrix}\begin{pmatrix} (0.6 \cdot 0.8) + (0.2 \cdot 0.2) \\ (0.4 \cdot 0.8) + (0.8 \cdot 0.2) \end{pmatrix}\right) \\
&= \alpha\left(\begin{pmatrix} 0.9 & 0 \\ 0 & 0.15 \end{pmatrix}\begin{pmatrix} 0.52 \\ 0.48 \end{pmatrix}\right) \\
&= \alpha\begin{pmatrix} 0.9 \cdot 0.52 \\ 0.15 \cdot 0.48 \end{pmatrix} \\
&= \alpha\begin{pmatrix} 0.468 \\ 0.072 \end{pmatrix} \\
&= \frac{1}{0.468 + 0.072}\begin{pmatrix} 0.468 \\ 0.072 \end{pmatrix} = \begin{pmatrix} \frac{0.468}{0.54} \\ \frac{0.072}{0.54} \end{pmatrix} \\
&\approx \begin{pmatrix} 0.867 \\ 0.133 \end{pmatrix}
\end{aligned}$$

For $t = 3$, the director did *not* bring an umbrella, so we use $O_3 = O_F$:

$$\begin{aligned}
f_{1:3} &= \alpha\left(O_3 \cdot T^T \cdot f_{1:2}\right) \\
&= \alpha\left(\begin{pmatrix} 0.1 & 0 \\ 0 & 0.85 \end{pmatrix}\begin{pmatrix} 0.6 & 0.2 \\ 0.4 & 0.8 \end{pmatrix}\begin{pmatrix} 0.867 \\ 0.133 \end{pmatrix}\right) \\
&= \alpha\left(\begin{pmatrix} 0.1 & 0 \\ 0 & 0.85 \end{pmatrix}\begin{pmatrix} (0.6 \cdot 0.867) + (0.2 \cdot 0.133) \\ (0.4 \cdot 0.867) + (0.8 \cdot 0.133) \end{pmatrix}\right) \\
&\approx \alpha\left(\begin{pmatrix} 0.1 & 0 \\ 0 & 0.85 \end{pmatrix}\begin{pmatrix} 0.547 \\ 0.453 \end{pmatrix}\right) \\
&= \alpha\begin{pmatrix} 0.1 \cdot 0.547 \\ 0.85 \cdot 0.453 \end{pmatrix} \\
&\approx \alpha\begin{pmatrix} 0.055 \\ 0.385 \end{pmatrix} \\
&= \frac{1}{0.055 + 0.385}\begin{pmatrix} 0.055 \\ 0.385 \end{pmatrix} = \begin{pmatrix} \frac{0.055}{0.440} \\ \frac{0.385}{0.440} \end{pmatrix} \\
&\approx \begin{pmatrix} 0.125 \\ 0.875 \end{pmatrix}
\end{aligned}$$

At day 3, there is a 12.5% chance it is raining, and an 87.5% it is not raining.

Prediction

Idea

Prediction is filtering without new evidence, i.e. we can use filtering until t, and then continue as follows:

$$\mathbf{P}(X_{t+k} \mid E_{1:t}^{=e}) = \sum_x \mathbf{P}(X_{t+k} \mid X_{t+k-1} = x) \cdot P(X_{t+k-1} = x \mid E_{1:t}^{=e})$$

Note

By recursively applying this formula k times, starting from our current filtered belief state $\mathbf{P}(X_t \mid E_{1:t}^{=e})$, we can predict k steps into the future.

Prediction Matrix Notation

$$\mathbf{P}(X_{t+k} \mid E_{1:t}^{=e}) = T^T \cdot \mathbf{P}(X_{t+k-1} \mid E_{1:t}^{=e})$$

Smoothing Derivation

Idea

Smoothing computes $\mathbf{P}(X_{t-k} \mid E_{1:t}^{=e})$ for $k > 0$. The intuition is to split the evidence into two independent parts: the past/current evidence up to the target state ($E_{1:t-k}$) and the future evidence after the target state ($E_{t-k+1:t}$). We then compute a forward filtering pass up to time $t-k$, and a backward pass from time t down to $t-k$.

Step 1: Divide the Evidence (Product Rule)
We want to find our belief about state X_{t-k} given all evidence $E_{1:t}$. Let's divide the evidence at the target state: $E_{1:t} = E_{1:t-k}, E_{t-k+1:t}$

$$\mathbf{P}(X_{t-k} \mid E_{1:t-k}, E_{t-k+1:t}) = \frac{\mathbf{P}(X_{t-k}, E_{t-k+1:t}, E_{1:t-k})}{\mathbf{P}(E_{t-k+1:t}, E_{1:t-k})}$$

We apply the product rule to the numerator ($\mathbf{P}(A, B, C) = \mathbf{P}(A \mid B, C) \cdot \mathbf{P}(B \mid C) \cdot \mathbf{P}(C)$) and denominator ($\mathbf{P}(A, B) = \mathbf{P}(A \mid B) \cdot \mathbf{P}(B)$), keeping $E_{1:t-k}$ as the conditioning context:

$$\mathbf{P}(X_{t-k} \mid E_{1:t}) = \frac{\mathbf{P}(E_{t-k+1:t} \mid X_{t-k}, E_{1:t-k}) \cdot \mathbf{P}(X_{t-k} \mid E_{1:t-k}) \cdot \mathbf{P}(E_{1:t-k})}{\mathbf{P}(E_{t-k+1:t} \mid E_{1:t-k}) \cdot \mathbf{P}(E_{1:t-k})}$$

we cancel out $\mathbf{P}(E_{1:t-k})$ and replace the normalizing denominator with α:

$$\mathbf{P}(X_{t-k} \mid E_{1:t}) = \alpha\Big(\mathbf{P}(E_{t-k+1:t} \mid X_{t-k}, E_{1:t-k}) \cdot \mathbf{P}(X_{t-k} \mid E_{1:t-k})\Big)$$

Step 2: Apply Conditional Independence
By the Markov property, if we lock in our knowledge of the target state X_{t-k}, then the future evidence $E_{t-k+1:t}$ becomes independent of the past evidence $E_{1:t-k}$. (We say the state X_{t-k} "d-separates" the past and future). Thus, we drop the past evidence from the conditional:

$$\mathbf{P}(E_{t-k+1:t} \mid X_{t-k}, E_{1:t-k}) = \mathbf{P}(E_{t-k+1:t} \mid X_{t-k})$$

Substituting this back gives us the foundational equation for smoothing:

$$\mathbf{P}(X_{t-k} \mid E_{1:t}) = \alpha\Big(\underbrace{\mathbf{P}(E_{t-k+1:t} \mid X_{t-k})}_{\text{backward message } b_{t-k+1:t}} \cdot \underbrace{\mathbf{P}(X_{t-k} \mid E_{1:t-k})}_{\text{forward message } f_{1:t-k}}\Big)$$

Notice that the second term is exactly our filtering formula result! We call this the **forward message** $f_{1:t-k}$. The first term is a new concept, denoting the probability of observing all upcoming future evidence given the target state. We formally define this as the **backward message** $b_{t-k+1:t}$.

Step 3: Deriving the Backward Message Recursion
Let $b_{m:t} = \mathbf{P}(E_{m:t} \mid X_{m-1})$ be a generic backward message representing future evidence $E_{m:t}$ given the state immediately preceding it (X_{m-1}). We want to recursively express this in terms of the next state down the line, X_m. We do this by marginalizing over all possible values of X_m:

$$\mathbf{P}(E_{m:t} \mid X_{m-1}) = \sum_{x \in \text{dom}(X)} \mathbf{P}(E_{m:t}, X_m = x \mid X_{m-1})$$

To split the inner joint probability, we apply the definition of conditional probability ($\mathbf{P}(A, B \mid C) = \frac{\mathbf{P}(A,B,C)}{\mathbf{P}(C)}$) and then use the product rule on the numerator ($\mathbf{P}(A, B, C) = \mathbf{P}(A \mid B, C) \cdot \mathbf{P}(B, C)$):

$$\begin{aligned}\mathbf{P}(E_{m:t}, X_m = x \mid X_{m-1}) &= \frac{\mathbf{P}(E_{m:t}, X_m = x, X_{m-1})}{\mathbf{P}(X_{m-1})} \\ &= \frac{\mathbf{P}(E_{m:t} \mid X_m = x, X_{m-1}) \cdot \mathbf{P}(X_m = x, X_{m-1})}{\mathbf{P}(X_{m-1})}\end{aligned}$$

Since $\frac{\mathbf{P}(X_m=x, X_{m-1})}{\mathbf{P}(X_{m-1})} = \mathbf{P}(X_m = x \mid X_{m-1})$ (by definition of conditional probability again), substituting this back gives us the conditionalized product rule:

$$\mathbf{P}(E_{m:t} \mid X_{m-1}) = \sum_{x \in \text{dom}(X)} \mathbf{P}(E_{m:t} \mid X_m = x, X_{m-1}) \cdot \mathbf{P}(X_m = x \mid X_{m-1})$$

Next, we split the future evidence $E_{m:t}$ into purely the current observation E_m and the remaining future observations $E_{m+1:t}$:

$$= \sum_{x \in \text{dom}(X)} \mathbf{P}(E_m, E_{m+1:t} \mid X_m = x, X_{m-1}) \cdot \mathbf{P}(X_m = x \mid X_{m-1})$$

First, by the Markov property, future observations ($E_{m:t}$) are conditionally independent of the past state X_{m-1} given the current state X_m. In other words, knowing the current state completely shields the future from the past. We can thus safely drop X_{m-1} from the conditioning:

$$= \sum_{x \in \text{dom}(X)} \mathbf{P}(E_m, E_{m+1:t} \mid X_m = x) \cdot \mathbf{P}(X_m = x \mid X_{m-1})$$

Next, we must separate the joint probability $\mathbf{P}(E_m, E_{m+1:t} \mid X_m = x)$. Unpacking this completely using the definition of conditional probability ($\mathbf{P}(A, B \mid C) = \frac{\mathbf{P}(A,B,C)}{\mathbf{P}(C)}$) gives:

$$\mathbf{P}(E_m, E_{m+1:t} \mid X_m = x) = \frac{\mathbf{P}(E_{m+1:t}, E_m, X_m = x)}{\mathbf{P}(X_m = x)}$$

We apply the standard product rule to the numerator ($\mathbf{P}(A, B, C) = \mathbf{P}(A \mid B, C) \cdot \mathbf{P}(B, C)$):

$$= \frac{\mathbf{P}(E_{m+1:t} \mid E_m, X_m = x) \cdot \mathbf{P}(E_m, X_m = x)}{\mathbf{P}(X_m = x)}$$

And applying the definition of conditional probability once more on the remaining fraction ($\frac{\mathbf{P}(A,B)}{\mathbf{P}(B)} = \mathbf{P}(A \mid B)$):

$$= \mathbf{P}(E_{m+1:t} \mid E_m, X_m = x) \cdot \mathbf{P}(E_m \mid X_m = x)$$

Finally, we recognize that the current evidence E_m and future evidence $E_{m+1:t}$ are conditionally independent given the current state X_m (because the sensor model depends solely on the immediate state). Due to this conditional independence, knowing E_m provides no extra information about $E_{m+1:t}$ if we already know X_m, so we can drop E_m from the second term:

$$\mathbf{P}(E_m, E_{m+1:t} \mid X_m = x) = \mathbf{P}(E_m \mid X_m = x) \cdot \mathbf{P}(E_{m+1:t} \mid X_m = x)$$

Substituting this fully separated expression back into our main summation yields:

$$= \sum_{x \in \text{dom}(X)} \mathbf{P}(E_m \mid X_m = x) \cdot \mathbf{P}(E_{m+1:t} \mid X_m = x) \cdot \mathbf{P}(X_m = x \mid X_{m-1})$$

Rearranging these terms, we see the recursive relationship emerges beautifully:

$$b_{m:t} = \sum_{x \in \text{dom}(X)} \underbrace{\mathbf{P}(E_m \mid X_m = x)}_{\text{sensor model}} \cdot \underbrace{\mathbf{P}(E_{m+1:t} \mid X_m = x)}_{\text{next backward msg } b_{m+1:t}} \cdot \underbrace{\mathbf{P}(X_m = x \mid X_{m-1})}_{\text{transition model}}$$

If we substitute our target $m = t - k + 1$, we get the recursive definition:

$$b_{t-k+1:t} = \sum_{x \in \text{dom}(X)} \mathbf{P}(E_{t-k+1} = e_{t-k+1} \mid X_{t-k+1} = x) \cdot b_{t-k+2:t} \cdot \mathbf{P}(X_{t-k+1} = x \mid X_{t-k})$$

Note

As a starting point for the backward recursion at time t, we let $b_{t+1:t}$ be the explicit uniform vector with 1 in every component, since there is no evidence after t.

Backward Algorithm Matrix Notation

The generic backward message recursion algorithm represented as matrix multiplication is:

$$b_{t-k+1:t} = T \cdot O_{t-k+1} \cdot b_{t-k+2:t}$$

Notice that we evaluate using T, not T^T like we do in forward filtering.

Smoothing Final Algorithm Matrix Notation

Combining the forward and backward passes, the full **smoothing algorithm** becomes:

$$\mathbf{P}(X_{t-k} \mid E_{1:t}^{=e}) = \alpha\left(f_{1:t-k} \times b_{t-k+1:t}\right)$$

Where $\times$ denotes component-wise (Hadamard) vector multiplication.

Smoothing Umbrellas

Using filtering we already calculated

$$f_{1:1} = \begin{pmatrix}0.8\\0.2\end{pmatrix}, f_{1:2} = \begin{pmatrix}0.867\\0.133\end{pmatrix}, f_{1:3} = \begin{pmatrix}0.125\\0.875\end{pmatrix}$$

Let's compute our smoothed belief for day 1 ($k = 2$):

$$\mathbf{P}(R_1 \mid E_{1:3}) = \mathbf{P}(R_1 \mid U_1 = T, U_2 = T, U_3 = F) = \alpha(f_{1:1} \times b_{2:3})$$

For that, we need to compute $b_{2:3}$ and $b_{3:3}$ using the backward algorithm: $b_{m:t} = T \cdot O_m \cdot b_{m+1:t}$. The base case at time $t = 3$ is $b_{4:3} = \begin{pmatrix}1\\1\end{pmatrix}$.

1. Calculate backward message $b_{3:3}$

$$\begin{aligned}
b_{3:3} &= T \cdot O_3 \cdot b_{4:3} \\
&= \begin{pmatrix}0.6 & 0.4\\0.2 & 0.8\end{pmatrix}\begin{pmatrix}0.1 & 0\\0 & 0.85\end{pmatrix}\begin{pmatrix}1\\1\end{pmatrix} \\
&= \begin{pmatrix}0.6 & 0.4\\0.2 & 0.8\end{pmatrix}\begin{pmatrix}0.1\\0.85\end{pmatrix} \\
&= \begin{pmatrix}(0.6 \cdot 0.1) + (0.4 \cdot 0.85)\\(0.2 \cdot 0.1) + (0.8 \cdot 0.85)\end{pmatrix} = \begin{pmatrix}0.06 + 0.34\\0.02 + 0.68\end{pmatrix} = \begin{pmatrix}0.4\\0.7\end{pmatrix}
\end{aligned}$$

2. Calculate backward message $b_{2:3}$

$$\begin{aligned}
b_{2:3} &= T \cdot O_2 \cdot b_{3:3} \\
&= \begin{pmatrix}0.6 & 0.4\\0.2 & 0.8\end{pmatrix}\begin{pmatrix}0.9 & 0\\0 & 0.15\end{pmatrix}\begin{pmatrix}0.4\\0.7\end{pmatrix} \\
&= \begin{pmatrix}0.6 & 0.4\\0.2 & 0.8\end{pmatrix}\begin{pmatrix}0.36\\0.105\end{pmatrix} \\
&= \begin{pmatrix}(0.6 \cdot 0.36) + (0.4 \cdot 0.105)\\(0.2 \cdot 0.36) + (0.8 \cdot 0.105)\end{pmatrix} = \begin{pmatrix}0.216 + 0.042\\0.072 + 0.084\end{pmatrix} = \begin{pmatrix}0.258\\0.156\end{pmatrix}
\end{aligned}$$

3. Combine Forward and Backward messages

Now we take our forward filtered probability for day 1 ($f_{1:1}$) and element-wise multiply ($\times$) it with the backward message

from the future ($b_{2:3}$):

$$\begin{aligned}\mathbf{P}(R_1 \mid E_{1:3}) &= \alpha\,(f_{1:1} \times b_{2:3}) \\ &= \alpha\left(\begin{pmatrix}0.8\\0.2\end{pmatrix} \times \begin{pmatrix}0.258\\0.156\end{pmatrix}\right) \\ &= \alpha\begin{pmatrix}0.8 \cdot 0.258\\0.2 \cdot 0.156\end{pmatrix} \\ &= \alpha\begin{pmatrix}0.2064\\0.0312\end{pmatrix}\end{aligned}$$

Calculating the normalization constant $\alpha = \frac{1}{0.2064+0.0312} = \frac{1}{0.2376} \approx 4.2088$:

$$\begin{aligned}\mathbf{P}(R_1 \mid E_{1:3}) &\approx 4.2088 \cdot \begin{pmatrix}0.2064\\0.0312\end{pmatrix} \\ &\approx \begin{pmatrix}0.8687\\0.1313\end{pmatrix}\end{aligned}$$

Note

Compare this with the original filtered probability for day 1 ($f_{1:1} = \langle 0.8, 0.2\rangle$). Given that the director brought an umbrella on day 2, it made sense in hindsight that it was even more likely to be raining on day 1 (86.87% vs 80%).

Def 5.16. Dynamic Bayesian Networks (DBN): A Bayesian Network $\mathcal{D}$ is called **dynamic** iff its random variables are indexed by a time structure. We assume $\mathcal{D}$ is:

- **time sliced**, i.e. that the **time slices** $\mathcal{D}_t$ - the subgraphs of t-indexed random variables and the edges between them - are isomorphic.
- a stationary Markov chain, i.e. that variables X_t can only have parents in $\mathcal{D}_t$ and $\mathcal{D}_{t-1}$

Observation

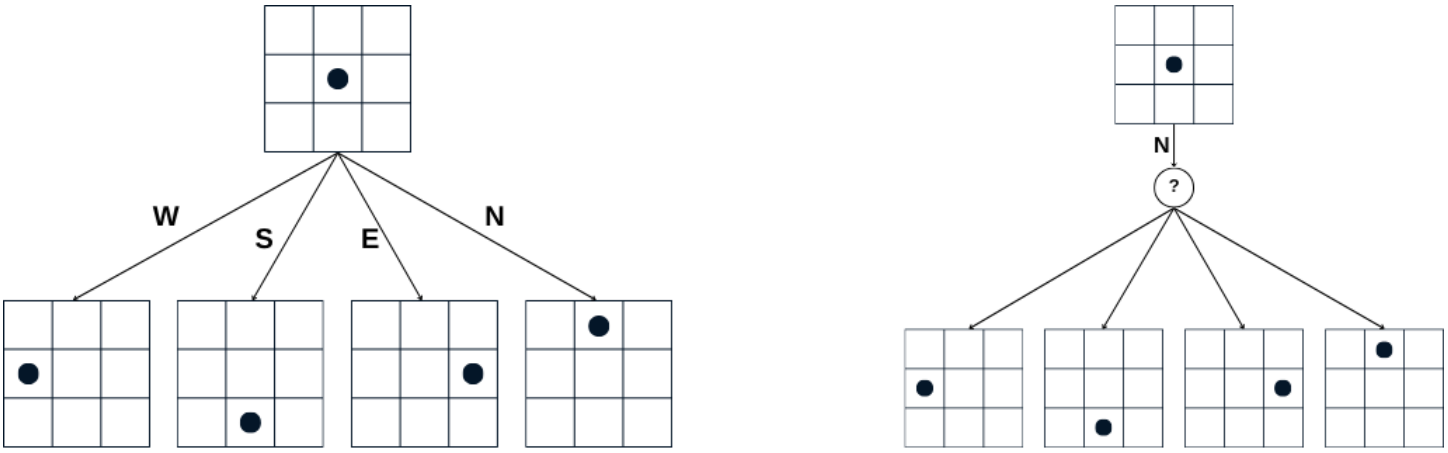
- Every HMM is a single-variable DBN *(trivially)*
- Every DBN can be turned into an HMM *(combine variables into tuple $\rightsquigarrow$ lose information about dependencies)*
- DBNs have sparse dependencies $\rightsquigarrow$ exponentially fewer parameters

6 Markov Decision Processes

Motivation

In deterministic environments, the next state is completely determined by the current state and the chosen action. For these problems, we define a corresponding deterministic search problem where the objective is to find a solution consisting of a specific sequence of actions—an optimal plan—leading from the initial state to a goal state.
However, in a stochastic environment, the next state is **not** uniquely determined by the current state and action. In these stochastic search problems, the agent controls the action but lacks direct control over the environment's transitions. While the exact outcome of an action is unknown, the agent knows the probabilities of transitioning to various states given a specific action.
Consequently, a fixed sequence of actions is no longer sufficient. Instead, the goal is to find an **optimal policy**, π^*, which is a function $\pi^* : \mathcal{S} \to \mathcal{A}$ that dictates the best action for any given state. By following this policy, the agent navigates the environment's uncertainty to reach a goal state efficiently.

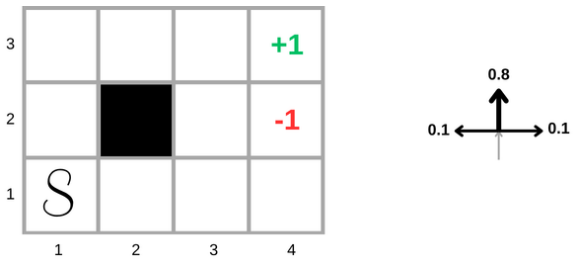
Example 6.1. Consider a grid world where the agent can go west, east, north, or south. Notice the difference between deterministic and stochastic environments:



Note

We are still assuming fully observable environments

Def 6.1. MDP Running Example: We will consider the following fully observable 4×3 grid world with stochastic actions:



Def 6.2. MDP: A **Markov Decision Process (MDP)** is the structure $\langle \mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{I}, R \rangle$ where:

- $\mathcal{S}$: the set of states
- $\mathcal{I}$: the set of initial states. Since we are in a fully observable environment, $\mathcal{I} = \{s_0\}$.
- $\mathcal{A}$: the set of actions, with $\mathcal{A}_s$ denoting the set of action that can be taken from state s.
- $\mathcal{T}$: a transition model $\mathcal{T}(s, a) = \mathbf{P}(\mathcal{S} \mid s, a)$
- R: a **reward function** $R : \mathcal{S} \to \mathbb{R}$ (we call $R(s)$ a **reward**)

Idea

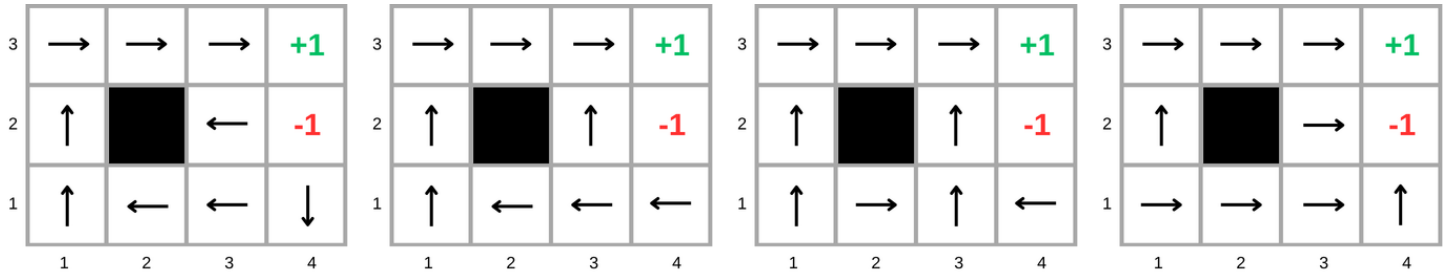
We use the rewards as a utility function: the goal is to choose actions such that the expected comulative rewards for the foreseeable future is maximized.

Note

In MDPs, the aim is to find an **optimal policy** $\pi(s)$, which tells us the best action for every possible state s. (because we can't predict where we might end up, we need to consider all states)

Def 6.3. Policy: A **policy** π for an MDP is a function mapping each state $s \in \mathcal{S}$ to an action $a \in \mathcal{A}_s$. An **optimal policy** is a policy that maximizes the expected total rewards.

Example 6.2.



(a) $R(s) = -0.01$ (b) $R(s) = -0.03$ (c) $R(s) = -0.4$ (d) $R(s) = -2$

Figure 2: Different optimal policies for different reward functions

For $R(s) = -0.01$, we have a small negative reward for each state. Hence, each action will cost a little bit and not too much. Therefore, in state $(3, 2)$, the optimal action is to go west where there is a 80% chance we stay where we are. We cannot go north since there is a 10% of ending up in the pit (-1). Because the action costs only 0.01, the agent prefers to bang his head to the wall until by chance goes north or south. Similarly, in $(4, 1)$.

For $R(s) = -0.03$, the action cost is a bit high. The agent prefer to take the chance of going north (in $(3, 2)$) and west in $(4, 1)$ regardless of the 10% probability of falling in the pit.

For $R(s) = -0.4$, the cost now is much higher, the agent will try to finish as soon as possible.

For $R(s) = -2$, life is very painful, the agent might commit suicide to alleviate the pain.

Question

Assume we have two sequences of three actions, the first one will collect the rewards $1, 2, 2$, and second one will collect $2, 3, 4$. Which sequence should the agent choose?

Answer

Well, since we agreed that we want to maximize the total sum of rewards, obviously we should prefer $2, 3, 4$ because they sum to $9 > 5$.

Question

What about $0, 0, 1$ and $1, 0, 0$?

Observation

They have the same total sum of rewards.

Idea

- It is reasonable to maximize the sum of rewards
- But it is also reasonable to prefer rewards **now** over rewards **later**

Answer

Then the agent should prefer $1, 0, 0$ over $0, 0, 1$

Question

But how can we model this into the agent's brain?

Answer

We should let the values of the rewards decay exponentially over time

Def 6.4. We call preferences on reward sequences **stationary** iff

$$[r, r_0, r_1, r_2, \cdots] \succ [r, r'_0, r'_1, r'_2, \cdots] \Leftrightarrow [r_0, r_1, r_2, \cdots] \succ [r'_0, r'_1, r'_2, \cdots]$$

Theorem 6.1. For stationary preferences, there are only two ways to combine rewards over time:

- **additive rewards**: $U([s_0, s_1, s_2, \cdots]) = R(s_0) + R(s_1) + R(s_2) + \cdots$
- **discounted rewards**: $U([s_0, s_1, s_2, \cdots]) = R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) + \cdots$ where $0 \leq \gamma \leq 1$ is called **discount factor**

Question

Given discounted rewards with $\gamma = 0.5$, what is the utility of the rewards sequences $[1, 2, 3]$, and $[3, 2, 1]$?

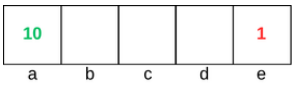
Answer

- $U([1, 2, 3]) = 1 + 0.5 \cdot 2 + 0.25 \cdot 3 = 2.75$
- $U([3, 2, 1]) = 3 + 0.5 \cdot 2 + 0.25 \cdot 1 = 4.25$

$U([3, 2, 1]) > U([1, 2, 3])$ (the sooner the better)

Dicounting Exercise

Consider the following world:

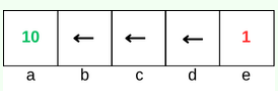


- $\mathcal{S} = \{a, b, c, d, e\}$
- $\mathcal{A}_s = \{E, W\} \quad \forall s \in \{b, c, d\}$ (East and West)
- $\mathcal{T}$ is deterministic
- If we are in a, we get reward 10.
- If we are in e, we get reward 1.
- $R(s) = 0 \quad \forall s \in \{b, c, d\}$

Question

For $\gamma = 1$ (no discounted rewards), what is the optimal policy ?

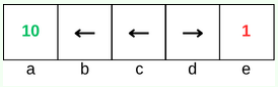
Answer



Question

For $\gamma = 0.1$, what is the optimal policy ?

Answer



Question

For which γ are West and East equally good when agent in state d?

Answer

- Going East: $0 + \gamma \cdot 1$
- Going West: $0 + \gamma \cdot 0 + \gamma^2 \cdot 0 + \gamma^3 \cdot 10$
- $\gamma = 10 \cdot \gamma^3$
- $1 = 10 \cdot \gamma^2$
- $\gamma^2 = \frac{1}{10} \rightsquigarrow \gamma = \frac{1}{\sqrt{10}}$

What if we do not have terminal states ?

Observation

If we are using additive rewards and we do not have terminal states, we can get infinite reward!

Idea

We have two solutions to this problem:

- Finite Horizon: we terminate after a fixed T steps. $\rightsquigarrow$ Gives non-stationary policies (π depends on the time left)
- Absorbing States: for any policy π, the agent eventually dies with probability $= 1$. $\rightsquigarrow$ Expected utility of every state is finite

Observation

A better solution is to use discounted rewards with $0 \leq \gamma < 1$.

Proof.

- Let $R_{\max}$ be the maximum reward the agent can get.
- $U([r_0, r_1, r_2, \cdots, r_\infty]) = \sum_{t=0}^{\infty} \gamma^t r_t$
- $\sum_{t=0}^{\infty} \gamma^t r_t \leq \sum_{t=0}^{\infty} \gamma^t R_{\max}$
- $\sum_{t=0}^{\infty} \gamma^t R_{\max}$ is a convergent geometric series (since $\gamma < 1$)
- $\sum_{t=0}^{\infty} \gamma^t R_{\max} = \frac{R_{\max}}{1-\gamma}$
- Therefore, $U([r_0, r_1, r_2, \cdots, r_\infty]) \leq \frac{R_{\max}}{1-\gamma}$

□

Remember - The Geometric Series

Def 6.5. A **geometric series** is the sum of an infinite sequence where each term is found by multiplying the previous one by a fixed, non-zero constant r, called the **common ratio**. Given an initial term $a \neq 0$, the series is expressed as:

$$\sum_{n=0}^{\infty} ar^n = a + ar + ar^2 + ar^3 + \dots$$

To determine if the infinite sum exists, we define the **k-th partial sum**, S_k, as the sum of the first k terms:

$$S_k = \sum_{n=0}^{k-1} ar^n = a + ar + ar^2 + \dots + ar^{k-1}$$

Through algebraic manipulation, this can be written in closed form as:

$$S_k = a\left(\frac{1-r^k}{1-r}\right) \quad \text{for } r \neq 1$$

The infinite series converges if the limit of the partial sums exists and is finite:

$$S = \lim_{k\to\infty} S_k$$

If the magnitude of the ratio is less than one ($|r| < 1$), then the term r^k vanishes as k approaches infinity:

$$\lim_{k\to\infty} r^k = 0$$

Substituting this into the partial sum formula yields the **sum of the infinite geometric series**:

$$S = \lim_{k\to\infty} a\left(\frac{1-r^k}{1-r}\right) = a\left(\frac{1-0}{1-r}\right) = \frac{a}{1-r}$$

- If $|r| < 1$, the series **converges** to $\frac{a}{1-r}$.
- If $|r| \geq 1$, the series **diverges** (the sum does not approach a finite limit).

Def 6.6. Given a sequence of state $S = s_0, s_1, s_2, \cdots$ and a discount factor $0 \leq \gamma < 1$, the **utility** of the sequence is :

$$U(S) = \sum_{t=0}^{\infty} \gamma^t R(s_t)$$

Def 6.7. Given a policy π and a starting state s_0. We define a random variable $S^{\pi}_{s_0}$ which represent the sequence of states resulting from executing π at every state starting from s_0 (because the environment is stochastic, we do not know the exact sequence, hence the random variable treatment).

Def 6.8. The **expected utility** obtained by executing the policy π starting in s_0 is given by:

$$U^{\pi}(s_0) := \text{EU}(S^{\pi}_{s_0})$$

Def 6.9. The **optimal policy** π^* is a policy that maximizes the expected utility for the agent:

$$\pi^*_{s_0} := \arg\max_{\pi} U^{\pi}(s_0)$$

Observation

While we initially define it relative to a starting state s_0. It is a fundamental property of MDPs that there exists a single policy π^* that is optimal for **all** starting states simultaneously.

Def 6.10. The **true utility** $U(s)$ of a state is the expected utility obtained by following the optimal policy starting in that state:

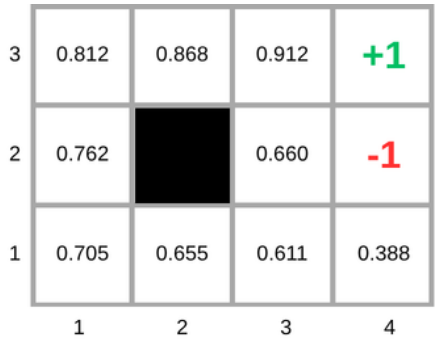
$$U(s) = U^{\pi^*}(s)$$

Def 6.11. Once these utilities are known, the optimal policy can be extracted locally using the MEU principle. Choosing the best action is just maximizing the expected utility of the immediate successor states:

$$\pi^*(s) = \arg\max_{a\in\mathcal{A}_s} \left(\sum_{s'} P(s' \mid s, a) \cdot U(s')\right)$$

Given the true utilities, we can compute the optimal policy and vice versa.

Example 6.3. Assume we know the utilities of our grid world:



What is the optimal action for state $(3, 4)$?

$$\pi^*(s) = \arg\max_{a \in \mathcal{A}_s} \left(\sum_{s'} P(s' \mid s, a) \cdot U(s') \right)$$

$$\pi^*((3,4)) = \arg\max_{a \in \{\text{N, S, E, W}\}} \left(\sum_{s'} P(s' \mid s, a) \cdot U(s') \right)$$

For $a = N$, we have $s' \in \{(3,2), (4,1), (2,1)\}$

$$\sum_{s'} P(s' \mid s, a) \cdot U(s') = 0.8 \cdot 0.660 + 0.1 \cdot 0.388 + 0.1 \cdot 0.655 = 0.6326$$

For $a = S$

$$\sum_{s'} P(s' \mid s, a) \cdot U(s') = 0.8 \cdot 0.611 + 0.1 \cdot 0.388 + 0.1 \cdot 0.655 = 0.5931$$

For $a = E$

$$\sum_{s'} P(s' \mid s, a) \cdot U(s') = 0.8 \cdot 0.388 + 0.1 \cdot 0.660 + 0.1 \cdot 0.611 = 0.4375$$

For $a = W$

$$\sum_{s'} P(s' \mid s, a) \cdot U(s') = 0.8 \cdot 0.655 + 0.1 \cdot 0.611 + 0.1 \cdot 0.660 = 0.6511$$

Therefore:

$$\pi^*((3,4)) = \arg\max_{a \in \{\text{N, S, E, W}\}} \left(\sum_{s'} P(s' \mid s, a) \cdot U(s') \right) = W$$

Question

How to compute these utilities without knowing the optimal policy?

Observation

The expected sum of rewards = current reward + $\gamma\cdot$ expected reward sum after taking the optimal action

Bellman Equation

Theorem 6.2.

$$U(s) := R(s) + \gamma \cdot \max_{a \in \mathcal{A}_s} \sum_{s'} U(s') \cdot P(s' \mid s, a)$$

Example 6.4. Assume our grid world with $R(s) = -0.04$. Then the utility for, e.g. $(1,1)$ is:

$$U(s) := R(s) + \gamma \cdot \max_{a \in \mathcal{A}_s} \sum_{s'} U(s') \cdot P(s' \mid s, a)$$

$$U((1,1)) := R((1,1)) + \gamma \cdot \max_{a \in \{N,S,E,W\}} \sum_{s'} U(s') \cdot P(s' \mid (1,1), a)$$

$$:= -0.04 + \gamma \cdot \max_{a \in \{N,S,E,W\}} \sum_{s'} U(s') \cdot P(s' \mid (1,1), a)$$

$$U(1,1) = -0.04 + \gamma \cdot \max\{$$

$0.8U(1,2) + 0.1U(2,1) + 0.1U(1,1),$	*North*
$0.9U(1,1) + 0.1U(1,2),$	*West*
$0.9U(1,1) + 0.1U(2,1),$	*South*
$0.8U(2,1) + 0.1U(1,2) + 0.1U(1,1)\}$	*East*

The Computational Challenge: The Bellman equations are inherently **nonlinear** due to the max operator. This coupling, combined with a high-dimensional state space, renders direct analytical solutions via standard Linear Algebra **intractable**.

Idea

Solve iteratively using e.g., dynamic programming:

- Start with arbitrary utility values
- At every iteration, update the utility values for each state using the values from the previous iteration
- Keep iterating until convergence $\rightsquigarrow$ every s, eventually will have the true $U(s)$

Def 6.12.

Value Iteration

$$U_{i+1}(s) \leftarrow R(s) + \gamma \cdot \max_{a \in \mathcal{A}_s} \sum_{s'} U_i(s') \cdot P(s' \mid s, a)$$

Terminate when $\max_{s \in S} |U_i(s) - U_{i+1}(s)|$ is small enough

Def 6.13. The **maximum norm** is defined as $\|U\| = \max_s |U(s)|$. Consequently, $\|U - V\|$ represents the maximum difference between U and V.

Example 6.5. Consider a state space $S = \{s_1, s_2, s_3\}$ with the following utility values:

$$U = \begin{bmatrix} 2 \\ -5 \\ 4 \end{bmatrix}$$

To find the maximum norm $\|U\|$, we calculate the absolute value for each state and select the largest:

$$\begin{aligned} \|U\| &= \max_{s \in S} |U(s)| \\ &= \max(|2|, |-5|, |4|) \\ &= \max(2, 5, 4) \\ &= 5 \end{aligned}$$

The maximum norm is 5 because -5 is the value furthest from zero in this vector.

Example 6.6. Consider two utility vectors U and V for the state space $S = \{s_1, s_2, s_3\}$:

$$U = \begin{bmatrix} 10 \\ 2 \\ 5 \end{bmatrix}, \quad V = \begin{bmatrix} 7 \\ 2 \\ 8 \end{bmatrix}$$

To find $\|U - V\|$, we find the maximum absolute difference across all states:

$$\begin{aligned}\|U - V\| &= \max_{s \in S} |U(s) - V(s)| \\ &= \max(|10 - 7|, |2 - 2|, |5 - 8|) \\ &= \max(|3|, |0|, |-3|) \\ &= \max(3, 0, 3) \\ &= 3\end{aligned}$$

This value represents the largest "error" or shift between the two approximations.

Theorem 6.3. Let U^t and U^{t+1} be successive approximations to the true utility vector U during value iteration. Then for any two approximations U^t, V^t we have:

$$\|U^{t+1} - V^{t+1}\| \leq \gamma \|U^t - V^t\|$$

where:

- $U, V \in \mathbb{R}^{|S|}$ are vectors where each component $U(s)$ represents the expected long-term reward from state s.
- $\|U - V\|$ is the "distance" between these two utility estimates.

$\rightsquigarrow$ The update rule ensures that U and V move closer together at a rate defined by the discount factor γ.

Note

- That means, any distinct approximations get closer to each other over time.
- In our case, they get closer to the true utility vector U
- Consequently, value iteration converges to a unique, stable, optimal solution

Theorem 6.4. If $\left\|U^{t+1} - U^t\right\| < \epsilon$, then:

$$\left\|U^{t+1} - U\right\| < \frac{2\epsilon\gamma}{1 - \gamma}$$

Note

- In practice, we cannot check the distance to the true utility vector U because it is unknown. We can only check the change between steps: $\|U^{t+1} - U^t\|$.
- Theorem 6.3 guarantees that if the change in the last step is less than ϵ, then the absolute error from the optimal solution is no larger than $\frac{2\epsilon\gamma}{1-\gamma}$.
- As the discount factor γ approaches 1, the denominator $(1 - \gamma)$ becomes very small, which makes the error bound much larger. This reflects how "far-sighted" problems are harder to converge.
- This allows me to pre-calculate a stopping threshold ϵ to ensure the final result is within a desired accuracy range.

Idea

To bridge the gap between theory and code, we use a specific termination condition in the value iteration algorithm:

- We want the final error to be small, specifically $\|U^{t+1} - U\| < \epsilon$.
- During the loop, we track $\delta = \|U^{t+1} - U^t\|$, which is the change between the current and previous guess.
- Based on Theorem 6.4, if we want the true error to be less than ϵ, we must iterate until our observed change δ satisfies:

$$\delta < \epsilon \frac{(1 - \gamma)}{\gamma}$$

- As the discount factor γ gets closer to 1, the value $(1 - \gamma)$ becomes very small. This forces the algorithm to wait until δ is extremely tiny, ensuring that even small "leftover" changes don't accumulate into a large error over an infinite time horizon.

Def 6.14. The **Value Iteration Algorithm** is given by Algorithm 1

Algorithm 1 Value Iteration

function VALUE-ITERATION(mdp, ϵ)
  inputs: mdp, an MDP with states S, actions $A(s)$, transition model $P(s' \mid s, a)$, rewards $R(s)$, discount γ
  inputs: ϵ, the maximum error allowed in the utility of any state
  local variables: U, U', vectors of utilities for states in S, initially zero
  local variables: δ, the maximum change in the utility of any state in an iteration
  repeat
    $U \leftarrow U'$
    $\delta \leftarrow 0$
    for each state s in S **do**
      $U'[s] \leftarrow R(s) + \gamma \cdot \max_{a \in A(s)} \sum_{s'} P(s' \mid s, a) U[s']$
      if $|U'[s] - U[s]| > \delta$ **then**
        $\delta \leftarrow |U'[s] - U[s]|$
      end if
    end for
  until $\delta < \epsilon(1-\gamma)/\gamma$
  return U
end function

Example 6.7. Assume again our grid world. We will use $\gamma = 1$ (additive rewards) for simplicity and $R(s) = -0.04$. We start with $U_0(s) = 0$, we want to iterate twice to calculate $U_1((3,2))$, $U_1((3,3))$, $U_2((3,2))$ and $U_2((3,3))$:

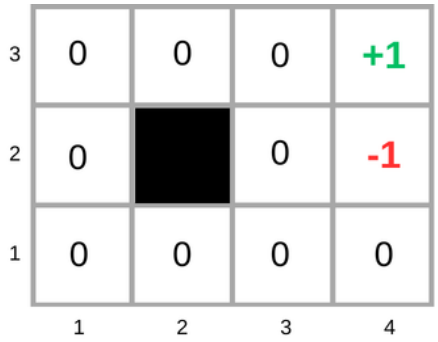


Figure 3: U_0

$$U_{i+1}(s) \leftarrow R(s) + \gamma \cdot \max_a \sum_{s'} U_i(s') \cdot P(s' \mid s, a)$$

$$U_1(s_{3,2}) \leftarrow R(s_{3,2}) + 1 \cdot \max_a \sum_{s'} U_0(s') \cdot P(s' \mid s_{3,2}, a)$$

$$U_1(s_{3,2}) \leftarrow -0.04 + 1 \cdot \max_a \sum_{s'} U_0(s') \cdot P(s' \mid s_{3,2}, a)$$

$$\begin{aligned} &\leftarrow -0.04 + \max\{ \\ &0 \cdot 0.8 + -1 \cdot 0.1 + 0 \cdot 0.1, && \text{North} \\ &-1 \cdot 0.8 + 0 \cdot 0.1 + 0 \cdot 0.1, && \text{East} \\ &0 \cdot 0.8 + 0 \cdot 0.1 + 0 \cdot 0.1, && \text{West} \\ &0 \cdot 0.8 + -1 \cdot 0.1 + 0 \cdot 0.1\} && \text{South} \end{aligned}$$

$$U_1(s_{3,2}) \leftarrow -0.04 + \max\{-0.1, -0.8, 0, -0.1\}$$

$$U_1(s_{3,2}) \leftarrow -0.04 + 0$$

$$U_1(s_{3,2}) \leftarrow -0.04$$

$$
\begin{aligned}
U_1(s_{3,3}) &\leftarrow -0.04 + 1 \cdot \max_a \sum_{s'} U_0(s') \cdot P(s' \mid s_{3,3}, a) \\
&\leftarrow -0.04 + \max\{ \\
&\quad 0 \cdot 0.8 + 1 \cdot 0.1 + 0 \cdot 0.1, && \text{North} \\
&\quad 1 \cdot 0.8 + 0 \cdot 0.1 + 0 \cdot 0.1, && \text{East} \\
&\quad 0 \cdot 0.8 + 0 \cdot 0.1 + 0 \cdot 0.1, && \text{West} \\
&\quad 0 \cdot 0.8 + 1 \cdot 0.1 + 0 \cdot 0.1\} && \text{South} \\
U_1(s_{3,3}) &\leftarrow -0.04 + \max\{0.1, 0.8, 0, 0.1\} \\
U_1(s_{3,3}) &\leftarrow -0.04 + 0.8 \\
U_1(s_{3,3}) &\leftarrow 0.76
\end{aligned}
$$

Note

For the first iteration, the only state that will have a positive utility is the one that is adjacent to the state where $R(s) = +1$. For every other state, one step will not take us to that state, hence, we will end up having $U(s) = R(s) = -0.04$ for every other state.

Figure 4: U_1

To generalize, If we are calculating the utility estimate for iteration i, only the states that can reach the "+1" state in i steps or less will have a positive estimate of utility.

$$
\begin{aligned}
U_{i+1}(s) &\leftarrow R(s) + \gamma \cdot \max_a \sum_{s'} U_i(s') \cdot P(s' \mid s, a) \\
U_2(s_{3,2}) &\leftarrow R(s_{3,2}) + 1 \cdot \max_a \sum_{s'} U_1(s') \cdot P(s' \mid s_{3,2}, a) \\
U_2(s_{3,2}) &\leftarrow -0.04 + 1 \cdot \max_a \sum_{s'} U_1(s') \cdot P(s' \mid s_{3,2}, a) \\
&\leftarrow -0.04 + \max\{ \\
&\quad 0.76 \cdot 0.8 + -1 \cdot 0.1 + -0.04 \cdot 0.1, && \text{North} \\
&\quad -1 \cdot 0.8 + 0.76 \cdot 0.1 + -0.04 \cdot 0.1, && \text{East} \\
&\quad -0.04 \cdot 0.8 + 0.76 \cdot 0.1 + -0.04 \cdot 0.1, && \text{West} \\
&\quad -0.04 \cdot 0.8 + -1 \cdot 0.1 + -0.04 \cdot 0.1\} && \text{South} \\
U_2(s_{3,2}) &\leftarrow -0.04 + \max\{0.504, -0.728, 0.04, -0.136\} \\
U_2(s_{3,2}) &\leftarrow -0.04 + 0.504 \\
U_2(s_{3,2}) &\leftarrow 0.464
\end{aligned}
$$

$$U_{i+1}(s) \leftarrow R(s) + \gamma \cdot \max_a \sum_{s'} U_i(s') \cdot P(s' \mid s, a)$$

$$U_2(s_{3,3}) \leftarrow R(s_{3,3}) + 1 \cdot \max_a \sum_{s'} U_1(s') \cdot P(s' \mid s_{3,3}, a)$$

$$U_2(s_{3,3}) \leftarrow -0.04 + 1 \cdot \max_a \sum_{s'} U_1(s') \cdot P(s' \mid s_{3,3}, a)$$

$$\begin{aligned}
&\leftarrow -0.04 + \max\{ \\
&0.76 \cdot 0.8 + -0.04 \cdot 0.1 + 1 \cdot 0.1, && \text{North} \\
&1 \cdot 0.8 + 0.76 \cdot 0.1 + -0.04 \cdot 0.1, && \text{East} \\
&-0.04 \cdot 0.8 + 0.76 \cdot 0.1 + -0.04 \cdot 0.1, && \text{West} \\
&-0.04 \cdot 0.8 + 1 \cdot 0.1 + -0.04 \cdot 0.1\} && \text{South}
\end{aligned}$$

$$U_2(s_{3,3}) \leftarrow -0.04 + \max\{0.704, 0.872, 0.04, 0.064\}$$

$$U_2(s_{3,3}) \leftarrow -0.04 + 0.872$$

$$U_2(s_{3,3}) \leftarrow 0.832$$

$U_2(s_{2,3})$ will be 0.56, All other states will have utility $= -0.08$. Therefore, second iteration utilities:

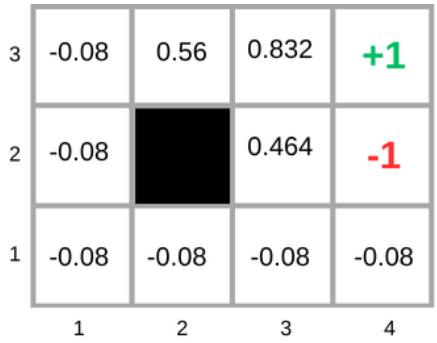


Figure 5: U_2

Again, the states that have positive utilities in the second iteration are those which can reach the "+1" state in 2 steps or less.

Observation

While executing the value iteration algorithm, you may observe that despite the utility values haven't converged yet, the optimal policy already stopped changing.

Deriving the optimal policy doesn't necessarily require us to have accurate estimates of the true utilities.

Idea

Search for optimal policy and utility values simultaneously $\rightsquigarrow$ **Policy Iteration**

Def 6.15. Policy Iteration alternates between two steps:

- **Policy Evaluation**: Given a policy π_i, calculate $U_i^{\pi}(s)$ which is the utility of each state s if π_i were to be executed.
- **Policy Improvement**: Calculate a new policy π_{i+1} using U_i^{π}

Terminate when there is no change in policy.

Value vs Policy Iteration

Value Iteration:

$$U_0(s) \to U_1(s) \to U_2(s) \to \cdots \to U^*(s)$$

Then calculate optimal action for each using:

$$\pi^*(s) = \arg\max_{a \in \mathcal{A}_s} \left(\sum_{s'} P(s' \mid s, a) \cdot U(s') \right)$$

Policy Iteration:

$$\pi_0(s) \xrightarrow{\text{eval}} U_0(s) \xrightarrow{\text{impr}} \pi_1(s) \xrightarrow{\text{eval}} U_1(s) \cdots \pi^*(s) \xrightarrow{\text{eval}} U^*(s) \xrightarrow{\text{impr}} \pi^*(s)$$

Policy Improvement

$$\pi_{i+1}(s) := \arg\max_a \sum_{s'} P(s' \mid a, s) \cdot U_i(s')$$

Policy Evaluation

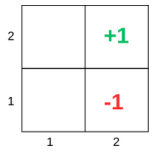
$$U_i(s) := R(s) + \gamma \cdot \sum_{s'} P(s' \mid s, \pi_i(s)) \cdot U_i(s')$$

Note

- Notice that we do not have the max operator anymore since we are already using a policy $\pi_i(s)$.
- This yields a linear system of equations where we know everything except the utilities (what we solve for)
- Solving this system is easier than solving the Bellman Equation where we have a nonlinearity because we do not know the actions (max operator)
- For n states, this will take $\mathcal{O}(n^3)$ time.

Policy Iteration Mini Exercise

Consider the following grid world with $R(s) = -0.04$ and $\gamma = 1$. Assume the initial policy $\pi_0(s_{1,1}) = \pi_0(s_{1,2}) = E$ (east) and the same transition probabilities as before.



We evaluate the initial policy:

$$U_i(s) := R(s) + \gamma \cdot \sum_{s'} P(s' \mid s, \pi_i(s)) \cdot U_i(s')$$

$$U_0(s) := R(s) + \gamma \cdot \sum_{s'} P(s' \mid s, \pi_0(s)) \cdot U_0(s')$$

$$U_0(s_{1,1}) := -0.04 + \sum_{s'} P(s' \mid s_{1,1}, E) \cdot U_0(s')$$

$$U_0(s_{1,1}) := -0.04 + 0.8 \cdot U_0(s_{2,1}) + 0.1 \cdot U_0(s_{1,2}) + 0.1 \cdot U_0(s_{1,1})$$

$$U_0(s_{1,1}) := -0.04 + 0.8 \cdot -1 + 0.1 \cdot U_0(s_{1,2}) + 0.1 \cdot U_0(s_{1,1})$$

$$U_0(s_{1,2}) := -0.04 + \sum_{s'} P(s' \mid s_{1,2}, E) \cdot U_0(s')$$

$$U_0(s_{1,2}) := -0.04 + 0.8 + 0.1 \cdot U_0(s_{1,2}) + 0.1 \cdot U_0(s_{1,1})$$

We have the following system (two equations and two variables):

$$U_0(s_{1,1}) = -0.84 + 0.1U_0(s_{1,2}) + 0.1U_0(s_{1,1}) \tag{1}$$
$$U_0(s_{1,2}) = 0.76 + 0.1U_0(s_{1,2}) + 0.1U_0(s_{1,1}) \tag{2}$$

Rearranging terms to the left-hand side:

$$0.9U_0(s_{1,1}) - 0.1U_0(s_{1,2}) = -0.84 \tag{1'}$$
$$-0.1U_0(s_{1,1}) + 0.9U_0(s_{1,2}) = 0.76 \tag{2'}$$

Adding (1') and (2') yields:

$$0.8U_0(s_{1,1}) + 0.8U_0(s_{1,2}) = -0.08 \implies U_0(s_{1,1}) + U_0(s_{1,2}) = -0.1$$

Solving for $U_0(s_{1,1}) = -0.1 - U_0(s_{1,2})$ and substituting into (2'):

$$\begin{aligned} -0.1(-0.1 - U_0(s_{1,2})) + 0.9U_0(s_{1,2}) &= 0.76 \\ 0.01 + 0.1U_0(s_{1,2}) + 0.9U_0(s_{1,2}) &= 0.76 \\ U_0(s_{1,2}) &= 0.75 \end{aligned}$$

Substituting back to find $U_0(s_{1,1})$:

$$U_0(s_{1,1}) = -0.1 - 0.75 = -0.85$$

The final utilities are:

$$U_0(s_{1,1}) = -0.85, \quad U_0(s_{1,2}) = 0.75$$

Now we improve:

$$\pi_{i+1}(s) := \arg\max_a \sum_{s'} P(s' \mid a, s) \cdot U_i(s')$$
$$\pi_1(s) := \arg\max_a \sum_{s'} P(s' \mid a, s) \cdot U_0(s')$$
$$\pi_1(s_{1,1}) := \arg\max_a \sum_{s'} P(s' \mid a, s_{1,1}) \cdot U_0(s')$$
$$\pi_1(s_{1,1}) := \arg\max_a \{$$

$0.8 \cdot 0.75 + 0.1 \cdot -1 + 0.1 \cdot -0.85,$	North
$0.8 \cdot -0.85 + 0.1 \cdot -0.85 + 0.1 \cdot -1,$	South
$0.8 \cdot -1 + 0.1 \cdot 0.75 + 0.1 \cdot -0.85,$	East
$0.8 \cdot -0.85 + 0.1 \cdot 0.75 + 0.1 \cdot -0.85\}$	West

$$\pi_1(s_{1,1}) := \arg\max_a \{$$

$0.415,$	North
$-0.865,$	South
$-0.81,$	East
$-0.69\}$	West

$$\pi_1(s_{1,1}) := N$$

Similarly, for $s_{1,2}$:

$$\pi_1(s_{1,2}) := \arg\max_a \sum_{s'} P(s' \mid a, s_{1,2}) \cdot U_0(s')$$

$$\pi_1(s_{1,2}) := \arg\max_a \{$$

$0.8 \cdot 0.75 + 0.1 \cdot 0.75 + 0.1 \cdot 1,$	North
$0.8 \cdot -0.85 + 0.1 \cdot 0.75 + 0.1 \cdot 1,$	South
$0.8 \cdot 1 + 0.1 \cdot 0.75 + 0.1 \cdot -0.85,$	East
$0.8 \cdot 0.75 + 0.1 \cdot 0.75 + 0.1 \cdot -0.85\}$	West

$$\pi_1(s_{1,2}) := \arg\max_a \{$$

$0.775,$	North
$-0.505,$	South
$0.79,$	East
$0.59\}$	West

$\pi_1(s_{1,2}) := E$ (didn't change)

Now we evaluate π_1:

$$U_i(s) := R(s) + \gamma \cdot \sum_{s'} P(s' \mid s, \pi_i(s)) \cdot U_i(s')$$

$$U_1(s) := R(s) + \gamma \cdot \sum_{s'} P(s' \mid s, \pi_1(s)) \cdot U_1(s')$$

$$U_1(s_{1,1}) := -0.04 + \sum_{s'} P(s' \mid s, \pi_1(s_{1,1})) \cdot U_1(s')$$

$$U_1(s_{1,1}) := -0.04 + 0.8 \cdot U_1(s_{1,2}) + 0.1 \cdot -1 + 0.1 \cdot U_1(s_{1,1})$$

$$U_1(s_{1,2}) := -0.04 + \sum_{s'} P(s' \mid s, \pi_1(s_{1,2})) \cdot U_1(s')$$

$$U_1(s_{1,2}) := -0.04 + 0.8 \cdot 1 + 0.1 \cdot U_1(s_{1,2}) + 0.1 \cdot U_1(s_{1,1})$$

We have the following system (two equations and two variables):

$$U_1(s_{1,1}) := -0.04 + 0.8 \cdot U_1(s_{1,2}) - 0.1 + 0.1 \cdot U_1(s_{1,1}) \tag{3}$$

$$U_1(s_{1,2}) := -0.04 + 0.8 + 0.1 \cdot U_1(s_{1,2}) + 0.1 \cdot U_1(s_{1,1}) \tag{4}$$

Rearranging:

$$0.9U_1(s_{1,1}) - 0.8U_1(s_{1,2}) = -0.14 \tag{3'}$$

$$-0.1U_1(s_{1,1}) + 0.9U_1(s_{1,2}) = 0.76 \tag{4'}$$

From (4'), we express $U_1(s_{1,1})$ in terms of $U_1(s_{1,2})$:

$$0.1U_1(s_{1,1}) = 0.9U_1(s_{1,2}) - 0.76 \implies U_1(s_{1,1}) = 9U_1(s_{1,2}) - 7.6$$

Substituting this into (3'):

$$\begin{aligned} 0.9(9U_1(s_{1,2}) - 7.6) - 0.8U_1(s_{1,2}) &= -0.14 \\ 8.1U_1(s_{1,2}) - 6.84 - 0.8U_1(s_{1,2}) &= -0.14 \\ 7.3U_1(s_{1,2}) &= 6.7 \\ U_1(s_{1,2}) &= \frac{67}{73} \approx 0.918 \end{aligned}$$

Substituting back to find $U_1(s_{1,1})$:

$$U_1(s_{1,1}) = 9\left(\frac{67}{73}\right) - 7.6 = \frac{603 - 554.8}{73} = \frac{48.2}{73} \approx 0.660$$

The final utilities for π_1 are:

$$U_1(s_{1,1}) \approx 0.660, \quad U_1(s_{1,2}) \approx 0.918$$

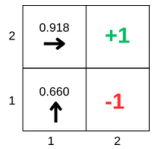
Now we improve:

$$\begin{aligned}
\pi_{i+1}(s) &:= \arg\max_a \sum_{s'} P(s' \mid a, s) \cdot U_i(s') \\
\pi_2(s) &:= \arg\max_a \sum_{s'} P(s' \mid a, s) \cdot U_1(s') \\
\pi_2(s_{1,1}) &:= \arg\max_a \sum_{s'} P(s' \mid a, s_{1,1}) \cdot U_1(s') \\
\pi_2(s_{1,1}) &:= \arg\max_a \{ \\
&\qquad 0.8 \cdot 0.918 + 0.1 \cdot -1 + 0.1 \cdot 0.660, && \text{North} \\
&\qquad 0.8 \cdot 0.660 + 0.1 \cdot 0.660 + 0.1 \cdot -1, && \text{South} \\
&\qquad 0.8 \cdot -1 + 0.1 \cdot 0.918 + 0.1 \cdot 0.660, && \text{East} \\
&\qquad 0.8 \cdot 0.660 + 0.1 \cdot 0.918 + 0.1 \cdot 0.660\} && \text{West} \\
\pi_2(s_{1,1}) &:= \arg\max_a \{ \\
&\qquad 0.7004, && \text{North} \\
&\qquad 0.494, && \text{South} \\
&\qquad -0.6422, && \text{East} \\
&\qquad 0.6858\} && \text{West} \\
\pi_2(s_{1,1}) &:= N && \text{(didn't change)}
\end{aligned}$$

Similarly, for $s_{1,2}$:

$$\begin{aligned}
\pi_2(s_{1,2}) &:= \arg\max_a \sum_{s'} P(s' \mid a, s_{1,2}) \cdot U_1(s') \\
\pi_2(s_{1,2}) &:= \arg\max_a \{ \\
&\qquad 0.8 \cdot 0.918 + 0.1 \cdot 0.918 + 0.1 \cdot 1, && \text{North} \\
&\qquad 0.8 \cdot 0.660 + 0.1 \cdot 0.918 + 0.1 \cdot 1, && \text{South} \\
&\qquad 0.8 \cdot 1 + 0.1 \cdot 0.918 + 0.1 \cdot 0.660, && \text{East} \\
&\qquad 0.8 \cdot 0.918 + 0.1 \cdot 0.918 + 0.1 \cdot 0.660\} && \text{West} \\
\pi_2(s_{1,2}) &:= \arg\max_a \{ \\
&\qquad 0.9262, && \text{North} \\
&\qquad 0.7198, && \text{South} \\
&\qquad 0.9578, && \text{East} \\
&\qquad 0.8922\} && \text{West} \\
\pi_2(s_{1,2}) &:= E && \text{(didn't change)}
\end{aligned}$$

Since $\pi_1(s) = \pi_2(s) \quad \forall s$, we terminate.



Motivation

So far, we assumed that the environment is fully observable. This means that the agent always knows exactly which state it is currently in.
However, many environments are only partially observable. In such environments, the agent cannot directly observe the true state. Instead, it only receives incomplete or noisy information about the state.
To model such situations, we use **Partially Observable Markov Decision Processes (POMDPs)**.

Example 6.8. Consider an agent moving on an infinite discrete line. The current position of the agent is represented by an integer.

The agent can:

- move one step left
- move one step right
- stay where it is

However, actions are stochastic:

- moving succeeds only with probability 0.75
- otherwise, the agent does not move

Additionally, the agent cannot directly observe its exact position. Instead, it receives noisy measurements:

- the correct position is measured with probability 0.5
- a neighboring position is measured with probability 0.25

Def 6.16. POMDP: A **Partially Observable Markov Decision Process (POMDP)** is the structure

$$\langle \mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{I}, R, \mathcal{E}, O \rangle$$

where:

- $\mathcal{S}$ is the set of states
- $\mathcal{A}$ is the set of actions
- $\mathcal{T}$ is the transition model $\mathcal{T}(s, a) = \mathbf{P}(\mathcal{S} \mid s, a)$
- $\mathcal{I}$ is the initial probability distribution over states
- $R : \mathcal{S} \to \mathbb{R}$ is the reward function
- $\mathcal{E}$ is the set of possible observations
- O is the observation model such that $O(s, e) = P(e \mid s)$

Example 6.9. For our line-navigation problem:

$$\mathcal{S} = \mathbb{Z}$$

The actions are:

$$\mathcal{A} = \{-1, 0, +1\}$$

Example initial probability distribution:

$$\mathcal{I} = \{(4, 0.6), (5, 0.4)\}$$

This means the agent initially believes:

- it is in position 4 with probability 0.6
- it is in position 5 with probability 0.4

Observation

Compared to an MDP, a POMDP introduces an additional source of uncertainty:
- in an MDP, the current state is known exactly
- in a POMDP, the agent only receives observations about the state

Def 6.17. Belief State: A **belief state** is a probability distribution over states:

$$b = \mathbf{P}(\mathcal{S})$$

where $b(s)$ denotes the probability assigned to state s.

Idea

In a POMDP, the agent does not know the true current state.
Therefore, instead of storing a single state, it stores a probability distribution over all possible states.
This distribution is called the belief state.

Def 6.18. POMDP Decision Cycle: A POMDP proceeds in the following cycle:

1. the agent starts with a belief state
2. the agent executes an action
3. the environment changes according to the transition model
4. the agent receives an observation
5. the agent updates its belief state

Example 6.10. Suppose the agent currently has:

$$b_0 = \{(4, 0.6), (5, 0.4)\}$$

and executes action $+1$.

The agent then:

1. computes the new probabilities after the stochastic movement
2. receives a noisy observation
3. updates the belief state using the observation

Def 6.19. Transition Model in a POMDP: The **transition model** describes how actions probabilistically change the state:

$$\mathcal{T}(s, a) = \mathbf{P}(\mathcal{S} \mid s, a)$$

Example 6.11. For action $+1$:

$$\mathcal{T}(n, +1) = \{(n + 1, 0.75), (n, 0.25)\}$$

This means:

- with probability 0.75, the agent successfully moves right
- with probability 0.25, it stays where it is

Similarly:

$$\mathcal{T}(n, -1) = \{(n - 1, 0.75), (n, 0.25)\}$$

Def 6.20. Observation Model: The **observation model** specifies the probability of receiving observation e while being in state s:

$$O(s, e) = P(e \mid s)$$

Example 6.12.

$$O = \{(s, s - 1, 0.25), (s, s, 0.5), (s, s + 1, 0.25)\}$$

This means:

- if the agent observes it is in s, there is a 0.5 chance it is actually there,
- and there is a 0.25 chance it is at $s - 1$, and another 0.25 chance it is at $s + 1$.

Belief State Update

Assume the current belief state is:

$$b_0 = \{(4, 0.6), (5, 0.4)\}$$

The agent executes action $+1$.

Recall:

$$\mathcal{T}(n, +1) = \{(n + 1, 0.75), (n, 0.25)\}$$

We now compute the probability for each possible successor state separately.

State 4

The only way to end in state 4 is:

- start in state 4
- fail to move

Thus:

$$b_a(4) = 0.6 \cdot 0.25 = 0.15$$

State 5

There are two ways to end in state 5:

- start in state 4 and successfully move right
- start in state 5 and fail to move

Thus:

$$b_a(5) = 0.6 \cdot 0.75 + 0.4 \cdot 0.25 = 0.45 + 0.10 = 0.55$$

State 6

The only way to end in state 6 is:

- start in state 5
- successfully move right

Thus:

$$b_a(6) = 0.4 \cdot 0.75 = 0.3$$

Therefore, the resulting belief state is:

$$b_a = \{(4, 0.15), (5, 0.55), (6, 0.3)\}$$

Suppose the agent now observes position 4.

Recall that the predicted belief state after executing action $+1$ was:

$$b_a = \{(4, 0.15), (5, 0.55), (6, 0.3)\}$$

This means:

- the agent believes it is in state 4 with probability 0.15
- in state 5 with probability 0.55
- in state 6 with probability 0.3

Now the agent receives the observation 4.

We use the observation model:

$$O = \{(s, s - 1, 0.25), (s, s, 0.5), (s, s + 1, 0.25)\}$$

We now ask:

> How likely is it to observe 4 from each possible state?

Case 1: True state is 4 If the true state is 4, then observing 4 is correct.

Thus:

$$O(4,4) = 0.5$$

The prior probability of being in state 4 was:

$$b_a(4) = 0.15$$

Therefore:

$$P(4,4) = b_a(4) \cdot O(4,4) = 0.15 \cdot 0.5 = 0.075$$

Case 2: True state is 5 If the true state is 5, then observing 4 means measuring one step too far left.

Thus:

$$O(5,4) = 0.25$$

The prior probability of being in state 5 was:

$$b_a(5) = 0.55$$

Therefore:

$$P(5,4) = b_a(5) \cdot O(5,4) = 0.55 \cdot 0.25 = 0.1375$$

Case 3: True state is 6 If the true state is 6, then observing 4 would require an error of two positions.

This is impossible in our observation model.

Thus:

$$O(6,4) = 0$$

Therefore:

$$P(6,4) = b_a(6) \cdot O(6,4) = 0.3 \cdot 0 = 0$$

Combining all values yields the unnormalized updated belief state:

$$\{(4, 0.075), (5, 0.1375)\}$$

However, this is not yet a probability distribution, because the probabilities do not sum to 1:

$$0.075 + 0.1375 = 0.2125$$

Therefore, we normalize the values.

Normalized probability for state 4

$$b'(4) = \frac{0.075}{0.2125} \approx 0.35$$

Normalized probability for state 5

$$b'(5) = \frac{0.1375}{0.2125} \approx 0.65$$

Thus, the updated belief state becomes:

$$b' \approx \{(4, 0.35), (5, 0.65)\}$$

Idea

Observation 4 increases the probability of state 5 more strongly than state 4 because state 5 already had a much larger prior probability before the observation.

Def 6.21. Belief-State MDP: Every POMDP can be transformed into an equivalent MDP whose states are belief states.

Observation

In the transformed MDP:

- a state is no longer a concrete world state
- instead, a state is a probability distribution over world states

Def 6.22. Reward Function on Belief States: The reward of a belief state is defined as its expected reward:

$$R'(b) = \sum_{s \in \mathcal{S}} R(s) \cdot b(s)$$

Example 6.13. Suppose:

$$R(0) = 1$$

and:

$$R(s) = -0.1 \qquad \text{for all } s \neq 0$$

For:

$$b = \{(4, 0.15), (5, 0.55), (6, 0.3)\}$$

we obtain:

$$R'(b) = (-0.1) \cdot 0.15 + (-0.1) \cdot 0.55 + (-0.1) \cdot 0.3 = -0.1$$

Although POMDPs can be reduced to belief-state MDPs, solving them exactly is computationally very difficult.

Theorem 6.5. Solving finite-horizon POMDPs is PSPACE-hard.

Note

The difficulty comes from the fact that the belief-state space is typically infinite. Even if the original state space is finite, there are infinitely many possible probability distributions over it.

7 Introduction to Machine Learning

Motivation

- You have data, and you think there is some *model* that can describe how your data would be generated (if it were)
- You do not know that *model*, but you are trying to find something that is as close as possible to that *model* (from some set of *models* (hypotheses) you are willing to consider)
- Simplest Example: Learning a function
  - There is a **target function** g (We do not know it)
  - We have input-output pairs $X_i = (x, g(x))$
  - The set of hypotheses you are willing to consider are in a space $\mathcal{H}$
  - Given a set (called training set) of examples X_i, we are trying to find a hypothesis $h \in \mathcal{H}$ such that $h \approx g$

Def 7.1. A **Supervised (Inductive) Learning Problem** is the triple $\langle \mathcal{H}, T, f \rangle$ where:

- $\mathcal{H}$: The set of **hypotheses** consisting of functions $h : A \to B$
- $T \subseteq A \times B$: A set of **examples** called the **Training Set**, such that for every $a \in A$, there is at most one $b \in B$ with $\langle a, b \rangle \in T$

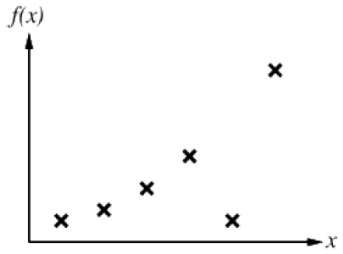
We assume there is an *unknown* function $f : A \to B$ called the **Target Function** with $T \subseteq f$

Def 7.2. **Inductive Learning Algorithms** solve supervised learning problems by finding a hypothesis $h \in \mathcal{H}$ such that $h \sim f$ (for some notion of similarity)

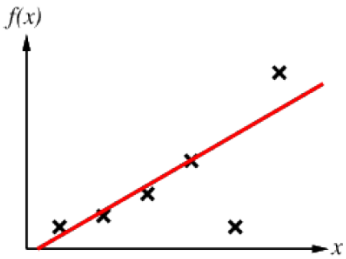
Def 7.3. We call a supervised learning problems with target function $f : A \to B$ a **classification problem** if B is finite, and call the members of B **classes/classifications**. We call it a **regression problem** if $B = \mathbb{R}$

Def 7.4. We call a hypothesis $h \in \mathcal{H}$ **consistent with** the unknown target function f (on a set $T \subseteq \text{dom}(f)$), if it agrees with f (on all examples in T)

Example 7.1. Assume we have the following data:

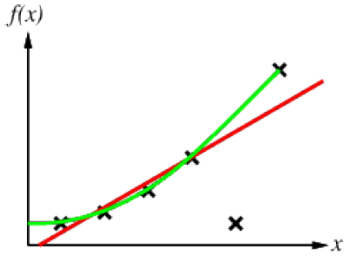


We can choose our hypotheses space $\mathcal{H}$ to be the linear functions, then the best linear hypothesis we can find might look like:

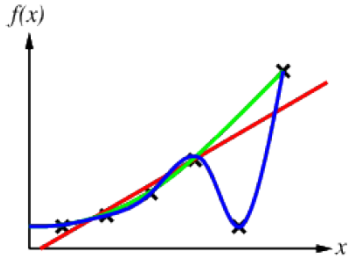


Which is *partially, approximately* consistent.

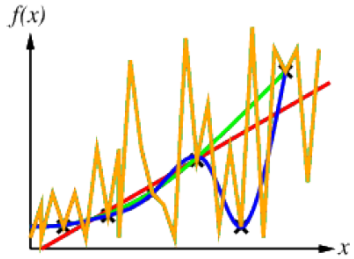
If $\mathcal{H}$ were the quadratic functions, we can find another *partially* consistent hypothesis that might look like:



If we consider Degree−4 polynomials, we get a consistent hypothesis:



If we consider High-degree polynomials we get more (noisy) consistent hypotheses, e.g.



Observation

Whether we can find a consistent hypothesis for a given training set depends on the chosen hypotheses space

Recall

Ockham's-razor: Maximize a combination of consistency and simplicity.

Def 7.5. We say that a supervised learning problem is **realizable**, iff there is a hypothesis $h \in \mathcal{H}$ that is consistent on the training set T

Problem

We do not always know whether a given learning problem is realizable, unless we have prior knowledge. A naive solution would be to make $\mathcal{H}$ as large as possible, e.g. all Turing Machines. However, the complexity of the supervised learning problem is tied to the size of the hypotheses space and consistency is not even decidable for general Turing Machines.

Much of the research in machine learning has concentrated on simple hypotheses spaces

Consistency vs. Simplicity

Following **Ockham's Razor**, among competing hypotheses, we prefer those that achieve a good balance between:

- **Consistency** with the observed training set
- **Simplicity** of the chosen hypothesis

This tradeoff is fundamental in machine learning:

- If the hypothesis is *too simple*, it may fail to capture important regularities in the data.
- If the hypothesis is *too complex*, it may adapt excessively to noise or accidental patterns in the training set.

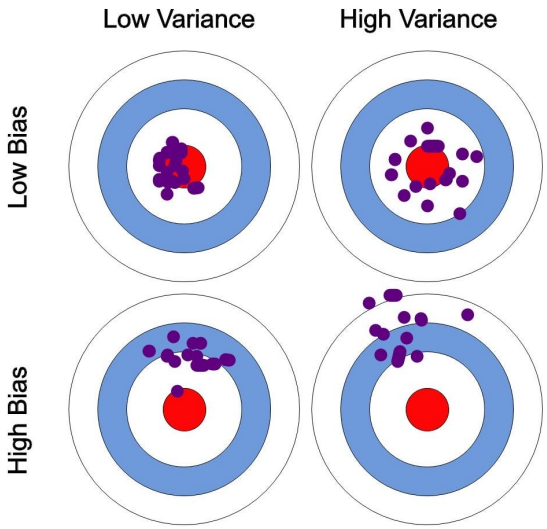
Def 7.6. We say that a hypothesis **underfits** the data if it is too simple to adequately represent the underlying regularities of the unknown target function. Such hypotheses usually fail to achieve good consistency even on the training set

Def 7.7. We say that a hypothesis **overfits** the data if it becomes excessively adapted to the specific training set, including noise or accidental irregularities. Such hypotheses may achieve very high consistency on the training data while generalizing poorly to unseen examples

Bias vs. Variance

The tradeoff between simplicity and consistency is often called the **Bias-Variance Tradeoff**:

- **High Bias** corresponds to overly simple models that tend to underfit
- **High Variance** corresponds to overly complex models that tend to overfit



Why is it called Variance?

Imagine repeatedly drawing different random training sets from the same underlying process.
For each training set, we compute the "best" hypothesis $h \in \mathcal{H}$.
This generates a sequence of learned hypotheses:

$$h_1, h_2, h_3, \ldots$$

If small changes in the training set produce large changes in the resulting hypothesis, then the learning method has **high variance**.
Thus, high variance means the learned model is highly sensitive to the particular sampled data. This sensitivity often causes overfitting, since the model begins adapting to noise and accidental fluctuations in the training data rather than the underlying structure.

Why is it called Bias?

Bias measures the systematic error introduced by restricting ourselves to an overly simple hypotheses space.
If the considered hypotheses are all "far" from the unknown target function, then even the best possible learned model will make systematic errors.
In this case, the learning method has **high bias**.
High bias typically causes underfitting, because the model is too simple to capture important regularities in the data.

Observation

In practice, good machine learning models attempt to balance:

simplicity and *consistency*

or equivalently:

bias and variance

Too much simplicity leads to underfitting, while too much complexity leads to overfitting.

Problem

We want to learn a hypothesis that will best predict future data.

Note

This only works if the training set is *"representative"* of the underlying process.

Idea

We think of examples (seen and unseen) as a sequence, and formalize *"representativeness"* via a *stationary assumption* on the probability distribution.

Each example, before observation, is a random variable E_j. The observed value $e_j = \langle x_j, y_j \rangle$ is a sample from its distribution.

Def 7.8. IID: A sequence $E_1, E_2, \dots, E_n$ of random variables is **independent and identically distributed (IID)** iff they are:

- independent, i.e. $\mathbf{P}(E_j \mid E_{j-1}, E_{j-2}, \dots) = \mathbf{P}(E_j)$ for every j, and
- **identically distributed**, i.e. $\mathbf{P}(E_i) = \mathbf{P}(E_j)$ for every pair of indices i, j.

Def 7.9. Stationary Assumption: We assume the sequence $\varepsilon = (E_t)$ of examples is **stationary** iff for every $k \geq 0$ and every pair of indices i, j:

$$\mathbf{P}(E_i, E_{i+1}, \dots, E_{i+k}) = \mathbf{P}(E_j, E_{j+1}, \dots, E_{j+k}).$$

Note

Stationarity does not imply independence; IID implies stationarity.

Def 7.10. In **attribute-based representations**, examples are described by

- **attributes**: functions on input samples,
- their **values**, and
- classifcations

Example 7.2. Situations where I will/won't wait for a table in a Restaurant

Table 3: The Restaurant Dataset (Russell & Norvig)

Example	Attributes										Target
	Alt	*Bar*	*Fri*	*Hun*	*Pat*	*Price*	*Rain*	*Res*	*Type*	*Est*	*WillWait*
X_1	T	F	F	T	Some	$$$	F	T	French	0–10	T
X_2	T	F	F	T	Full	$	F	F	Thai	30–60	F
X_3	F	T	F	F	Some	$	F	F	Burger	0–10	T
X_4	T	F	T	T	Full	$	F	F	Thai	10–30	T
X_5	T	F	T	F	Full	$$$	F	T	French	>60	F
X_6	F	T	F	T	Some	$$	T	T	Italian	0–10	T
X_7	F	T	F	F	None	$	T	F	Burger	0–10	F
X_8	F	F	F	T	Some	$$	T	T	Thai	0–10	T
X_9	F	T	F	T	Full	$	T	F	Burger	>60	F
X_{10}	T	T	T	T	Full	$$$	F	T	Italian	10–30	F
X_{11}	F	F	F	F	None	$	F	F	Thai	0–10	F
X_{12}	T	T	T	T	Full	$	F	F	Burger	30–60	T

The attributes are:

- **Alt:** Whether a suitable alternative restaurant is available nearby.
- **Bar:** Whether the restaurant has a comfortable bar area to wait in.
- **Fri:** True if the current day is Friday or Saturday.
- **Hun:** Whether the customer is genuinely hungry.
- **Pat:** The number of patrons currently in the restaurant (*None*, *Some*, or *Full*).
- **Price:** The restaurant's price tier ($, $$, or $$$).
- **Rain:** Whether it is actively raining outside.
- **Res:** Whether the customer made a reservation in advance.
- **Type:** The culinary style of the restaurant (*French*, *Thai*, *Burger*, or *Italian*).
- **Est:** The estimated waiting time in minutes (*0–10*, *10–30*, *30–60*, or *>60*).

Def 7.11. For a Boolean classifcation, we say that an examples is **positive** (T) or **negative** (F) depending on its class

Def 7.12. Decision Trees: A **Decision Tree (DT)** for a given attribute-based representation is a tree, where the non-leaf nodes are labeled by attributes, their outgoing edges by disjoint sets of attribute values, and the leaf nodes are labeled by the classifcations.

Decision Trees are one possible representation for hypotheses

Example 7.3. Here is the *true* DT for example 7.2:

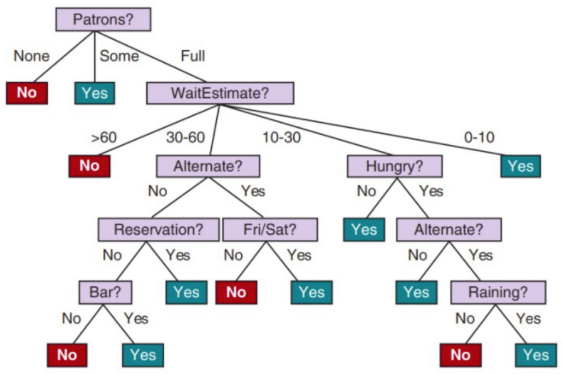


Figure 6: The true decision tree for the restaurant dataset.

Def 7.13. We call an attribute together with a set of attribute values (an inner node with outgoing edge labels) an **attribute test**.

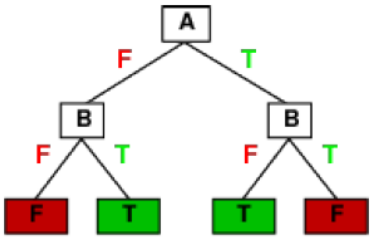
Note

The target function is a function $A_1 \times \cdots \times A_n \to C$, where A_i are the domains of the attributesvand C is the set of classifcations.

Decision Trees can express any function of the input attributes $\rightsquigarrow \mathcal{H} = A_1 \times \cdots \times A_n$

Example 7.4. For Boolean functions, a path from the root to a leaf corresponds to a row in a truth table (and the DT corresponds to a truth table):

A	B	A xor B
F	F	F
F	T	T
T	F	T
T	T	F



Problem

Trivially, for any training set there is a consistent hypothesis with one path to a leaf for each example, but it probably won't generalize to new examples.

Solution

We should find more *compact* Decision Trees.

Idea

A *good* attribute splits the examples into subsets that are *ideally* all positive or all negative.

Def 7.14. Let $\langle p_1, \cdots, p_n \rangle$ the distirbution of a random variable P. The **information** (also called **entropy**) of P is:

$$I(\langle p_1, \cdots, p_n \rangle) := \sum_{i=1}^{n} -(p_i \cdot \log_2(p_i))$$

Note

Because $\log_2(0)$ is undefined, for $p_i = 0$, we consider $p_i \cdot \log_2(p_i) = 0$

Def 7.15. The unit of information is called **bit (b)** , with:

$$\begin{aligned} 1b := I(\left\langle \frac{1}{2}, \frac{1}{2} \right\rangle) &:= -(\frac{1}{2} \cdot \log_2(\frac{1}{2})) + -(\frac{1}{2} \cdot \log_2(\frac{1}{2})) \\ &:= -(\frac{1}{2} \cdot (-1)) + -(\frac{1}{2} \cdot (-1)) \\ &:= -(-\frac{1}{2}) + -(-\frac{1}{2}) \\ &:= \frac{1}{2} + \frac{1}{2} = 1b \end{aligned}$$

Example 7.5.

- For a fair coin toss, we have $I(\langle \frac{1}{2}, \frac{1}{2} \rangle) = 1b$

- for a coin toss with 99% heads and 1% tails, we have $I(\langle \frac{1}{100}, \frac{99}{100} \rangle) = 0.08b$
- $\rightsquigarrow$ information goes to 0 as head probability goes to 1.

Example 7.6. Suppose we have two different coins:

- Coin A is a fair coin:
$$P(H) = \frac{1}{2}, \quad P(T) = \frac{1}{2}$$
- Coin B is a biased coin:
$$P(H) = 0.99, \quad P(T) = 0.01$$

We now compute the entropy of both probability distributions.

Entropy of Coin A

$$I\left\langle \frac{1}{2}, \frac{1}{2} \right\rangle = 1b$$

Entropy of Coin B

$$I\left\langle 0.99, 0.01 \right\rangle = -\left(0.99 \log_2(0.99) + 0.01 \log_2(0.01)\right)$$

Using:

$$\log_2(0.99) \approx -0.014 \qquad \text{and} \qquad \log_2(0.01) \approx -6.644$$

we obtain:

$$I\left\langle 0.99, 0.01 \right\rangle \approx -\left(0.99(-0.014) + 0.01(-6.644)\right)$$

$$\approx -\left(-0.0139 - 0.0664\right) \approx 0.08b$$

Intuition Behind Entropy

Entropy measures how uncertain or unpredictable a random variable is.

- The fair coin has high entropy because the outcome is completely uncertain. Before tossing the coin, we genuinely do not know whether we will obtain heads or tails. Therefore, observing the outcome gives us a lot of information.
- The biased coin has very low entropy because the result is almost always heads. Seeing "heads" is not surprising, so observing the outcome gives us very little new information.

Entropy tells us how much information we expect to gain when observing the result of a random process.

Idea

Suppose we have p examples classified as positive and n classified as negative. We can then estimate the probability distribution of the classifcation C with $\mathbf{P}(C) = \left\langle \frac{p}{p+n}, \frac{n}{p+n} \right\rangle$, and need $I(\mathbf{P}(C))$ bits to correctly classify a new example.

Example 7.7. For the restaurant data, we have total of 12 examples and $n = p = 6$, hence $I(\mathbf{P}(\text{WillWait})) = I(\langle \frac{6}{12}, \frac{6}{12} \rangle) = 1b$ of information

Observation

If we know, e.g. $Pat = Full$, we would only need $I(\mathbf{P}(\text{WillWait} \mid \text{Pat} = \text{Full})) = I(\langle \frac{4}{6}, \frac{2}{6} \rangle) \approx 0.9$ bits of information.

Note

Treating attributes also as random variables, we can compute how much information is needed after knowing the value for one attribute.

Def 7.16. The **conditional entropy** (or expected remaining information) of C given attribute A, written $H(C \mid A)$, is

$$\begin{aligned} H(C \mid A) &:= \sum_a P(A=a) \cdot I\big(\mathbf{P}(C \mid A=a)\big) \\ &:= -\sum_a P(A=a) \sum_c P(C=c \mid A=a) \cdot \log_2 P(C=c \mid A=a) \end{aligned}$$

Example 7.8. We want to compute the conditional entropy for attribute *Pat* on the restaurant dataset

Pat = None. The examples with Pat = None are X_7, X_{11}; both have $C = \text{WillWait} = F$. Thus

$$P(C=T \mid \text{Pat}=\text{None}) = 0, \qquad P(C=F \mid \text{Pat}=\text{None}) = 1,$$

Hence:

$$I\big(\mathbf{P}(C \mid \text{Pat}=\text{None})\big) = -\big(0 \log_2 0 + 1 \log_2 1\big) = 0.$$

Pat = Some. The examples with Pat = Some are X_1, X_3, X_6, X_8; all have $C = T$. Hence

$$P(C=T \mid \text{Pat}=\text{Some}) = 1, \qquad P(C=F \mid \text{Pat}=\text{Some}) = 0,$$

so

$$I\big(\mathbf{P}(C \mid \text{Pat}=\text{Some})\big) = -\big(1 \log_2 1 + 0 \log_2 0\big) = 0.$$

Pat = Full. The examples with Pat = Full are $X_2, X_4, X_5, X_9, X_{10}, X_{12}$. There are 2 positives (T) and 4 negatives (F), so

$$P(C=T \mid \text{Pat}=\text{Full}) = \tfrac{2}{6} = \tfrac{1}{3}, \qquad P(C=F \mid \text{Pat}=\text{Full}) = \tfrac{4}{6} = \tfrac{2}{3}.$$

Apply the entropy formula:

$$\begin{aligned} I\big(\mathbf{P}(C \mid \text{Pat}=\text{Full})\big) &= -\Big(\tfrac{1}{3} \log_2 \tfrac{1}{3} + \tfrac{2}{3} \log_2 \tfrac{2}{3}\Big) \\ &= -\Big(\tfrac{1}{3}(-\log_2 3) + \tfrac{2}{3}(\log_2 2 - \log_2 3)\Big) \\ &= -\Big(-\tfrac{1}{3} \log_2 3 + \tfrac{2}{3}(1 - \log_2 3)\Big) \\ &= -\Big(-\tfrac{1}{3} \log_2 3 + \tfrac{2}{3} - \tfrac{2}{3} \log_2 3\Big) \\ &= -\Big(\tfrac{2}{3} - \log_2 3\Big) \cdot (-1) \\ &= \log_2 3 - \tfrac{2}{3}. \end{aligned}$$

Numerically, with $\log_2 3 \approx 1.5850$,

$$I\big(\mathbf{P}(C \mid \text{Pat}=\text{Full})\big) \approx 1.5850 - 0.6667 \approx 0.9183 \text{ bits}.$$

Combine to get conditional entropy:

$$H(C \mid \text{Pat}) = \sum_a P(\text{Pat}=a)\, I\big(\mathbf{P}(C \mid \text{Pat}=a)\big) = \tfrac{2}{12} \cdot 0 + \tfrac{4}{12} \cdot 0 + \tfrac{6}{12} \cdot 0.9183 \approx 0.4592 \text{ bits}.$$

Def 7.17. The **information gain** from an attribute test A is:

$$\begin{aligned} \text{Gain}(A) &:= I(\mathbf{P}(C)) - H(C \mid A) \\ &:= I(\mathbf{P}(C)) - \sum_a P(A=a) \cdot I\big(\mathbf{P}(C \mid A=a)\big) \end{aligned}$$

Def 7.18. Assume we know the results of some attribute tests

$$b := B_1 = b_1 \wedge B_2 = b_2 \wedge \cdots \wedge B_n = b_n$$

The **conditional entropy** (or expected remaining information) of the class variable C **given the attribute** A **under background information** b, is:

$$\begin{aligned} H(C \mid A, b) &:= \sum_a P(A = a \mid b) \cdot I\big(\mathbf{P}(C \mid A = a, b)\big) \\ &:= -\sum_a P(A = a \mid b) \sum_c P(C = c \mid A = a, b) \log_2 P(C = c \mid A = a, b) \end{aligned}$$

Def 7.19. Assume we know the results of some attribute tests

$$b := B_1 = b_1 \wedge B_2 = b_2 \wedge \cdots \wedge B_n = b_n$$

Then the **conditional information gain** from an attribute test A is:

$$\begin{aligned} \text{Gain}(A \mid b) &:= I(\mathbf{P}(C \mid b)) - H(C \mid A, b) \\ &:= I(\mathbf{P}(C \mid b)) - \sum_a P(A = a \mid b) \cdot I\big(\mathbf{P}(C \mid A = a, b)\big) \end{aligned}$$

Example 7.9. Let's compute information gain for the attribute *Patrons*:

$$\begin{aligned} \text{Gain}(A) &:= I(\mathbf{P}(C)) - H(C \mid A) \\ &:= I(\mathbf{P}(C)) - \sum_a P(A = a) \cdot I\big(\mathbf{P}(C \mid A = a)\big) \end{aligned}$$

$$\text{Gain}(\text{Patrons}) := I(\mathbf{P}(C)) - \sum_a P(\text{Patrons} = a) \cdot I\big(\mathbf{P}(C \mid \text{Patrons} = a)\big)$$

We know that $I(\mathbf{P}(C)) = I(\langle P(C = T), P(C = F)\rangle) = I(\langle \frac{6}{12}, \frac{6}{12}\rangle) = 1b$

We have the following examples with Pat=None:

	Attributes										**Target**
Example	*Alt*	*Bar*	*Fri*	*Hun*	*Pat*	*Price*	*Rain*	*Res*	*Type*	*Est*	*WillWait*
X_7	F	T	F	F	None	$	T	F	Burger	0–10	F
X_{11}	F	F	F	F	None	$	F	F	Thai	0–10	F

So, $P(\text{Pat=None}) = \frac{2}{12}$

And $\mathbf{P}(C \mid \text{Pat=None}) = \langle \frac{0}{2}, \frac{2}{2}\rangle = \langle 0, 1\rangle$

We have the following examples with Pat=Some:

	Attributes										**Target**
Example	*Alt*	*Bar*	*Fri*	*Hun*	*Pat*	*Price*	*Rain*	*Res*	*Type*	*Est*	*WillWait*
X_1	T	F	F	T	Some	$$$	F	T	French	0–10	T
X_3	F	T	F	F	Some	$	F	F	Burger	0–10	T
X_6	F	T	F	T	Some	$$	T	T	Italian	0–10	T
X_8	F	F	F	T	Some	$$	T	T	Thai	0–10	T

So, $P(\text{Pat=Some}) = \frac{4}{12}$

And $\mathbf{P}(C \mid \text{Pat=Some}) = \langle \frac{4}{4}, \frac{0}{4}\rangle = \langle 1, 0\rangle$

We have the following examples with Pat=Full:

Example	Attributes										Target
	Alt	*Bar*	*Fri*	*Hun*	*Pat*	*Price*	*Rain*	*Res*	*Type*	*Est*	*WillWait*
X_2	T	F	F	T	Full	$	F	F	Thai	30–60	F
X_4	T	F	T	T	Full	$	F	F	Thai	10–30	T
X_5	T	F	T	F	Full	$$$	F	T	French	>60	F
X_9	F	T	F	T	Full	$	T	F	Burger	>60	F
X_{10}	T	T	T	T	Full	$$$	F	T	Italian	10–30	F
X_{12}	T	T	T	T	Full	$	F	F	Burger	30–60	T

So, $P(\text{Pat=Full}) = \frac{6}{12}$

And $\mathbf{P}(C \mid \text{Pat=Full}) = \langle \frac{2}{6}, \frac{4}{6} \rangle = \langle \frac{1}{3}, \frac{2}{3} \rangle$

Therefore:

$$\begin{aligned}
\text{Gain(Patrons)} &= 1 - \left(\frac{2}{12} \cdot I(\langle 0, 1 \rangle) + \frac{4}{12} \cdot I(\langle 1, 0 \rangle) + \frac{6}{12} \cdot I(\left\langle \frac{1}{3}, \frac{2}{3} \right\rangle) \right) \\
&= 1 - \left(\frac{2}{12} \cdot \Big[-(0 \log_2 0 + 1 \log_2 1) \Big] + \frac{4}{12} \cdot \Big[-(1 \log_2 1 + 0 \log_2 0) \Big] \right. \\
&\qquad \left. + \frac{6}{12} \cdot \Big[-\Big(\frac{1}{3} \log_2 \frac{1}{3} + \frac{2}{3} \log_2 \frac{2}{3} \Big) \Big] \right) \\
&= 1 - \left(\frac{2}{12} \cdot 0 + \frac{4}{12} \cdot 0 + \frac{6}{12} \cdot \Big[-\Big(\frac{1}{3} \log_2 \frac{1}{3} + \frac{2}{3} \log_2 \frac{2}{3} \Big) \Big] \right) \\
&= 1 - \left(\frac{1}{2} \cdot \Big[-\Big(\frac{1}{3} \log_2 \frac{1}{3} + \frac{2}{3} \log_2 \frac{2}{3} \Big) \Big] \right) \\
&= 1 - \frac{1}{2} I\left(\left\langle \frac{1}{3}, \frac{2}{3} \right\rangle \right).
\end{aligned}$$

$$\begin{aligned}
I\left(\left\langle \frac{1}{3}, \frac{2}{3} \right\rangle \right) &= -\left(\frac{1}{3} \log_2 \frac{1}{3} + \frac{2}{3} \log_2 \frac{2}{3} \right) \\
&= -\left(\frac{1}{3}(-\log_2 3) + \frac{2}{3}(\log_2 2 - \log_2 3) \right) \\
&= -\left(-\frac{1}{3} \log_2 3 + \frac{2}{3}(1 - \log_2 3) \right) \\
&= -\left(-\frac{1}{3} \log_2 3 + \frac{2}{3} - \frac{2}{3} \log_2 3 \right) \\
&= \log_2 3 - \frac{2}{3} \\
&\approx 1.5850 - 0.6667 \approx 0.9183.
\end{aligned}$$

Therefore,

$$\begin{aligned}
\text{Gain(Patrons)} &= 1 - \frac{1}{2} \cdot 0.9183 \\
&\approx 1 - 0.4592 \\
&\approx 0.5408b
\end{aligned}$$

Example 7.10. Similarly, we can compute the information gain of the attribute *Type*:

$$\text{Gain(Type)} = 1 - \left(\frac{2}{12} \cdot I(\langle 0.5, 0.5 \rangle) + \frac{2}{12} \cdot I(\langle 0.5, 0.5 \rangle) + \frac{4}{12} \cdot I(\langle 0.5, 0.5 \rangle) + \frac{4}{12} \cdot I(\langle 0.5, 0.5 \rangle) \right) \approx 0b$$

Idea

In order to construct a **simple** DT, we better prioritize the attributes that **maximize** information gain

Def 7.20. The following algorithm performs **Decision Tree Learning (DTL)** by recursively choosing the attribute that **maximizes** information gain as root of (sub)-tree:

Algorithm 2 Decision Tree Learning (DTL)

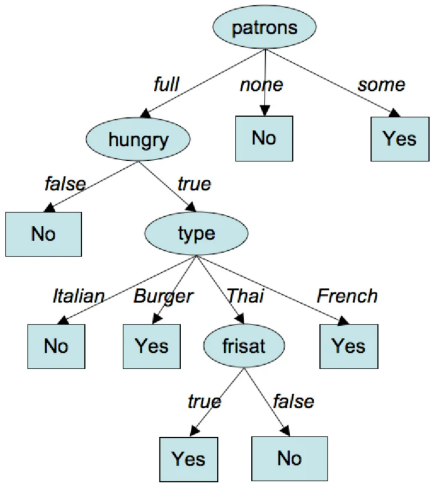
```
function DTL(examples, attributes, default)
    if examples is empty then
        return default
    else if all examples have the same classification then
        return the classification
    else if attributes is empty then
        return MODE(examples)
    else
        best ← CHOOSE-ATTRIBUTE(attributes, examples)
        tree ← a new decision tree with root test best
        m ← MODE(examples)
        for each value v_i of best do
            examples_i ← { e ∈ examples | best(e) = v_i }
            subtree ← DTL(examples_i, attributes \ best, m)
            add a branch to tree with label v_i and subtree subtree
        end for
        return tree
    end if
end function
```

Note

`MODE(examples)` returns the most frequent class label in the dataset. For examples X where $X_i = \langle x_1, x_2, \cdots, c \rangle$:

$$\text{MODE(examples)} = \arg\max_c |\{e \in X \mid \text{class}(e) = c\}|$$

Example 7.11. Here is a DT learned by DTL from the 12 examples using information gain maximization for `CHOOSE-ATTRIBUTE`:



Notice how the DT in example 7.11 is substantially simpler than the *true* DT showed in example 7.3.

Def 7.21. Given a supervised learning problem with a set of examples $T \subseteq A \times B$, we define the **error rate** of a hypothesis $h \in \mathcal{H}$ as the fraction of errors:

$$\frac{|\{\langle x, y \rangle \in T \mid h(x) \neq y\}|}{|T|}$$

Observation

A low error rate on the training set does not mean that a hypothesis generalizes well. (it might be too consistent)

Idea

Split example into training and testing.

Def 7.22. Holdout Cross Validation: a practice of splitting the data available for learning into a training set from which the learning algorithm produces a hypothesis h, and a **test set**, which is used for evaluating h

Note

A **test set** is a set of examples that is drawn from the same distribution as the training set, but is kept separate from it and is used only to estimate how well a learned hypothesis generalizes to new data.

Def 7.23. K-fold Cross Validation: Performing k rounds of learning, each with $\frac{1}{k}$ of the data as test set and average over the k error rates

Note

If $k = |\text{dom}(f)|$, then K-fold CV is called **leave one out cross validation (LOOCV)**

Def 7.24. Vector Norm: Let V be a vector space over $\mathbb{R}$. A **norm** on V is a function

$$\|\cdot\| : V \to \mathbb{R}$$

such that for all $u, v \in V$ and all scalars $\alpha \in \mathbb{R}$:

- **Positivity:** $\|v\| \geq 0$, and $\|v\| = 0$ iff $v = 0$
- **Absolute homogeneity:** $\|\alpha v\| = |\alpha| \, \|v\|$
- **Triangle inequality:** $\|u + v\| \leq \|u\| + \|v\|$

Def 7.25. ℓ_1 Norm: For $v = (v_1, \ldots, v_n) \in \mathbb{R}^n$, the ℓ_1 **norm** is

$$\|v\|_1 := \sum_{i=1}^{n} |v_i|.$$

Def 7.26. ℓ_2 Norm: For $v = (v_1, \ldots, v_n) \in \mathbb{R}^n$, the ℓ_2 **norm** is

$$\|v\|_2 := \sqrt{\sum_{i=1}^{n} v_i^2}.$$

Def 7.27. ℓ_p Norm: For $v = (v_1, \ldots, v_n) \in \mathbb{R}^n$ and $p \geq 1$, the ℓ_p **norm** is

$$\|v\|_p := \left(\sum_{i=1}^{n} |v_i|^p \right)^{1/p}.$$

Def 7.28. Loss Vector: Let $T = \langle (x_1, y_1), \ldots, (x_m, y_m) \rangle$ be a training set and let h be a hypothesis. The **loss vector** of h on T is

$$\vec{\ell}(h, T) := \langle\, y_1 - h(x_1),\ y_2 - h(x_2),\ \ldots,\ y_m - h(x_m) \,\rangle\,.$$

Notation

We sometimes write y_i as $f(x_i)$ where f is the target function we are trying to learn. And we express the predicted value by our hypothesis as $\hat{y}_i = h(x_i)$

Def 7.29. A **loss function** is a function

$$L : A \times B \times B \to \mathbb{R}_{\geq 0}$$

that assigns to each input $x \in A$, true label $y \in B$, and prediction $\hat{y} \in B$ a nonnegative real number measuring how bad it is to predict $\hat{y}$ instead of y on x.

If the loss does not depend on the input x, we write

$$L : B \times B \to \mathbb{R}_{\geq 0},$$

and simply use $L(y, \hat{y})$.

Def 7.30. ℓ_1 **Loss:** The ℓ_1 **loss** of a hypothesis h on a training set T is the ℓ_1 norm of its loss vector:

$$L_1(h, T) := \|\vec{\ell}(h, T)\|_1 = \sum_{i=1}^{m} |y_i - \hat{y}_i| .$$

Note

We might refer to the ℓ_1 Loss as the **absolute value loss**

Def 7.31. ℓ_2 **Loss:** The ℓ_2 **loss** of a hypothesis h on a training set T is the square of the ℓ_2 norm of its loss vector:

$$L_2(h, T) := \|\vec{\ell}(h, T)\|_2^2 = \sum_{i=1}^{m} \left(y_i - \hat{y}_i\right)^2.$$

Note

We might refer to the ℓ_2 Loss as the **squared error loss**

Def 7.32. 0/1 Loss: For a true label y and a prediction $\hat{y}$, the **0/1 loss** is

$$L_{0/1}(y, \hat{y}) := \begin{cases} 0, & \text{if } y = \hat{y}, \\ 1, & \text{if } y \neq \hat{y}. \end{cases}$$

This is the standard loss function for classification, and it is equal to the error rate on a single example.

Def 7.33. Generalization Loss / Expected Risk: Let $\mathcal{E} = X \times Y$ be the set of all possible examples and $\mathbf{P}(X, Y)$ is the prior probability distribution over its components, then the **expected generalization loss** for a hypothesis $h \in \mathcal{H}$ with respect to a loss function L is:

$$\text{GenLoss}_L(h) := \sum_{\langle x,y \rangle \in \mathcal{E}} L(y, h(x)) \cdot p(x, y)$$

The *best* hypothesis h^* is the one that minimizes the generalization loss:

$$h^* := \arg\min_{h \in \mathcal{H}} \text{GenLoss}_L(h)$$

Problem

$\mathbf{P}(X, Y)$ is unknown $\rightsquigarrow$ learners only **estimate** generalization loss

Def 7.34. Empirical Loss: Let L be a loss function and E a set of examples with $|E| = N$, then we define the **empirical loss** as:

$$\text{EmpLoss}_{L,E}(h) := \frac{1}{N} \left(\sum_{\langle x,y \rangle \in E} L(y, h(x)) \right)$$

The estimated best hypothesis $\hat{h}^*$ is the one that minimizes the empirical loss:

$$\hat{h}^* := \arg\min_{h \in \mathcal{H}} \text{EmpLoss}_{L,E}(h)$$

Intuition

Our aim is not to only minimize the empirical loss, we also want a *simple* hypothesis (i.e. finding a good hypotheses space $\mathcal{H}$) to avoid overfitting

Idea

Minimize the weighted sum of empirical loss and hypothesis complexity

Def 7.35. Let $\lambda \in \mathbb{R}$, $h \in \mathcal{H}$, and E a set of examples, then we call

$$\text{Cost}_{L,E} := \text{EmpLoss}_{L,E}(h) + \lambda \text{Complexity}(h)$$

the **total cost** of h on E

The total cost minimizing hypothesis is

$$\hat{h}^* := \arg\min_{h \in \mathcal{H}} \text{Cost}_{L,E}$$

Def 7.36. The process of finding a **total cost minimizing hypothesis**

$$\hat{h}^* := \arg\min_{h \in \mathcal{H}} \text{Cost}_{L,E}$$

is called **regularization**. Complexity in $\lambda \text{Complexity}(h)$ is called the **regularization function** or **hypothesis complexity**

Question

How can we be sure that our learning algorithm has produced a hypothesis that will predict the correct value for previously unseen inputs? In other words, how do we know that the hypothesis h is close to the target function f if we don't know what f is?

Some more questions:

- How many examples do we need to get a good h ?
- What hypotheses space $\mathcal{H}$ should we use?
- If the hypotheses space $\mathcal{H}$ is very complex, can we find the best $h \in \mathcal{H}$, or do we have to settle for a local maximum in $\mathcal{H}$?
- How complex h should be?
- How do we avoid overfitting?
- $\dots$

"Computational Learning Theory" tries to answer such questions using concepts from AI, Statistics, and Theoretical Computer Science

Basic idea of Computational Learning Theory

- Any hypothesis h that is **seriously wrong** will almost certainly be *found out* with high probability after a small number of examples, because it will make an incorrect prediction.
- Thus, if h is consistent with a sufficiently large set of training examples, it is unlikely to be **seriously wrong**
- $\rightsquigarrow$ h is **probably approximately correct**

Def 7.37. PAC Learning: Any inductive learning algorithm that returns hypotheses that are **probably approximately correct** is called a **PAC Learning** algorithm.

Let's only discuss PAC theorems for Boolean functions, for which 0/1 Loss is appropriate.

Idea

The error rate of a hypothesis h is the probability that h misclassifies a new example. What about if we fix an acceptable $\epsilon > 0$ as an **error threshold** for *"seriously bad"* hypotheses? (an error/loss that we can tolerate)

Def 7.38. A hypothesis h is called **approximately correct**, iff $\text{error}(h) \leq \epsilon$. We write $\mathcal{H}_b := \{h \in \mathcal{H} \mid \text{error}(h) > \epsilon\}$ for the **seriously bad** hypotheses

Let's compute the probability that $h_b \in \mathcal{H}_b$ is consistent with the first N examples

Let $h_b \in \mathcal{H}_b$ be a **seriously bad** hypothesis, i.e.

$$\text{error}(h_b) > \epsilon$$

This means that h_b misclassifies more than an ϵ fraction of all possible examples.

Probability that a bad hypothesis survives

For a single randomly drawn example:

$$P(h_b \text{ classifies the example correctly}) = 1 - \text{error}(h_b)$$

Since $\text{error}(h_b) > \epsilon$, we obtain:

$$P(h_b \text{ classifies correctly}) < 1 - \epsilon$$

Therefore, the probability that h_b correctly classifies all N independent training examples is at most:

$$P(h_b \text{ agrees with all } N \text{ examples}) \leq (1 - \epsilon)^N$$

Note

That means, even a bad hypothesis may accidentally classify a few examples correctly. However, the probability that it survives many examples without making a mistake decreases exponentially fast with N.

Considering all bad hypotheses

We now ask:

What is the probability that there exists at least one seriously bad hypothesis that is still consistent with all training examples?

Let:

$$\mathcal{H}_b = \{h \in \mathcal{H} \mid \text{error}(h) > \epsilon\}$$

be the set of all seriously bad hypotheses.
Each bad hypothesis survives with probability at most:

$$(1 - \epsilon)^N$$

Using the Union Bound

For each bad hypothesis:

$$h_b \in \mathcal{H}_b$$

define the event:

$$E_{h_b} = \text{“}h_b \text{ is consistent with all } N \text{ examples”}$$

Earlier we showed:

$$P(E_{h_b}) \leq (1-\epsilon)^N$$

We are interested in the probability that *at least one* bad hypothesis survives, i.e.

$$P(\mathcal{H}_b \text{ contains a consistent hypothesis}) = P\left(\bigcup_{h_b \in \mathcal{H}_b} E_{h_b}\right)$$

Using the union bound:

$$P\left(\bigcup_{h_b \in \mathcal{H}_b} E_{h_b}\right) \leq \sum_{h_b \in \mathcal{H}_b} P(E_{h_b})$$

Since every bad hypothesis satisfies:

$$P(E_{h_b}) \leq (1-\epsilon)^N$$

we obtain:

$$P(\mathcal{H}_b \text{ contains a consistent hypothesis}) \leq |\mathcal{H}_b|(1-\epsilon)^N$$

Thus, the probability that some seriously bad hypothesis survives decreases exponentially with the number of training examples.

Because $\mathcal{H}_b \subseteq \mathcal{H}$, that means $|\mathcal{H}_b| \leq |\mathcal{H}|$. We multiply both sides by $(1-\epsilon)^N$, we get:

$$|\mathcal{H}_b|\,(1-\epsilon)^N \leq |\mathcal{H}|\,(1-\epsilon)^N$$

Therefore:

$$P(\mathcal{H}_b \text{ contains a consistent hypothesis}) \leq |H_b|(1-\epsilon)^N \leq |\mathcal{H}|\,(1-\epsilon)^N$$

Considering only the upper bound:

$$P(\mathcal{H}_b \text{ contains a consistent hypothesis}) \leq |\mathcal{H}|\,(1-\epsilon)^N$$

This quantity is an **upper bound** on the probability that a seriously bad hypothesis survives all training examples.

We now introduce a parameter:

$$\delta > 0$$

which represents the maximum probability of failure that we are willing to tolerate.

In other words:

We want the probability that a bad hypothesis survives to be at most δ.

Which also means:

with probability at least $1-\delta$, no bad hypothesis survives training

Note

If no bad hypothesis survives training, that means every hypothesis that fits the training data must have error $\leq \epsilon$, so the learning algorithm must output a good hypothesis.

Therefore, we require:

$$P(\mathcal{H}_b \text{ contains a consistent hypothesis}) \leq \delta$$

Hence, it is sufficient to enforce:

$$|\mathcal{H}|\,(1-\epsilon)^N \leq \delta$$

This guarantees that the probability of accepting a seriously bad hypothesis is at most δ.

Let's solve this inequality for N. Divide both sides by $|\mathcal{H}|$:

$$(1-\epsilon)^N \leq \frac{\delta}{|\mathcal{H}|}$$

Take logarithms:

$$\log_2\left((1-\epsilon)^N\right) \leq \log_2\left(\frac{\delta}{|\mathcal{H}|}\right)$$

Using the logarithm rule:

$$\log_2(a^N) = N\log_2(a)$$

we obtain:

$$N\log_2(1-\epsilon) \leq \log_2(\delta) - \log_2(|\mathcal{H}|)$$

For small ϵ, we have: $\log_2(1-\epsilon) \approx -\epsilon$:

$$N(-\epsilon) \leq \log_2(\delta) - \log_2(|\mathcal{H}|)$$

Multiply by -1 (which flips the inequality):

$$N(\epsilon) \geq -\log_2(\delta) + \log_2(|\mathcal{H}|)$$

We use the identity $-\log_2(a) = \log_2(\frac{1}{a})$

$$N\epsilon \geq \log_2(|\mathcal{H}|) + \log_2\left(\frac{1}{\delta}\right)$$

Finally, divide by ϵ:

$$N \geq \frac{1}{\epsilon}\left(\log_2\left(\frac{1}{\delta}\right) + \log_2(|\mathcal{H}|)\right)$$

That means, to guarantee that a learned hypothesis is within error ϵ (approximately correct) with probability at least $1-\delta$, you need at least:

$$\frac{1}{\epsilon}\left(\log_2\left(\frac{1}{\delta}\right) + \log_2(|\mathcal{H}|)\right) \text{ examples}$$

Interpretation

- Smaller ϵ means we tolerate less error $\Rightarrow$ more examples are required
- Smaller δ means we want higher confidence $\Rightarrow$ more examples are required
- Larger hypothesis spaces $|\mathcal{H}|$ require more examples because there are more hypotheses that could accidentally fit the training data

Def 7.39. The number of required training examples N as a function of ϵ, δ, and $|\mathcal{H}|$ is called the **sample complexity**.

Example 7.12. If $\mathcal{H}$ is the set of n-ary Boolean functions, then $|\mathcal{H}| = 2^{2^n}$.

- sample complexity grows with $\mathcal{O}(\log_2(2^{2^n})) = \mathcal{O}(2^n)$
- There are 2^n possible examples
- $\rightsquigarrow$ PAC learning for Boolean functions needs to see (nearly) all examples!

If your hypotheses space is too expressive (like all Boolean functions), then sample complexity becomes exponential.

Idea

Restrict $\mathcal{H}$ in some way. For example, focusing on "*learnable subsets*" of $\mathcal{H}$

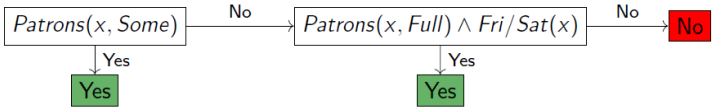
Def 7.40. A **decision list** consists of a sequence of tests, each of which is a conjunction of literals.

- If a test succeeds when applied to an example description, the decision list specifies the value to be returned
- If the test fails, processing continues with the next test in the list

Note

Like Decision Trees, but restricted branching, but more complex tests.

Example 7.13. Here is a decision list for the Restaurant problem:



Lemma 7.1. Given arbitrary size conditions, decision lists can represent arbitrary Boolean functions

Def 7.41. The set of decision lists where tests are of conjunctions of at most k literals is denoted by k-DL

Example 7.14. The decision list from example 7.13 is in 2-DL

Observation

k-DL contains k-DT, the set of Decision Trees of depth at most k

Def 7.42. We denote the set of k-DL decision lists with at most n Boolean attributes with k-DL(n). The set of conjunctions of at most k literals over n attributes is written as $\text{Conj}(k, n)$

Example 7.15. If we have two attributes x_1, x_2 and we want at most 2 literals, then all the possible tests are:

$$\text{Conj}(k, n) = \{x_1, \neg x_1, x_2, \neg x_2, x_1 \wedge x_2, x_1 \wedge \neg x_2, \neg x_1 \wedge x_2, \neg x_1 \wedge \neg x_2, x_1 \wedge \neg x_2, x_2 \wedge x_2\}$$

Note

In the previous example, $|\text{Conj}(k, n)| = 10$

Question

Can we generalize this?

Solution

If we have n attributes. Each attribute x_i can appear as a positive literal or as a negative literal. That gives us $2n$ literals to choose from. A test is made by choosing i literals from this pool of $2n$. The number of ways to choose exactly i items from $2n$ is the binomial coefficient:

$$\binom{2n}{i}$$

Since a test can have at most k literals, we have to sum up all possibilities for tests of length 1, 2, all the way up to k:

$$|\text{Conj}(k, n)| = \sum_{i=1}^{k} \binom{2n}{i}$$

Note

k is usually treated as a small constant (like 2, or 3), while n (the number of features) can be very large. The largest term in this sum is:

$$\binom{2n}{k}$$

$$\binom{2n}{k} \approx \frac{(2n)^k}{k!} \qquad \text{by definition of combinations}$$

Note

Since $2k$ and $k!$ are small constants, the dominant growth factor is n^k

Idea

We can bound the number of possible tests as:

$$|\text{Conj}(k,n)| = \mathcal{O}(n^k)$$

Question

What is the size of this restricted learnable hypotheses space (k-DL(n))?

We need to calculate the maximum number of unique decision lists we can build. Let $C = |\text{Conj}(k,n)|$ be the total number of available tests. To build a decision list, we must do two things:

1. Select tests and assign outputs: For every single test out of the C avaiable, it has 3 possible states in my list:
   - it is included and returns Yes
   - it is included and returns No
   - it is not included in the list at all

   Because there are C tests, and each has 3 independent choices, there are exactly 3^C ways to select tests and assign their return values.
2. Order the tests: A decision list is strictly ordered. If we choose a subset of tests, we can arrange them in any sequence. The absolute maximum number of tests we could possibly select is C. The number of ways to order C items is $C!$

Putting it all together, the total number of decision lists is bounded by the combinations of selections multiplied by their possible orderings:

$$|k - \text{DL}(n)| \leq 3^C \cdot C!$$

Take the logarithm:

$$\log_2 |k - \text{DL}(n)| \leq \log_2(3^C \cdot C!)$$

$$\log_2 |k - \text{DL}(n)| \leq C \log_2 3 \cdot \log_2 C!$$

$$\log_2 C! \leq C \log_2(C) \qquad \text{(Stirling's approximation)}$$

$$\log_2 |k - \text{DL}(n)| \leq C \log_2 3 \cdot C \log_2(C)$$

Note

$C \log_2(C)$ grows much faster than $C \log_2 3$

Idea

We ignore $C \log_2 3$ and bound the size using $\mathcal{O}(\cdot)$ notation

$$\log_2 |k - \text{DL}(n)| = \mathcal{O}(C \log_2(C))$$

Observation

$C = |\text{Conj}(k,n)| = \mathcal{O}(n^k)$

$$\log_2 |k - \text{DL}(n)| = \mathcal{O}(n^k \log_2(n^k))$$

We raise 2 to the power of both sides:

$$|k - \text{DL}(n)| = 2^{\mathcal{O}(n^k \log_2(n^k))}$$

Plug this into the equation for the sample complexity to obtain:

$$N \geq \frac{1}{\epsilon}\left(\log_2\left(\frac{1}{\delta}\right) + \log_2(\mathcal{O}(n^k \log_2(n^k)))\right)$$

Intuition

Before, when our hypotheses space was all Boolean functions $\rightsquigarrow$ exponential sample complexity. With $k - \text{DL}(n)$ as our hypotheses space, we have polynomial sample complexity

Take home message: If you restirct your hypotheses space structure, learning becomes feasible

8 Regression

Def 8.1. A function with one argument is called **univariate** or **unary**

Def 8.2. A univariate function $f : \mathbb{R} \to \mathbb{R}$ is called **linear** iff it simply scales its input by a constant factor:

$$f(x) = wx$$

for some constant scalar $w \in \mathbb{R}$.

Observation

Geometrically, this represents a straight line that passes exactly through the origin $(0, 0)$.

Example 8.1. If $w = 3$, then $f(x) = 3x$.

- If $x = 2 \implies f(2) = 6$
- If $x = 0 \implies f(0) = 0$

Def 8.3. A univariate function $h_w : \mathbb{R} \to \mathbb{R}$ of the form:

$$h_w(x) = w_1 x + w_0$$

is called an **affine linear model**. The parameter w_1 is a scalar called the **slope**, and w_0 is a scalar called the **intercept** or **bias**.

Observation

An affine linear function is simply a **strict linear** function plus a constant baseline offset. Because of the $+w_0$ term, if you pass in zero, the output is the offset ($h_w(0) = w_0$). Geometrically, this is a straight line that can cross the vertical axis anywhere, not just at the origin.

Example 8.2. Let $w_1 = 3$ and $w_0 = 2$, which gives the model $h_w(x) = 3x + 2$.

- If $x = 2 \implies h_w(2) = 3(2) + 2 = 8$
- If $x = 0 \implies h_w(0) = 3(0) + 2 = 2$

Def 8.4. Let $E \subseteq \mathbb{R} \times \mathbb{R}$ be a set of examples (where each example is a pair of scalar inputs and scalar outputs (x_i, y_i)). The task of finding the constant scalars w_1 and w_0 such that the affine linear model $h_w(x) = w_1 x + w_0$ fits the data points in E as closely as possible is called **univariate linear regression**.

Observation

In this task, we are strictly mapping a single scalar input variable to a single scalar output variable. Geometrically, this means finding the optimal straight line through a cloud of data points on a two-dimensional grid.

Example 8.3. We have a dataset $E = \{(1, 3), (2, 5), (3, 7)\}$. The goal of univariate linear regression is to find the scalars w_1 and w_0. Here, the perfect fit is reached when $w_1 = 2$ and $w_0 = 1$, yielding the model $h_w(x) = 2x + 1$.

Question

How to find parameters w?

Idea

We minimize ℓ_2 Loss

Remember

ℓ_2 Loss (squared error loss) for N examples is $\sum_{i=1}^{N} \left(y_i - \hat{y}_i\right)^2$

Solution

We have to find the best tuple w, call it w^*.

$$w^* := \arg\min_{w_0, w_1} \sum_{i=1}^{N} \big(y_i - (w_1 x_i + w_0)\big)^2$$

Observation

$$\sum_{i=1}^{N} \big(y_i - (w_1 x_i + w_0)\big)^2$$

is differentiable, a minimum is attained when its partial derivatives with respect to the parameters vanish ($= 0$).

Let $L(h_w) = \sum_{i=1}^{N} \big(y_i - (w_1 x_i + w_0)\big)^2$. To minimize the loss, we find where the partial derivatives with respect to each parameter vanish ($= 0$):

$$\frac{\partial L}{\partial w_0} = 0 \qquad \text{and} \qquad \frac{\partial L}{\partial w_1} = 0$$

Applying the chain rule ($\frac{d}{dx}[g(x)^2] = 2g(x) \cdot g'(x)$), the derivatives distribute inside the summation:

$$\frac{\partial L}{\partial w_0} = \sum_{i=1}^{N} 2\big(y_i - (w_1 x_i + w_0)\big)(-1)$$

$$\frac{\partial L}{\partial w_1} = \sum_{i=1}^{N} 2\big(y_i - (w_1 x_i + w_0)\big)(-x_i)$$

Setting these total gradients to zero yields the normal system of equations:

$$\sum_{i=1}^{N} 2\big(y_i - (w_1 x_i + w_0)\big)(-1) = 0 \tag{5}$$

$$\sum_{i=1}^{N} 2\big(y_i - (w_1 x_i + w_0)\big)(-x_i) = 0 \tag{6}$$

Solving for w_0 Divide equation (5) by -2 and expand the summation across the individual scalar terms:

$$\sum_{i=1}^{N} \big(y_i - w_1 x_i - w_0\big) = 0$$

$$\sum_{i=1}^{N} y_i - w_1 \sum_{i=1}^{N} x_i - \sum_{i=1}^{N} w_0 = 0$$

Because w_0 is a constant independent of the index i, summing it N times simplifies to $\sum_{i=1}^{N} w_0 = N w_0$:

$$\sum_{i=1}^{N} y_i - w_1 \sum_{i=1}^{N} x_i - N w_0 = 0$$

Isolating $N w_0$ gives the explicit baseline offset formula:

$$w_0 = \frac{\sum_{i=1}^{N} y_i - w_1 \sum_{i=1}^{N} x_i}{N} \tag{7}$$

Solving for w_1 Divide equation (6) by -2 and distribute the factor of x_i inside the parenthesis:

$$\sum_{i=1}^{N} \big(y_i - w_1 x_i - w_0\big) x_i = 0$$

$$\sum_{i=1}^{N} x_i y_i - w_1 \sum_{i=1}^{N} x_i^2 - w_0 \sum_{i=1}^{N} x_i = 0$$

Substitute our expression for w_0 from equation (7) into this expression:

$$\sum_{i=1}^{N} x_i y_i - w_1 \sum_{i=1}^{N} x_i^2 - \left(\frac{\sum_{i=1}^{N} y_i - w_1 \sum_{i=1}^{N} x_i}{N} \right) \sum_{i=1}^{N} x_i = 0$$

Multiply the entire equation by N to clear out the denominator:

$$N \sum_{i=1}^{N} x_i y_i - N w_1 \sum_{i=1}^{N} x_i^2 - \left(\sum_{i=1}^{N} y_i \right) \left(\sum_{i=1}^{N} x_i \right) + w_1 \left(\sum_{i=1}^{N} x_i \right)^2 = 0$$

Group all terms containing w_1 on one side of the equation:

$$w_1 \left[N \sum_{i=1}^{N} x_i^2 - \left(\sum_{i=1}^{N} x_i \right)^2 \right] = N \sum_{i=1}^{N} x_i y_i - \left(\sum_{i=1}^{N} x_i \right) \left(\sum_{i=1}^{N} y_i \right)$$

Observation

The equations (5) and (6) yield a unique closed-form solution:

$$w_1 = \frac{N(\sum_{i=1}^{N} x_i y_i) - (\sum_{i=1}^{N} x_i)(\sum_{i=1}^{N} y_i)}{N(\sum_{i=1}^{N} x_i^2) - (\sum_{i=1}^{N} x_i)^2} \qquad w_0 = \frac{(\sum_{i=1}^{N} y_i) - w_1(\sum_{i=1}^{N} x_i)}{N}$$

Observation

Thus, univariate linear regression admits an exact closed-form solution and does not require iterative optimization methods.

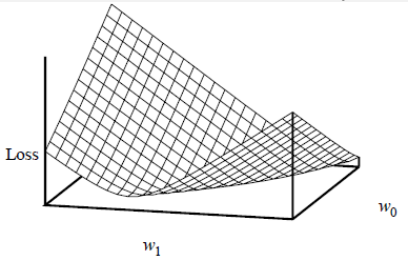
Def 8.5. The **weight space** of a parametric model is the space of all possible combinations of the parameters (weights). Loss minimization in a weight space is called **weight fitting**

Observation

The weight space of univariate linear regression is $\mathbb{R}^2$

Note

The ℓ_2 Loss is **convex** for any univariate linear regression problem $\rightsquigarrow$ there are no *local minima*



Def 8.6. A function that takes a collection of $n \geq 2$ distinct input arguments $(x_1, x_2, \ldots, x_n)$ and maps them to a single scalar output is called **multivariate**.

Observation

Following the same logic as the univariate affine function, a **multivariate affine function** scales each individual input argument by its own distinct scalar weight (w_i) and adds a shared baseline intercept (w_0):

$$f(x_1, \dots, x_n) = w_0 + w_1x_1 + w_2x_2 + \cdots + w_nx_n = w_0 + \sum_{i=1}^{n} w_ix_i$$

where all parameters $w_0, w_1, \dots, w_n$ are individual scalars.

Def 8.7. Let $E \subseteq \mathbb{R}^n \times \mathbb{R}$ be a set of examples (where each example consists of n scalar inputs and one scalar output $(x_{i,1}, x_{i,2}, \dots, x_{i,n}, y_i)$). The task of finding the constant scalar parameters $w_0, w_1, \dots, w_n$ such that the model:

$$h_w(x_1, \dots, x_n) = w_0 + w_1x_1 + w_2x_2 + \cdots + w_nx_n$$

fits the data points in E as closely as possible is called **multivariate linear regression**.

Observation

Geometrically, instead of fitting a 2D straight line to a flat grid of data points, multivariate linear regression with $n = 2$ features fits a flat 3D plane through a cloud of data points in a three-dimensional space. For $n > 2$, it fits a flat multi-dimensional hyperplane.

Notation

To avoid writing out long summations for n features, we package our individual scalar inputs and scalar weights into **vectors**.
First, we define a parameter weight vector:

$$\mathbf{w} := \begin{pmatrix} w_0 \\ w_1 \\ \vdots \\ w_n \end{pmatrix}$$

Next, to account for the baseline intercept w_0, we introduce a constant dummy scalar $x_0 := 1$. This allows us to define our complete input feature vector:

$$\mathbf{x} := \begin{pmatrix} x_0 \\ x_1 \\ \vdots \\ x_n \end{pmatrix}$$

Using these vector representations, the entire multivariate affine model can be compressed compactly into a single matrix multiplication or dot product:

$$h_{\mathbf{w}}(\mathbf{x}) = \mathbf{w}^T\mathbf{x} = \mathbf{w} \cdot \mathbf{x} = \sum_{i=0}^{n} w_ix_i$$

where $\mathbf{w}^T\mathbf{x}$ denotes matrix multiplication (multiplying a horizontal row vector by a vertical column vector) and $\mathbf{w} \cdot \mathbf{x}$ denotes the standard algebraic dot product.

Notation

When we have a dataset of N training examples, we use a superscript (i) to denote the i-th example vector. Each individual example vector $\mathbf{x}^{(i)}$ is composed of n distinct scalar features plus our constant dummy feature $x_0^{(i)} = 1$:

$$\mathbf{x}^{(i)} := \begin{pmatrix} x_0^{(i)} \\ x_1^{(i)} \\ \vdots \\ x_n^{(i)} \end{pmatrix} = \begin{pmatrix} 1 \\ x_{i,1} \\ \vdots \\ x_{i,n} \end{pmatrix}$$

where $x_{i,j}$ represents the scalar value of the j-th feature for the i-th data instance.

Def 8.8. Let $E = \{(\mathbf{x}^{(1)}, y_1), (\mathbf{x}^{(2)}, y_2), \ldots, (\mathbf{x}^{(N)}, y_N)\}$ be a set of N examples. The total ℓ_2 Loss for multivariate linear regression is the sum of the squared errors across all individual example vectors:

$$L(\mathbf{w}) = \sum_{i=1}^{N} \left(y_i - h_{\mathbf{w}}(\mathbf{x}^{(i)})\right)^2 = \sum_{i=1}^{N} \left(y_i - \mathbf{w}^T \mathbf{x}^{(i)}\right)^2$$

Observation

Just like the univariate case, this multivariate loss function has a unique, exact closed-form solution that can be computed analytically in a single step. This solution is known as the **Normal Equations**:

$$\mathbf{w}^* = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$

Problem

While a perfect mathematical equation exists to find $\mathbf{w}^*$ immediately, the matrix inversion operation: $(\mathbf{X}^T\mathbf{X})^{-1}$ is a bottleneck. Inverting a matrix of size $(n+1) \times (n+1)$ requires roughly $\mathcal{O}(n^3)$ computational operations. If a dataset has $n = 50,000$ features, computing $50,000^3$ operations requires an immense amount of time and memory, making it completely impossible to calculate on standard hardware.

Question

In a multi-dimensional parameter space, how do we know exactly which direction to tweak our weight vector $\mathbf{w}$ so that the loss decreases?

Def 8.9. The **gradient** of a multivariate function $L(\mathbf{w})$, denoted as $\nabla L(\mathbf{w})$, is a vector of its partial derivatives. It points in the direction of the steepest ascent of the function at a given point. Consequently, the negative gradient $-\nabla L(\mathbf{w})$ points in the direction of the steepest descent.

Idea

We use the **Gradient Descent** algorithm. By calculating the negative gradient at our current weight configuration, we determine the direction that decreases our error the fastest. We then take a small step in that direction and repeat the process until the weights converge to the minimum.

Def 8.10. Let $\alpha \in \mathbb{R}^+$ be a small scalar value called the **learning rate** (or step size). The iterative update rule for **Gradient Descent** is defined as:

$$\mathbf{w}^{(t+1)} := \mathbf{w}^{(t)} - \alpha \nabla L(\mathbf{w}^{(t)})$$

Before evaluating the entire gradient vector simultaneously, we can look at how gradient descent updates a single individual weight parameter w_j (where $j \in \{0, 1, \ldots, n\}$) using individual scalar components.

Applying the chain rule to our summation loss function, the partial derivative with respect to a specific weight w_j is:

$$\frac{\partial L}{\partial w_j} = \sum_{i=1}^{N} 2\left(y_i - \mathbf{w}^T \mathbf{x}^{(i)}\right)\left(-x_j^{(i)}\right)$$

Observation

Factoring out the constant -2, the partial derivative shows how much our j-th scalar feature contributes to the total error across all N examples:

$$\frac{\partial L}{\partial w_j} = -2 \sum_{i=1}^{N} \left(y_i - \mathbf{w}^T \mathbf{x}^{(i)}\right) x_j^{(i)}$$

Note

By isolating this specific coordinate of the gradient, the individual scalar update rule for a single parameter w_j at iteration step t is defined as:

$$w_j^{(t+1)} := w_j^{(t)} + 2\alpha \sum_{i=1}^{N} \left(y_i - \mathbf{w}^T \mathbf{x}^{(i)}\right) x_j^{(i)}$$

where $x_j^{(i)}$ is the specific scalar value of the j-th feature for the i-th training instance, and $x_0^{(i)} = 1$ for the bias parameter w_0.

Def 8.11. When the gradient is calculated by summing the errors across the **entire dataset** of N examples before performing a single weight update, the algorithm is called **Batch Gradient Descent**.

$$w_j^{(t+1)} := w_j^{(t)} + 2\alpha \sum_{i=1}^{N} \left(y_i - \mathbf{w}^T \mathbf{x}^{(i)}\right) x_j^{(i)}$$

Problem

While Batch Gradient Descent takes very stable, direct steps toward the minimum, it scales poorly. If the dataset contains millions of examples ($N > 10^6$), the computer must perform millions of operations just to make a single tiny adjustment to the weights, making it incredibly slow.

Def 8.12. When the weight parameters are updated iteratively after examining only **one single, randomly selected training example** $(\mathbf{x}^{(i)}, y_i)$, the algorithm is called **Stochastic Gradient Descent**.

$$w_j^{(t+1)} := w_j^{(t)} + 2\alpha \left(y_i - \mathbf{w}^T \mathbf{x}^{(i)}\right) x_j^{(i)}$$

Note

Remember Def 7.36. For multivariate linear regression, the complexity of a hypothesis $h_{\mathbf{w}}$ is determined by the size of its weight parameters. To prevent overfitting, we exclude the baseline intercept w_0 from the complexity calculation and only penalize the feature weights $w_1, w_2, \ldots, w_n$.

Def 8.13. When we define the hypothesis complexity as the sum of the **squared magnitudes** of the feature weights, the process is called L_2 **Regularization** (or **Ridge Regression**):

$$\text{Complexity}(h_{\mathbf{w}}) = \|\mathbf{w}_{1:n}\|_2^2 = \sum_{j=1}^{n} w_j^2$$

The total cost function becomes:

$$\text{Cost}_{L,E}(\mathbf{w}) = \sum_{i=1}^{N} \left(y_i - \mathbf{w}^T \mathbf{x}^{(i)}\right)^2 + \lambda \sum_{j=1}^{n} w_j^2$$

Observation

L_2 regularization heavily penalizes exceptionally large weight values because it squares each parameter. This forces the optimization to distribute weight softly across all features, driving all parameters closer to zero without making them exactly zero.

Def 8.14. When we define the hypothesis complexity as the sum of the **absolute magnitudes** of the feature weights, the process is called L_1 **Regularization** (or **Lasso Regression**):

$$\text{Complexity}(h_{\mathbf{w}}) = \|\mathbf{w}_{1:n}\|_1 = \sum_{j=1}^{n} |w_j|$$

The total cost function becomes:

$$\text{Cost}_{L,E}(\mathbf{w}) = \sum_{i=1}^{N} \left(y_i - \mathbf{w}^T \mathbf{x}^{(i)}\right)^2 + \lambda \sum_{j=1}^{n} |w_j|$$

Observation

L_1 Regularization tends to produce **sparse models**, i.e. it sets many weights to 0, effectively declaring the corresponding attributes to be irrelevant. hypotheses that discard attributes can be easier for a human to understand, and may be less likely to overfit

Question

Can we use our existing multivariate linear model $h_{\mathbf{w}}(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$ to predict binary categorical outcomes, such as deciding whether an email is **Spam** (1) or **Not Spam** (0)?

Problem

Using linear regression for binary classification causes severe issues:

- **Unbounded Outputs:** A linear model $\mathbf{w}^T\mathbf{x}$ can output any real value from $-\infty$ to $+\infty$. However, probabilities must strictly reside between 0 and 1.
- **Sensitivity to Outliers:** If we look at data points far away from the decision boundary, the linear fit tilts drastically, altering the classification threshold even though the underlying classification logic hasn't changed.

Def 8.15. A **decision boundary** is a line (or a plane) that separates two classes of points. A linear decision boundary is called a **linear separator** and data that admits one are called **linearly separable**

Idea

To enforce a hard binary classification across this decision boundary, we can pass our linear model through a thresholding mechanism that maps any real number directly to exactly 0 or 1.

Def 8.16. The **Heaviside step function** (or threshold activation function) $\mathcal{T} : \mathbb{R} \to \{0, 1\}$ is defined as:

$$\mathcal{T}(z) := \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases}$$

Def 8.17. When we wrap our multivariate linear model inside a step function, we obtain the **Perceptron** hypothesis model:

$$h_{\mathbf{w}}(\mathbf{x}) := \mathcal{T}(\mathbf{w}^T\mathbf{x}) = \begin{cases} 1 & \text{if } \mathbf{w}^T\mathbf{x} \geq 0 \\ 0 & \text{if } \mathbf{w}^T\mathbf{x} < 0 \end{cases}$$

Observation

Because the step function cannot be differentiated to find a gradient (its derivative is zero everywhere except at $z = 0$, where it is undefined), we cannot use standard gradient descent. Instead, we use an error-driven learning rule to adjust the weights whenever the model makes a mistake.

Def 8.18. Let $\alpha \in \mathbb{R}^+$ be the learning rate, $y_i \in \{0, 1\}$ be the true label, and $h_{\mathbf{w}}(\mathbf{x}^{(i)}) \in \{0, 1\}$ be the Perceptron's prediction. The **Perceptron Learning Rule** updates a single individual weight component w_j at iteration step t according to:

$$w_j^{(t+1)} := w_j^{(t)} + \alpha\big(y_i - h_{\mathbf{w}}(\mathbf{x}^{(i)})\big)x_j^{(i)}$$

where an update is only triggered if the model misclassifies the training instance i.

Observation

Let's analyze how this error-driven correction rule automatically fixes mistakes:

- **Correct Prediction:** If $y_i = h_{\mathbf{w}}(\mathbf{x}^{(i)})$, the error term $(y_i - h_{\mathbf{w}}(\mathbf{x}^{(i)})) = 0$. The weight remains unchanged: $w_j^{(t+1)} := w_j^{(t)}$.
- **False Negative:** If $y_i = 1$ but $h_{\mathbf{w}}(\mathbf{x}^{(i)}) = 0$, the error term is $+1$. The rule adds $\alpha x_j^{(i)}$ to the weight, which increases $\mathbf{w}^T\mathbf{x}^{(i)}$ for the next iteration.
- **False Positive:** If $y_i = 0$ but $h_{\mathbf{w}}(\mathbf{x}^{(i)}) = 1$, the error term is -1. The rule subtracts $\alpha x_j^{(i)}$ from the weight, lowering $\mathbf{w}^T\mathbf{x}^{(i)}$.

Theorem 8.1. If a training set E is linearly separable, then the Perceptron Learning Rule will converge to a valid linear separator in a finite number of iteration steps, regardless of the choice of the initial weight vector $\mathbf{w}^{(0)}$.

Problem

If the training data is **not** linearly separable (for example, the classic non-linear XOR function), the Perceptron Learning Rule loop will run indefinitely, fluctuating back and forth between bad weight states without ever converging.

Theorem 8.2. If examples E is not linearly separable, finding a weight configuration $\mathbf{w}$ that minimizes the classification error is **NP-hard**. However, an approximation of the minimal-error hypothesis can be reached by applying an iterative sequence where the learning rate α decays over time:

$$\lim_{t\to\infty} \alpha^{(t)} = 0 \qquad \text{and} \qquad \sum_{t=1}^{\infty} \alpha^{(t)} = \infty$$

Idea

To overcome the optimization bottlenecks of the non-differentiable step function, we shift from hard binary decisions to soft, probabilistic outputs. Instead of predicting a discrete class label directly, we predict the continuous probability that an example belongs to a positive class.

Def 8.19. The standard **logistic function** (commonly referred to as the **sigmoid function**) is a smooth, differentiable mapping $\sigma : \mathbb{R} \to (0, 1)$ defined as:

$$\sigma(z) := \frac{1}{1 + e^{-z}}$$

Observation

The logistic function possesses structural properties that resolve our classification issues:

- **Bounded Range:** For any real input $z \in \mathbb{R}$, the output is strictly bounded within the open interval $(0, 1)$, matching the mathematical requirements of a probability.
- **Symmetry:** At $z = 0$, $\sigma(0) = \frac{1}{1+1} = 0.5$. As $z \to +\infty$, $\sigma(z) \to 1$, and as $z \to -\infty$, $\sigma(z) \to 0$.

Because the logistic function is smooth and continuous, we can compute its exact analytical derivative. This derivative exhibits a unique property: it can be expressed completely in terms of the function's own output.

Observation

To find the derivative of $\sigma(z) = (1 + e^{-z})^{-1}$, we apply the chain rule and the quotient rule:

$$\frac{d}{dz}\sigma(z) = -1(1 + e^{-z})^{-2} \cdot (-e^{-z}) = \frac{e^{-z}}{(1 + e^{-z})^2}$$

We can split this fraction into two separate terms to expose the original function:

$$\frac{d}{dz}\sigma(z) = \left(\frac{1}{1 + e^{-z}}\right)\left(\frac{e^{-z}}{1 + e^{-z}}\right) = \sigma(z)\left(\frac{1 + e^{-z} - 1}{1 + e^{-z}}\right)$$

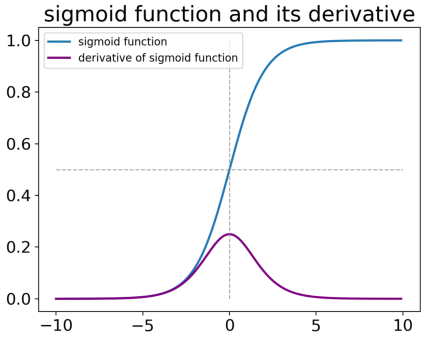
$$\frac{d}{dz}\sigma(z) = \sigma(z)\left(\frac{1 + e^{-z}}{1 + e^{-z}} - \frac{1}{1 + e^{-z}}\right) = \sigma(z)(1 - \sigma(z))$$

Def 8.20. The **derivative of the logistic function** with respect to its input argument z is given by:

$$\sigma'(z) := \sigma(z)\big(1 - \sigma(z)\big)$$

Observation

The bell-shaped curve of $\sigma'(z)$ reveals an important optimization trait: the gradient reaches its maximum value of 0.25 at exactly $z = 0$. As z moves far away from zero toward $\pm\infty$, the output $\sigma(z)$ approaches 0 or 1, causing the derivative $\sigma'(z)$ to vanish toward 0.



Idea

By passing our vector dot-product through the logistic mapping, we construct a hypothesis model that outputs a continuous probability score representing the confidence that a given instance belongs to the positive class.

Def 8.21. Let $\mathbf{x} \in \mathbb{R}^{n+1}$ be an input feature vector and $\mathbf{w} \in \mathbb{R}^{n+1}$ be the parameter weight vector. The **Logistic Regression** hypothesis model $h_{\mathbf{w}}(\mathbf{x}) : \mathbb{R}^{n+1} \to (0, 1)$ is defined as:

$$h_{\mathbf{w}}(\mathbf{x}) := \sigma(\mathbf{w}^T\mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^T\mathbf{x}}}$$

Note

This output explicitly models the conditional probability of the target label being positive given the features: $P(y = 1 \mid \mathbf{x}, \mathbf{w})$.

Idea

Because the output probability $h_{\mathbf{w}}(\mathbf{x})$ is a smooth, continuous curve between 0 and 1, we must establish a clear thresholding standard to make a final discrete binary prediction $\hat{y} \in \{0, 1\}$. By default, we assign an instance to the positive class if the model is more likely than not to belong there ($P \geq 0.5$).

To find the exact coordinates where the model switches its prediction from a negative class (0) to a positive class (1), we set the output probability to this 0.5 threshold:

$$h_{\mathbf{w}}(\mathbf{x}) = 0.5$$

Substituting the definition of the logistic regression hypothesis:

$$\frac{1}{1 + e^{-\mathbf{w}^T\mathbf{x}}} = \frac{1}{2}$$

For these fractions to be equal, their denominators must be identical:

$$1 + e^{-\mathbf{w}^T\mathbf{x}} = 2 \implies e^{-\mathbf{w}^T\mathbf{x}} = 1$$

Taking the natural logarithm (ln) of both sides isolates our input exponent:

$$-\mathbf{w}^T\mathbf{x} = \ln(1)$$

Since $\ln(1) = 0$, multiplying by -1 gives the threshold boundary equation:

$$\mathbf{w}^T\mathbf{x} = 0$$

Note

The algebraic condition $\mathbf{w}^T\mathbf{x} = 0$ defines the **decision boundary** of the logistic regression model. Because this dot product is a linear combination of the parameters and features, the boundary forms a flat, straight hyperplane embedded in the feature space $\mathbb{R}^n$.

Observation

This derivation reveals our final classification rule explicitly:

$$\hat{y} = \begin{cases} 1 & \text{if } h_{\mathbf{w}}(\mathbf{x}) \geq 0.5 \iff \mathbf{w}^T\mathbf{x} \geq 0 \\ 0 & \text{if } h_{\mathbf{w}}(\mathbf{x}) < 0.5 \iff \mathbf{w}^T\mathbf{x} < 0 \end{cases}$$

Take-Home Message

Even though Logistic Regression models continuous, curved probabilities using the non-linear sigmoid function (σ), the boundary it creates to separate classes is determined entirely by the linear equation $\mathbf{w}^T\mathbf{x} = 0$. Consequently, the boundary is always a flat straight line in 2D, a flat plane in 3D, or a flat hyperplane in higher dimensions. This proves mathematically that Logistic Regression is strictly a **linear classifier** and can only separate data that is linearly separable.

Idea

We can use ℓ_2 Loss and update the weights according to gradient descent

Remember: The update rule for gradient descent:

$$\mathbf{w}^{(t+1)} := \mathbf{w}^{(t)} - \alpha \nabla L(\mathbf{w}^{(t)})$$

Remember: ℓ_2 Loss:

$$\sum_{i=1}^{m} \big(y_i - h_{\mathbf{w}}(\mathbf{x})\big)^2.$$

Let's derive stochastic gradient descent for logistic regression

$$h_{\mathbf{w}}(\mathbf{x}) := \sigma(\mathbf{w}^T\mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^T\mathbf{x}}}$$

Applying the chain rule to the single-instance squared loss function, the partial derivative with respect to a specific weight component w_j is:

$$\frac{\partial L}{\partial w_j} = \frac{\partial}{\partial w_j}\big(y_i - h_{\mathbf{w}}(\mathbf{x}^{(i)})\big)^2$$

By the power rule, we bring down the exponent 2 and multiply by the derivative of the inner function with respect to w_j:

$$\frac{\partial L}{\partial w_j} = 2\big(y_i - h_{\mathbf{w}}(\mathbf{x}^{(i)})\big) \cdot \frac{\partial}{\partial w_j}\big(y_i - h_{\mathbf{w}}(\mathbf{x}^{(i)})\big)$$

Since the true scalar label y_i is a constant, its derivative vanishes ($= 0$). This leaves the negative derivative of the hypothesis function:

$$\frac{\partial L}{\partial w_j} = -2\big(y_i - h_{\mathbf{w}}(\mathbf{x}^{(i)})\big) \cdot \frac{\partial}{\partial w_j} h_{\mathbf{w}}(\mathbf{x}^{(i)})$$

Substituting our logistic hypothesis model $h_{\mathbf{w}}(\mathbf{x}^{(i)}) = \sigma(\mathbf{w}^T\mathbf{x}^{(i)})$, we apply the chain rule again using the outer derivative of the sigmoid function, which we defined as $\sigma'(z)$:

$$\frac{\partial L}{\partial w_j} = -2\big(y_i - h_{\mathbf{w}}(\mathbf{x}^{(i)})\big) \cdot \sigma'\big(\mathbf{w}^T\mathbf{x}^{(i)}\big) \cdot \frac{\partial}{\partial w_j}\big(\mathbf{w}^T\mathbf{x}^{(i)}\big)$$

We now evaluate the two remaining inner parts of this product using our previous definitions:

- The derivative of the sigmoid function satisfies $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, meaning $\sigma'\big(\mathbf{w}^T\mathbf{x}^{(i)}\big) = h_{\mathbf{w}}(\mathbf{x}^{(i)})\big(1 - h_{\mathbf{w}}(\mathbf{x}^{(i)})\big)$.
- The derivative of the vector linear combination with respect to the coordinate weight parameter is $\frac{\partial}{\partial w_j}(\mathbf{w}^T\mathbf{x}^{(i)}) = x_j^{(i)}$.

Plugging both analytical terms back into our product yields the complete scalar gradient component for a single training instance:

$$\frac{\partial L}{\partial w_j} = -2\big(y_i - h_{\mathbf{w}}(\mathbf{x}^{(i)})\big) \cdot h_{\mathbf{w}}(\mathbf{x}^{(i)})\big(1 - h_{\mathbf{w}}(\mathbf{x}^{(i)})\big) \cdot x_j^{(i)}$$

Def 8.22. By substituting this single coordinate derivative into the step update rule $w_j^{(t+1)} := w_j^{(t)} - \alpha \frac{\partial L}{\partial w_j}$, the double negatives cancel out. The final **logistic update rule** for a single parameter w_j using a single training example under squared error loss is defined as:

$$w_j^{(t+1)} := w_j^{(t)} + 2\alpha\big(y_i - h_{\mathbf{w}}(\mathbf{x}^{(i)})\big) \cdot h_{\mathbf{w}}(\mathbf{x}^{(i)})\big(1 - h_{\mathbf{w}}(\mathbf{x}^{(i)})\big) \cdot x_j^{(i)}$$

Note

We sometimes multiply the loss function by $\frac{1}{2}$ before deriving so we can get rid of the constant factor 2 in the final step adjustment:

$$w_j^{(t+1)} := w_j^{(t)} + \alpha\big(y_i - h_{\mathbf{w}}(\mathbf{x}^{(i)})\big) \cdot h_{\mathbf{w}}(\mathbf{x}^{(i)})\big(1 - h_{\mathbf{w}}(\mathbf{x}^{(i)})\big) \cdot x_j^{(i)}$$

9 Support Vector Machines

A Mathematical Vernacular and Induction

A.1 Mathematical Language

Def A.1. Natural Language: A natural language is any form of spoken or signed means of communication that has evolved naturally in humans through use and repetition without conscious planning or premeditation.

Def A.2. Domain of Discourse: The set of entities, objects, or concepts that a language, discussion, or formal system refers to. It specifies the scope within which terms, statements, and reasoning apply.

Def A.3. Jargon (Terminology): A jargon (or terminology) is a set of specialized words or phrases (called technical terms or just terms) relating to concepts from a particular domain of discourse.

Def A.4. Technical Language: A technical language is a natural language extended by a terminology and (possibly) special idioms, discourse markers, and notations.

Def A.5. Stylized Language: Mathematicians use a stylized language that uses formulae to represent mathematical objects, uses math idioms for special situations (e.g. "iff", "hence", "let... be..., then..."), and classifies statements by role (e.g., definition, Lemma, Theorem, Proof, Example).

Def A.6. Mathematical Vernacular: A technical language that you need to master to be successful when moving to a new environment (symbolic AI).

Def A.7. Formulae: A mathematical formula can be a **mathematical statement** (clause) which can be true or false (e.g. $x > 5, 3 + 5 = 7$), or a **mathematical object** (e.g. $3, n, x^2 + y^2 + z^2, \int_1^0 x^{3/2} dx$)

Observation: mathematical vernacular loves to name objects/statements for precise references. For example: Let $p = 3x^2 + 7x$, then $p^3 + 17p + 1$ is $\cdots$

Moreover, mathematical vernacular aggregates objects/statements for cognitive efficiency, examples:

- $a \in S, b \in S,$ and $c \in S \rightsquigarrow a, b, c \in S$ (object aggregation)
- $i \geq 0$ and $i \leq n \rightsquigarrow 0 \leq i \leq n$ (statement aggregation)

Def A.8. Sequence: mathematical vernacular uses the concept of sequences instead of lists. Sequences are usually finite, but can be infinite as well.

Def A.9. Ellipses: mathematical vernacular uses ellipses ($\cdots$) as a constructor for sequences. The meaning of an ellipsis is usually considered "obvious" and left for interpretation by the reader.

ellipses allow to write down large objects easily, examples:

- $1, \cdots, n \rightsquigarrow$ the sequence of natural numbers between 1 and n in order.
- $1, 4, 9, 16, \cdots \rightsquigarrow$ the sequence of squares in orders.
- $e_1, \cdots, e_n \rightsquigarrow$ a sequence of objects e_i for $1 \leq i \leq n$

Mathematical statements come in different epistemic varieties:

- **Definitions**: statements that introduce new global identifier for important objects
- **Assertions**: statements that state properties of mathematical objects
- **Examples**: statements that exhibit a witness for some property
- **Axioms**: statements that characterize the objects of a certain domain of discourse or theory. An axiom (postulate) is a statement about mathematical objects that we *assume* to be true.

Mathematical assertions are pragmatically classified into categories:

- **proofs** are arguments that justify the truth of statements beyond any doubt. Proofs are not really statements, but we sometimes treat them together.
- **proposition** is a statement which is interesting in its own right.
- **lemma** is an easily proved statement which is helpful for proving other propositions and theorems, but is usually not particularly interesting in its own right. A lemma is an intermediate theorem that serves as part of a proof of a larger theorem.

- **theorem** is a more important statement than a proposition which says something definitive on the subject, and often takes more effort to prove than a proposition or lemma. A theorem is a statement about mathematical object that we *know* to be true.
- **corollary** is a quick consequence of a proposition or theorem that was proven rececently. It is a theorem that follows directly from another theorem.
- **conjecture** is a statement that is thought to be provable, but has not been yet. A conjecture that is proven is a theorem

Mathematical statement use abbreviations, examples:

- $\wedge$ and $\vee$ for "and" and "or",
- $\neg$ for "not"
- $\forall x.P \quad (\forall x \in S.P)$ for "condition P holds for all x (in S)"
- $\exists x.P \quad (\exists x \in S.P)$ for "there exists an x (in S) such that the condition P holds"
- $\nexists x.P \quad (\nexists x \in S.P)$ for "there exists no x (in S) such that the condition P holds"
- $\exists^1 x.P (\exists^1 x \in S.P)$ for "there exists one and only x (in S) such that the condition P holds"
- "*iff*" for "if and only if", symbolized by $\Leftrightarrow$
- $\Rightarrow$ for "implies"; we can read $A \Rightarrow B$ as "if A then B"

Def A.10. Declaration: In mathematical vernacular we call a clause that introduce new identifiers together with some properties a declaration. For example Let $\epsilon, \delta > 0 \cdots$

Def A.11. Scope: The scope of an identifier is the part of an expression where the reference is valid; that is, where the identifier can be used to refer. In the other parts, the identifier may refer to a different entity, or to nothing at all (unbound)

Observations: definition have *global* scope, declarations have *local* scope.

Def A.12. Definiendum: The definiendum is the symbol, expression, or term being introduced by a definition. It is the newly named object or concept.

Def A.13. Definiens: The definiens is the symbol, expression, or construction that specifies the meaning of the defineiendum. It explains what the defineiendum denotes.

Mathematics has developed various forms of definition (definition schemata). A **simple definition** introduces a name (the defineiendum) for a compound object or concept (the definiens). The defineiendum must be new, i.e. may not have been used for anything else, in particular, the defineiendum may not occur in the definiens. We use the symbols $:=$ (and the inverse $=:$) to write simple definitions. For example, we can give the natural number 2 the name ϕ. In a formula we write this as $\phi := 2$ or $2 =: \phi$.

On the other hand, in a **pattern definition** the defineiendum is the operator or relation applied to n distinct variables -called pattern variables- $v_1, \cdots, v_n$ as arguments, and the definiens is an expression in these variables. When the new operator is applied to arguments $a_1, \cdots, a_n$, then its value is the definiens expression where the v_i are replaced by the a_i. We use the symbol $:=$ for operator definitions and $:\Leftrightarrow$ for relation definitions. For example, the following is a pattern definition for the set intersection operator $\cap$:

$$A \cap B := \{x \mid x \in A \wedge x \in B\}$$

The pattern variables are A and B, and with this definition we have e.g. $\Phi \cap \Phi = \{x \mid x \in \Phi \wedge x \in \Phi\}$.

An **implicit definition** (definition by description) is a clause or expression A, such that we can prove "there is exactly one n such that A" $(\exists^1 n.A)$, where n is a new name - the defineiendum. If such a unique existence proof exists, we call n *well-defined*. For example, $\forall x.x \notin \Phi$ is an implicit definition for the empty set Φ (instead of explicitly saying $\Phi := \{\}$). We can prove unique existence of Φ by just exhibiting $\{\}$ and showing that any other set S with $\forall x.x \notin S$ we have $S \equiv \Phi$. S cannot have elements, so it has the same elements as Φ, and thus $S \equiv \Phi$.

Here is another implicit definition: The exponential function is that function $f : \mathbb{R} \to \mathbb{R}$ with $f' = f$ and $f(0) = 1$. Here A is the clause "$f' = f$ and $f(0) = 1$". Well-definedness is mathematically non-trivial.

Mathematics uses examples and counterexamples to support understanding a property P. Examples give us a sense of the extent of P and counterexamples help delineate the border of P.

Def A.14. Example: An example E is a mathematical statement that consists of:

- symbol p for the **exemplandum** (plural: exemplanda): the property to be exemplified,
- the **exemplans** (plural: exemplantia): an expression A denoting a mathematical object that acts as witness object for the property p, and
- (optionally) a **justification** of E, i.e. a proof π that $p(A)$ holds in the current context.

Def A.15. Counter Example: An example for the complement of the property p where π is a proof that $p(A)$ does not hold.

Def A.16. Running Example: An example that is recalled time and again during an argument, applying the newly discovered knowledge to it, presumably to show how things work.

For instance, the following mathematical statement is a mathematical example:

The identity function on $\mathbb{R}$ is *continuous*.

- the exemplandum is "*continuous*",
- the exemplans A is "The identity function on $\mathbb{R}$", and
- the justification π, a proof of continuity $\mathrm{Id}_{\mathbb{R}}$: Let $\epsilon > 0$, then we choose $\delta := \epsilon \cdots$ is omitted.

We can use examples for $\forall$-statements as well. Let $B = \forall x.A$ be a mathematical statement, then we call an example/counter example for B whose exemplandum is the property "A holds for x" an example/counter example for B. For example, 3 is an example for the statement $B = \forall n.\mathrm{odd}(n)$.

> Note
>
> An example (or even several) for a mathematical statement B do not constitute a proof for B. On the other hand, one counter example for a mathematical statement B does show that $\neg B$ is true ($\exists x.\neg A \equiv \neg(\forall x.A)$).

Def A.17. Syllogism: A syllogism (proof method) is an argument that applies deductive reasoning to arrive at a conclusion based on two (or more) propositions that are asserted or assumed to be true. A **syllogistic system** is simply a set of syllogisms

> Note
>
> Today we usually reserve the term syllogism to informal arguments, formal arguments are called inference rules.

In a **proof by contradiction**, we make an assumption ($\neg A$) to the contrary of what we want to prove (A), and then we show that the assumption leads to a contradiction and therefore must be false. That lets us to conclude that ($\neg\neg A$) must be true, and thus (A).

If we want to prove a statement of the form *"If A, then B"*, then we do that by:

- assuming A and proving B from that.
- after this subproof, we may not use A anymore.

We call this proof method a **proof by local hypothesis**

If we know a general rule of the form *"If A then B"*, and we also know a specific instance A', which arises from A by replacing its variables with concrete values. Then by **chaining** we can conclude the corresponding instance B', obtained from B by the same variable replacement.

Example.

1. We know: "Socrates is a human." This is A'.
2. We also know a general rule:
$$\forall x \ (\ x \text{ is human } \Rightarrow\ x \text{ is mortal }).$$
This is the "If A then B" statement, where
$$A(x) = (x \text{ is human}), \quad B(x) = (x \text{ is mortal}).$$
3. Substitute $x =$ Socrates:
$$A(x) \to \text{Socrates is human}, \qquad B(x) \to \text{Socrates is mortal}.$$

4. Since “Socrates is human” is true, chaining gives us:

Socrates is mortal.

If we want to prove *”A for all x”*, then we prove A without regard for x, then argue something like *”as x was chosen arbitrarily when we proved A, we know A for every x”*

If we know that one of the cases $A_1, \cdots, A_k$ must always hold, then we use **proof by cases** to show that *”if A_1 then C”* and ”if A_2 then C” and so on $\forall i : 1 \leq i \leq k$, hence we can conclude that C holds. The following is an example:

Lemma A.1. For all rational numbers a and b, if $ab = 0$, then $a = 0$ or $b = 0$.

Proof.

1. Let $a, b \in \mathbb{Q}$ and assume $ab = 0$.
2. Obviously, either $a = 0$ or $a \neq 0$.
3. We prove that $a = 0$ or $b = 0$ by the two cases induced.
   4. Case $a = 0$:
      4.1 Then the conclusion of the lemma is trivially true, and there is nothing to prove.
   5. Case $a \neq 0$:
      5.1 We can multiply both sides of the equation $ab = 0$ by $\frac{1}{a}$ and obtain
      $$\tfrac{1}{a} \cdot ab = \tfrac{1}{a} \cdot 0.$$
      5.2 Reducing the fractions gives $b = 0$.
6. In both cases, either $a = 0$ or $b = 0$, so the statement holds. □

Def A.18. Without Loss of Generality (WLOG): An inference principle used in proofs when a statement $Q(x)$ must be shown for all cases of x, but the different cases are equivalent by symmetry or trivial reduction. Formally, if the domain of x can be partitioned into cases $A_1, \ldots, A_k$ and it can be shown that:

- proving $Q(x)$ under one representative case A_i suffices, because
- for every other case A_j there is either a trivial proof of $Q(x)$ or a direct reduction to the representative case,

then we may assume A_i holds “without loss of generality”. This allows us to simplify the argument by only proving the representative case.

For example, if we want to prove the property $Q(p)$ for all primes p. We know that every prime is either 2 (the only even prime) or an odd prime. If the case $p = 2$ is easy, and the general odd prime case is the only real work, then we can say:

> WLOG, assume p is odd

This does not exclude the even case; it means we have already checked that the even case adds no new difficulty.

In mathematical vernacular, different alphabets are used to convey precise meanings. Table 4 presents the Greek letters that you are expected to read, recognize, and write. In addition, table 5 shows the curly Roman and Fraktur letters, which are also standard in mathematical writing. Becoming fluent with these alphabets is an essential part of mathematical literacy.

α	A	alpha	β	B	beta	γ	Γ	gamma
δ	Δ	delta	ϵ	E	epsilon	ζ	Z	zeta
η	H	eta	θ, ϑ	Θ	theta	ι	I	iota
κ	K	kappa	λ	Λ	lambda	μ	M	mu
ν	N	nu	ξ	Ξ	xi	o	O	omicron
π, ϖ	Π	pi	ρ	P	rho	σ	Σ	sigma
τ	T	tau	υ	Υ	upsilon	φ, ϕ	Φ	phi
χ	X	chi	ψ	Ψ	psi	ω	Ω	omega

Table 4: Greek Alphabet

Roman	$\mathcal{A}, \mathcal{B}, \mathcal{C}, \mathcal{D}, \mathcal{E}, \mathcal{F}, \mathcal{G}, \mathcal{H}, \mathcal{I}, \mathcal{J}, \mathcal{K}, \mathcal{L}, \mathcal{M}, \mathcal{N}, \mathcal{O}, \mathcal{P}, \mathcal{Q}, \mathcal{R}, \mathcal{S}, \mathcal{T}, \mathcal{U}, \mathcal{V}, \mathcal{W}, \mathcal{X}, \mathcal{Y}, \mathcal{Z}$
Fraktur	$\mathfrak{A}, \mathfrak{B}, \mathfrak{C}, \mathfrak{D}, \mathfrak{E}, \mathfrak{F}, \mathfrak{G}, \mathfrak{H}, \mathfrak{I}, \mathfrak{J}, \mathfrak{K}, \mathfrak{L}, \mathfrak{M}, \mathfrak{N}, \mathfrak{O}, \mathfrak{P}, \mathfrak{Q}, \mathfrak{R}, \mathfrak{S}, \mathfrak{T}, \mathfrak{U}, \mathfrak{V}, \mathfrak{W}, \mathfrak{X}, \mathfrak{Y}, \mathfrak{Z}$

Table 5: Roman and Fraktur Letters

A.2 Unary Natural Numbers

Def A.19. Unary Natural Numbers: Terms generated by the following: $u ::= 0 \mid s(u)$ where 0 denotes zero and $s(u)$ denotes the successor of u. i.e. $\{0, s(0), s(s(0)), s(s(s(0))), \cdots\}$.

Def A.20. Piano Axioms: The following set of axioms are called the Piano axioms. They can be used to *reason* about natural numbers:

- **P1.** 0 is a unary natural number.
- **P2.** Every unary natural number has a successor that is unary natural number and that is different from it.
- **P3.** 0 is not a successor of any unary natural number.
- **P4.** Different unary natural numbers have different successors.
- **P5. Induction Axiom**. Every unary natural number possess a property P if
  - 0 has the property P, and
  - the successor of every unary natural number that has the property P also possesses property P

More formally, we call the set of unary natural numbers $\mathbb{N}_1$, and we say $P(n)$ if a unary natural number $n \in \mathbb{N}_1$ has a property P. Therefore, we express the axioms as follows:

- $0 \in \mathbb{N}_1$
- $\forall n \in \mathbb{N}_1 . s(n) \in \mathbb{N}_1 \wedge n \neq s(n)$
- $\neg(\exists n \in \mathbb{N}_1 . 0 = s(n))$
- $\forall n \in \mathbb{N}_1 . \forall m \in \mathbb{N}_1 . n \neq m \Rightarrow s(n) \neq s(m)$
- $\forall P . P(0) \wedge (\forall n \in \mathbb{N}_1 . P(n) \Rightarrow P(s(n))) \Rightarrow (\forall m \in \mathbb{N}_1 . P(m))$

Theorem A.2. Every unary natural number is either zero (0) or the successor of a unary natural number.

Proof. We make use of the induction axiom P5: We use the property P of "being zero or a successor" and prove the statement by convincing ourselves of the prerequisites of:

1. 0 is zero so 0 is "zero or a successor".
2. Let n be arbitrary unary natural number that is "zero or a successor".
3. Then its successor is a "successor", so the successor of n is also "zero or a successor".
4. Since we have taken n arbitrary we have shown that for any n, its successor has property P.
5. Property P holds for all unary natural number by P5, so we have proven the assertion.

□

Def A.21. Mathematical Induction: A method for proving a statement $P(n)$ is true for every natural number n. A proof by mathematical induction consists of two steps:

- The **base case**: Prove that $P(0)$ is true.
- The **induction step**: Prove that for every n, if $P(n)$ is true then $P(n+1)$ is also true.

The hypothesis in the induction step, that $P(n)$ is true for a particular n, is called the **induction hypothesis**

Def A.22. The addition "function": We define the addition operation $+$ procedurally:

- Adding 0 to a number does not change it, expressed as: $n + 0 = n$.
- Adding m to the successor of n yields the successor of $m + n$, expressed as: $m + s(n) = s(m + n)$.

Theorem A.3. For all unary natural number n, m, l, we have $(n + m) + l = n + (m + l)$.

Proof. We prove this by induction on l.

Let $P(l)$ be the property:

$$(n + m) + l = n + (m + l).$$

- base case:

$$(n + m) + 0 = (n + m) = n + (m + 0)$$

- step case: Assume $(n+m)+l = n+(m+l)$ for an arbitrary l. We must show

$$(n+m)+s(l) = n+(m+s(l)).$$

$$\begin{aligned}(n+m)+s(l) &= s\big((n+m)+l\big)\\ &\overset{\text{IH}}{=} s\big(n+(m+l)\big)\\ &= n+s(m+l)\\ &= n+(m+s(l)).\end{aligned}$$

  Hence the property holds for $s(l)$.

□

Def A.23. Multiplication: The **unary multiplication** operation can be defined by the equations:

- $n \cdot 0 = 0$, and
- $n \cdot s(m) = n + (n \cdot m)$

Def A.24. Exponentiation: The **unary exponentiation** operation can be defined by the equations:

- $exp(n, 0) = s(0)$, and
- $exp(n, s(m)) = n \cdot exp(n, m)$

Def A.25. Summation: The **unary summation** operation can be defined by the equations:

- $\bigoplus_{i=0}^{0}(n_i) = 0$, and
- $\bigoplus_{i=0}^{s(m)}(n_i) = n_{s(m)} \bigoplus \bigoplus_{i=0}^{m}(n_i)$

Example:

$$\begin{aligned}\bigoplus_{i=0}^{s(s(0))}(n_i) &= n_{s(s(0))} \bigoplus \bigoplus_{i=0}^{s(0)}(n_i)\\ &= n_{s(s(0))} \bigoplus n_{s(0)} \bigoplus \bigoplus_{i=0}^{0}(n_i)\\ &= n_{s(s(0))} \bigoplus n_{s(0)} \bigoplus 0\end{aligned}$$

Def A.26. Product: The **unary product** operation can be defined by the equations:

- $\bigodot_{i=0}^{0}(n_i) = s(0)$, and
- $\bigodot_{i=0}^{s(m)}(n_i) = n_{s(m)} \bigodot \bigodot_{i=0}^{m}(n_i)$

B Set Theory and Foundations of Mathematics

B.1 Sets

Def B.1. Set: A collection of different things (called **elements** or **members** of the set).

We can represent sets by listing the elements within curly brackets, e.g. $\{a, b, c\}$ or by describing the elements via a property, e.g. $\{x \mid x \mod 2 = 0\}$ (the set of even numbers). We can state element-hood ($a \in S$) or not ($b \notin S$)

> Note
>
> Learn to distinguish between objects and their representations. ($\{a, b, c\}$ and $\{b, a, a, c\}$ are different representations of the same set)

Def B.2. Set Equality: $A \equiv B :\equiv \forall x. x \in A \Leftrightarrow x \in B$

Def B.3. Subset: $A \subseteq B :\equiv \forall x. x \in A \Rightarrow x \in B$

Def B.4. Proper Subset: $A \subset B :\equiv (A \subseteq B) \wedge (A \not\equiv B)$

Def B.5. Super Set: $A \supseteq B :\equiv \forall x. x \in B \Rightarrow x \in A$

Def B.6. Proper Superset: $A \supset B :\equiv (A \supseteq B) \wedge (A \not\equiv B)$

Def B.7. Set Union: $A \cup B := \{x \mid x \in A \vee x \in B\}$

Def B.8. Set Intersection: $A \cap B := \{x \mid x \in A \wedge x \in B\}$

Def B.9. Disjoint: Two sets A, B are **disjoint** iff $A \cap B = \emptyset$

Def B.10. Set Difference: $A \setminus B := \{x \mid x \in A \wedge x \notin B\}$

Def B.11. Power Set: $\mathcal{P}(A) := \{S \mid S \subseteq A\}$ (the set of all subsets)

> Note
>
> Set operators such as union ($\cup$), intersection ($\cap$), and set difference ($\setminus$) always return a *set*.
> For example, consider
> $$\{a, b\} \cap \{b, c\} = \{b\}.$$
> The result $\{b\}$ is a set.
> Now observe:
> - $\{b\} \notin \{a, b\}$, since the elements of $\{a, b\}$ are a and b, not the set $\{b\}$.
> - $\{b\} \notin \{b, c\}$, since the elements of $\{b, c\}$ are b and c, not the set $\{b\}$.
> - $\{b\} \subseteq \{a, b\}$, because every element of $\{b\}$ (namely b) is also in $\{a, b\}$.
> - $\{b\} \subseteq \{b, c\}$, for the same reason.
>
> Thus, the outcome of a set operation should always be compared to other sets using $\subseteq$ (subset relation), not $\in$ (element relation).

Def B.12. Cartesian Product: $A \times B := \{(a, b) \mid a \in A \wedge b \in B\}$, (a, b) is a *pair*

Def B.13. Family: A set of sets

Def B.14. Topology: A family of open sets

Def B.15. Indexing Function: Let I and X be sets. A function $f : I \to X$ is called an indexing function. The set I is called the index set. For each $i \in I$, we define $x_i := f(i)$, here i is called the index (or parameter) of x_i.

Def B.16. Indexed Family: Given an indexing function $f : I \to X$, the set of values $f(I) = \{f(i) : i \in I\}$ is called an indexed family. It is often written as $(x_i)_{i \in I}$, $\langle x_i \rangle_{i \in I}$, or $\{x_i\}_{i \in I}$.

Example: Consider index set $I := \{1, 2, 3\}$. Consider indexing function f defined as:

$$f(1) = \{a, b\} \quad f(2) = \{c\} \quad f(3) = \{d, f\}$$

the set of values $f(I) = \{f(1), f(2), f(3)\}$ is the indexed family:

$$(X_i)_{i \in I} = f(I) = \{\{a, b\}, \{c\}, \{d, f\}\}$$

where $x_i = f(i)$, i.e. $x_1 = f(1) = \{a, b\}, \cdots$

Def B.17. Union over a Family: Let $(X_i)_{i \in I}$ be an indexed family, then $\bigcup_{i \in I} X_i := \{x \mid \exists i \in I . x \in X_i\}$

Def B.18. Intersection over a Family: Let $(X_i)_{i \in I}$ be an indexed family, then $\bigcap_{i \in I} X_i := \{x \mid \forall i \in I . x \in X_i\}$

Def B.19. n-fold Cartesian Product: $\prod_{i=1}^{n} X_i := X_1 \times \cdots \times X_n := \{\langle x_1, \cdots, x_n \rangle \mid \forall i . 1 \leq i \leq n \Rightarrow x_i \in X_i\}$ where $\langle x_1, \cdots, x_n \rangle$ is called an n-tuple.

Example: consider the indexed family $(X_i)_{i \in \{1,2,3\}} = \{\{a, b\}, \{c\}, \{d, f\}\}$. $X_1 \times X_2 \times X_3 = \{\langle a, c, d \rangle, \langle a, c, f \rangle, \langle b, c, d \rangle, \langle b, c, f \rangle\}$.

Def B.20. n-dim Cartesian Space: $X^n := \{\langle x_1, \cdots, x_n \rangle \mid 1 \leq i \leq n \Rightarrow x_i \in X\}$ where $\langle x_1, \cdots, x_n \rangle$ is called a vector.

Example: Let $X = \{1, 2\}$, then $X^2 = \{\langle 1, 1 \rangle, \langle 1, 2 \rangle, \langle 2, 1 \rangle, \langle 2, 2 \rangle\}$. Another example is the usual 2D plane: $\mathbb{R}^2 := \{\langle x, y \rangle \mid x \in \mathbb{R}, y \in \mathbb{R}\}$

Def B.21. Size of a Set: The size $\Gamma(A)$ of a set A is the number of elements in A.

Note

- $\Gamma(A \cup B) \leq \Gamma(A) + \Gamma(B)$
- $\Gamma(A \cap B) \leq \min(\Gamma(A), \Gamma(B))$
- $\Gamma(A \times B) = \Gamma(A) \cdot \Gamma(B)$

Def B.22. Pairwise Disjoint: A family of sets is called **pairwise disjoint** or **mutually disjoint**, if any two of those sets are disjoint.

Def B.23. Multiset: A **multiset** (or **bag**) is a generalization of a set where elements can appear more than once. Formally, a multiset $\mathcal{M}$ over a domain D is a pair (D, m), where $m : D \to \mathbb{N}$ is a function mapping each element $x \in D$ to its **multiplicity** (the number of occurrences).

Def B.24. Multiset Membership: $x \in \mathcal{M} :\equiv m(x) > 0$

Def B.25. Multiset Size: The size of a multiset $\mathcal{M} = (D, m)$, denoted $\Gamma(\mathcal{M})$, is the sum of the multiplicities of all its elements: $\Gamma(\mathcal{M}) = \sum_{x \in D} m(x)$.

Def B.26. Multiset Sum (Union): The sum of two multisets $\mathcal{M}_1 = (D, m_1)$ and $\mathcal{M}_2 = (D, m_2)$, denoted $\mathcal{M}_1 \uplus \mathcal{M}_2$, is a multiset where the multiplicity of each element is the sum of its multiplicities: $m_{\uplus}(x) = m_1(x) + m_2(x)$.

Def B.27. Multiset Intersection: The intersection of two multisets $\mathcal{M}_1 = (D, m_1)$ and $\mathcal{M}_2 = (D, m_2)$, denoted $\mathcal{M}_1 \cap \mathcal{M}_2$, is a multiset where the multiplicity of each element is the minimum of its multiplicities in the two multisets: $m_{\cap}(x) = \min(m_1(x), m_2(x))$.

Def B.28. Multiset Equality: $\mathcal{M}_1 \equiv \mathcal{M}_2 :\equiv \forall x \in D . m_1(x) = m_2(x)$

B.2 Relations

Def B.29. Relation: $R \subseteq A \times B$ is a (binary) relation between A and B

Def B.30. Relation on: a relation $R \subseteq A \times B$ where $A = B$ is called a *relation on* A

Def B.31. Total: A relation $R \subseteq A \times B$ is called *total* (*left total*) iff $\forall x \in A. \exists y \in B. (x, y) \in R$

Def B.32. Converse Relation: $R^{-1} \subseteq B \times A := \{(y, x) \mid (x, y) \in R\}$ is the converse relation of R

Def B.33. Relation Composition: The composition of $R \subseteq A \times B$ and $S \subseteq B \times C$ is defined as $R \circ S := \{(a, c) \in A \times C \mid \exists b \in B. (a, b) \in R \wedge (b, c) \in S\}$

A relation on A: $R \subseteq A \times A$ is called:

- **reflexive** on A, iff $\forall a \in A. (a, a) \in R$
- **irreflexive** on A, iff $\forall a \in A. (a, a) \notin R$
- **symmetric** on A, iff $\forall a, b \in A. (a, b) \in R \Rightarrow (b, a) \in R$
- **asymmetric** on A, iff $\forall a, b \in A. (a, b) \in R \Rightarrow (b, a) \notin R$
- **antisymmetric** on A, iff $\forall a, b \in A. (a, b) \in R \wedge (b, a) \in R \Rightarrow a = b$
- **transitive** on A, iff $\forall a, b, c \in A. (a, b) \in R \wedge (b, c) \in R \Rightarrow (a, c) \in R$
- **equivalence relation** on A, iff R is reflexive symmetric, and transitive

Def B.34. Equality Relation: The equality relation is an equivalence relation on any set.

Def B.35. Divides Relation: The *divides* relation on the integers is defined as $| \subseteq \mathbb{Z} \times \mathbb{Z}$, where $| = \{(a, b) \in \mathbb{Z} \times \mathbb{Z} \mid \exists k \in \mathbb{Z} : b = a \cdot k\}$. We write $a \mid b$ to denote $(a, b) \in |$ (read as *"a divides b"*).

For example: $5 \mid 10$ because $\exists k : 10 = 5 \cdot k$ where $k = 2$.

Def B.36. Congruence Modulo: For a fixed $n \in \mathbb{N}$ with $n \geq 1$, the *congruence modulo n* relation on the integers is defined as

$$\equiv_n \subseteq \mathbb{Z} \times \mathbb{Z}, \qquad \equiv_n = \{(a, b) \in \mathbb{Z} \times \mathbb{Z} \mid n \mid (a - b)\}.$$

We write $a \equiv b \pmod{n}$ to denote $(a, b) \in \equiv_n$ (read as *"a is congruent to b modulo n"*).

Examples:

- $12 \equiv 2 \pmod{5}$ because $5 \mid 10$
- $7 \equiv 22 \pmod{5}$ because $5 \mid 15$
- $9 \equiv 9 \pmod{5}$ because $5 \mid 0$

> **Note**
>
> In the definition of congruence modulo n, we do *not* take the absolute value of the difference:
>
> $$a \equiv b \pmod{n} \iff n \mid (a - b).$$
>
> Since the divisor n can be multiplied by a negative integer, the sign of $(a - b)$ does not matter. For example, $2 \equiv 12 \pmod{5}$ because $2 - 12 = -10$ and $-10 = 5 \cdot (-2)$, so $5 \mid (2 - 12)$.

Exercise: Which of the following binary relations on integers are equivalence relations?

1. Φ. It is not reflexive because we must have $(a, a) \in R$ for all integers a. But Φ has no pairs at all. $\rightarrow$ NO.
2. $\mathbb{Z} \times \mathbb{Z}$
   - It is reflexive since $(a, a) \in R \quad \forall a$
   - It is symmetric because if $(a, b) \in R$ then (b, a) also is
   - It is transitive because every possible pair is in it anyway
   - $\rightarrow$ YES.
3. $m \leq n$
   - It is reflexive since $m \leq m$

- It is not symmetric, e.g. $2 \leq 3$ but not $3 \leq 2$
- $\rightarrow$ NO

4. $\exists z \in \mathbb{Z} : m + z = n$
   - This is actually the same as $\mathbb{Z} \times \mathbb{Z}$ because for any integers m, n, we can always pick $z = n - m$. $\rightarrow$ YES.
5. $\exists z \in \mathbb{Z} : m \cdot z = n$
   - This is the same as $m \mid n$. It is reflexive ($m \cdot 1 = 1$). However, it is not symmetric ($2 \mid 4$ but $4 \nmid 2$). $\rightarrow$ NO.
6. $5 \mid (m - 2)$
   - This means m is congruent to 2 modulo 5.
   - It is reflexive ($5 \mid 0$)
   - It is symmetric (negative or positive does not matter)
   - It is transitive, if $5 \mid m - n$ and $5 \mid n - k$, then $5 \mid (m - n) + (n - k) = m - k$
   - $\rightarrow$ YES.
7. $m \mid n$. It is reflexive but not symmetric. $\rightarrow$ NO.

Def B.37. Reflexive Closure: The **reflexive closure** of a relation R on a set A is the relation $R \cup \Delta$ where $\Delta = \{\langle a, a\rangle \mid a \in A\}$ is the identity relation. It is defined as:

$$R_r = R \cup \{\langle a, a\rangle \mid a \in A\}$$

Note

Alternative notation: $R_r = R \cup R^0$. Since R^n is the set of pairs that are connected by a chain of exactly n hops "reachable in n steps", we can think of R^0 as pairs that are connected by a chain of exactly 0 hops "reachable in 0 steps", i.e. the identity relation.

For example, let $A = \{1, 2, 3\}, R = \{\langle 1, 2\rangle, \langle 2, 3\rangle\}$, then $R_r = \{\langle 1, 2\rangle, \langle 2, 3\rangle, \langle 1, 1\rangle, \langle 2, 2\rangle, \langle 3, 3\rangle\}$

Def B.38. Symmetric Closure: The **symmetric closure** of a relation R on a set A is the union of R and its inverse relation R^{-1}. It is defined as:

$$R_s = R \cup R^{-1} = R \cup \{\langle b, a\rangle \mid \langle a, b\rangle \in R\}$$

e.g. $R_s = \{\langle 1, 2\rangle, \langle 2, 3\rangle, \langle 2, 1\rangle, \langle 3, 2\rangle\}$

Def B.39. Transitive Closure: The **transitive closure** R^+ of a relation R on a set A is the **smallest** transitive relation containing R. It is defined as:

$$R^+ = \bigcup_{n=1}^{\infty} R^n$$

For example, let $A = \{1, 2, 3, 4\}, R = \{\langle 1, 2\rangle, \langle 2, 3\rangle, \langle 3, 4\rangle\}$

- $R^1 = R$
- $R^2 = R \circ R = \{\langle 1, 3\rangle, \langle 2, 4\rangle\}$
- $R^3 = R \circ R \circ R = \{1, 4\}$
- No further compositions yields new pairs.

$\rightsquigarrow R^+ = R \cup R^2 \cup R^3 = \{\langle 1, 2\rangle, \langle 2, 3\rangle, \langle 3, 4\rangle, \langle 1, 3\rangle, \langle 2, 4\rangle, \{1, 4\}\}$

Def B.40. Reflexive Transitive Closure: The **reflexive transitive closure** (A.K.A **??**) of a relation R on a set A is the **smallest** relation that is both reflexive and transitive. It is defined as:

$$R^* = R^0 \cup R^+ = \bigcup_{n=0}^{\infty} R^n$$

Example: let $A = \{1, 2, 3\}, R = \{\langle 1, 2\rangle, \langle 2, 3\rangle\}$.

- R^0 is the identity relation on $A = \{\langle 1,1\rangle, \langle 2,2\rangle, \langle 3,3\rangle\}$
- $R^1 = R$
- $R^2 = \{\langle 1,3\rangle\}$
- No new pairs can be introduced

$\rightsquigarrow R^* = \{\langle 1,1\rangle, \langle 2,2\rangle, \langle 3,3\rangle\} \cup \{\langle 1,2\rangle, \langle 2,3\rangle\} \cup \{\langle 1,3\rangle\} = \{\langle 1,1\rangle, \langle 2,2\rangle, \langle 3,3\rangle, \langle 1,2\rangle, \langle 2,3\rangle, \langle 1,3\rangle\}$

Def B.41. Parital Ordering (non-strict): A relation $R \subseteq A \times A$ is called a **partial ordering** on A, iff R is reflexive, antisymmetric, and transitive on A.

Def B.42. Strict Partial Ordering: A relation $R \subseteq A \times A$ is called a **strict partial ordering** on A, iff R is irreflexive, and transitive on A.

Note

We usually write strict partial ordering with asymmetric symbols like $\prec$ and non-strict ones by adding a line that reminds us of equality ($\preceq$).

Def B.43. Comparable: In a non-strict partial ordering, two elements a, b are comparable if either $a \preceq b$ or $b \preceq a$. If neither holds, they are incomparable.

Def B.44. Linear Order: A partial ordering is called linear (or total) on A, iff all elements in A are comparable, i.e. if $(x,y) \in R$ or $(y,x) \in R$ for all $x, y \in A$.

For example, the $\leq$ relation is a linear order on $\mathbb{N}$. However, the "divides" ($|$) relation is a non-strict partial ordering on $\mathbb{N}$ that is not linear (e.g., neither $2 \mid 3$ nor $3 \mid 2$).

Lemma B.1. Strict partial orderings are asymmetric.

Proof. By contradiction: if $(a,b) \in R$ and $(b,a) \in R$, then $(a,a) \in R$ by transitivity, and that can't be because its irreflexive. □

Lemma B.2. If $\preceq$ is a non-strict partial order, then $\prec := \{(a,b) \mid a \preceq b \wedge a \neq b\}$ is a strict partial order. Conversely, if $\prec$ is a strict partial order, then $\preceq := \{(a,b) \mid a \prec b \vee a = b\}$ is a non-strict partial order.

Proof. **(1) Non-strict → Strict:** Assume $\preceq$ is reflexive, antisymmetric, and transitive. Define $a \prec b \iff (a \preceq b \wedge a \neq b)$.

- *Irreflexive:* $a \prec a$ would require $a \neq a$, impossible.
- *Transitive:* If $a \prec b$ and $b \prec c$, then $a \preceq b$, $b \preceq c$, and $a \neq b$, $b \neq c$. By transitivity of $\preceq$, $a \preceq c$. If $a = c$, antisymmetry would force $a = b$, contradiction. So $a \neq c$, hence $a \prec c$.

Thus $\prec$ is a strict partial order.

(2) Strict → Non-strict: Assume $\prec$ is irreflexive and transitive. Define $a \preceq b \iff (a = b \vee a \prec b)$.

- *Reflexive:* $a = a$ gives $a \preceq a$.
- *Antisymmetric:* If $a \preceq b$ and $b \preceq a$, then either $a = b$ or we would have $a \prec b$ and $b \prec a$, contradicting asymmetry.
- *Transitive:* If $a \preceq b$ and $b \preceq c$, then combining cases ($=$ or $\prec$) always gives $a \preceq c$.

Thus $\preceq$ is a non-strict partial order. □

Examples:

- On $\mathbb{N}$, the non-strict partial order $\leq$ induces the strict order $<$ by removing equality. Conversely, $<$ induces $\leq$ by adding equality.
- On sets, the non-strict partial order $\subseteq$ induces the strict order $\subset$. Conversely, $\subset$ induces $\subseteq$ by adding equality.

Exercise: which of the following binary relation on integers are partial ordering:

1. Φ. It is not reflexive because we must have $(a,a) \in R$ for all integers a. But Φ has no pairs at all. → NO.
2. $\mathbb{Z} \times \mathbb{Z}$
   - It is reflexive since $(a,a) \in R \quad \forall a$
   - It is symmetric because if $(a,b) \in R$ then (b,a) also is, → it is not antisymmetric, → NO
3. $m \leq n$

- It is reflexive since $m \leq m$
- It is asymmetric, e.g. $2 \leq 3$ but not $3 \leq 2$
- It is transitive
- $\rightarrow$ YES

4. $\exists z \in \mathbb{Z} : m + z = n$
   - This is actually the same as $\mathbb{Z} \times \mathbb{Z}$ because for any integers m, n, we can always pick $z = n - m$. $\rightarrow$ NO.
5. $\exists z \in \mathbb{Z} : m \cdot z = n$
   - This is the same as $m \mid n$. It is reflexive ($m \cdot 1 = 1$). It is asymmetric on $\mathbb{N}$ but not on $\mathbb{Z}$ ($2 \mid -2$ and $-2 \mid 2$ but $2 \neq -2$). $\rightarrow$ NO.
6. $5 \mid (m - 2)$
   - This means m is congruent to n modulo 5.
   - It is reflexive ($5 \mid 0$)
   - It is symmetric (not asymmetric)
   - It is transitive, if $5 \mid m - n$ and $5 \mid n - k$, then $5 \mid (m - n) + (n - k) = m - k$
   - $\rightarrow$ NO.
7. $m \mid n$. It is reflexive but not asymmetric. $\rightarrow$ NO.

Def B.45. Partially Ordered Set (Poset): A **Partially Ordered Set** (or **poset**) is a pair $\langle G, \preceq \rangle$ where G is a set and $\preceq$ is a non-strict partial ordering on G.

Def B.46. Least Element: Let $\langle G, \preceq \rangle$ be a poset and $T \subseteq G$. An element $t \in T$ is the **least** (or **smallest**, **minimal**, or $\preceq$**-minimal**) element of T iff $t \preceq t'$ for all $t' \in T$.

Def B.47. Greatest Element: Let $\langle G, \preceq \rangle$ be a poset and $T \subseteq G$. An element $t \in T$ is the **greatest** (or **largest**, **maximal**, or $\preceq$**-maximal**) element of T iff $t' \preceq t$ for all $t' \in T$.

Def B.48. Supremum and Infimum: Let $\langle G, \preceq \rangle$ be a non-strict partial ordering and $T \subseteq G$, then we call the **least upper bound** ($\sup(T) \in G$) of T the **supremum** of T and the **greatest lower bound** ($\inf(T) \in G$) of T the **infimum** of T (if they exist).

Example: Consider the non-strict partial ordering $\langle \mathbb{R}, \preceq \rangle$. Let T be the open interval $(0, 1) \subseteq \mathbb{R}$, then $\sup(T) = 1$, and $\inf(T) = 0$. Note that $0, 1 \in \mathbb{R}$ but $\notin T$. (its different from `min` and `max` of T).

B.3 Functions

Def B.49. Partial function: A relation $R \subseteq X \times Y$ is a *partial function* from X to Y ($f : X \rightharpoonup Y$) iff $\forall x \in X, \forall y_1, y_2 \in Y.\big((x, y_1) \in R \wedge (x, y_2) \in R \Rightarrow y_1 = y_2\big)$ (i.e. there is at most one $y \in Y$ with $(x, y) \in f$)

Def B.50. dom, codom: We call X the domain ($\text{dom}(f)$), and Y the codomain ($\text{codom}(f)$)

Def B.51. Function Application: Instead of writing $(x, y) \in f$ we write $f(x) = y$

Def B.52. Undefined: we call a partial function $f : X \rightharpoonup Y$ undefined at $x \in X$, iff $\forall y \in Y.(x, y) \notin f$ ($f(x) = \perp$).

Def B.53. Function: A partial function $f : X \rightharpoonup Y$ is called a *function* (or *total function*) from X to Y iff it is a total relation. $(\forall x \in X.\exists^1 y \in Y : (x, y) \in f)$

> Note
>
> A partial function guarantees uniqueness, but not existence. So each x has at most one y (some might have none). A total function guarantees both uniqueness and existence (at most and at least $\rightarrow$ exactly one)

Def B.54. Partial Function Space: Given two sets X and Y, the set of all possible partial functions from X to Y is called the partial function space, denoted as $X \rightharpoonup Y$

> Note
>
> If X and Y are finite, then for each element $x \in X$ we have $\Gamma(Y) + 1$ choices (either pick one of the $\Gamma(Y)$ values in Y, or leave it undefined). Therefore, $\Gamma(X \rightharpoonup Y) = (\Gamma(Y) + 1)^{\Gamma(X)}$.

Example, $X = Y = \{0, 1\}$

$$X \rightharpoonup Y = \{\Phi, \{(0,0)\}, \{(0,1)\}, \{(1,0)\}, \{(1,1)\}, \{(0,0),(1,0)\}, \{(0,1),(1,1)\}, \{(0,1),(1,0)\}, \{(0,0),(1,1)\}\}$$

Notice that $\Gamma(X \rightharpoonup Y) = 9 = (\Gamma(Y) + 1)^{\Gamma(X)} = (2 + 1)^2$

Def B.55. Function Space: Given two sets X and Y, the set of all possible functions from X to Y is called the partial function space, denoted as $X \rightarrow Y$

> Note
>
> If X and Y are finite, then for each element $x \in X$ we must pick exactly one of the $\Gamma(Y)$ values in Y. Therefore, $\Gamma(X \rightarrow Y) = (\Gamma(Y))^{\Gamma(X)}$.

Example, $X = Y = \{0, 1\}$

$$X \rightarrow Y = \{\{(0,0),(1,0)\}, \{(0,1),(1,1)\}, \{(0,1),(1,0)\}, \{(0,0),(1,1)\}\}$$

Notice that $\Gamma(X \rightarrow Y) = 4 = (\Gamma(Y))^{\Gamma(X)} = (2)^2$

A function $f : X \rightarrow Y$ is called

- **injective** iff $\forall x_1, x_2 \in X.f(x_1) = f(x_2) \Rightarrow x_1 = x_2$
- **surjective** iff $\forall y \in Y.\exists x \in X.f(x) = y$
- **bijective** iff f is injective and surjective

Let's look for some examples to make the previous definitions clear.

Assume $X = \{1, 2, 3\}$ $Y = \{a, b\}$ The relation $R = \{(1, a), (1, b)\}$ is just a relation, it is not a partial function nor a total function.

If $R = \{(1, a)\}$ it becomes a partial function but not a total one.

If $R = \{(1, a), (2, a), (3, a)\}$ it becomes a total function because each $x \in X$ has exactly one $y \in Y$. However, it is not injective ($1, 2, 3$ all map to the same value) and not surjective (b is not reached).

IF $R = \{(1, a), (2, a), (3, b)\}$ then this is a surjective function but not an injective one (since $1, 2$ map to the same value).

Assume $X = \{1, 2\}$ $Y = \{a, b, c\}$ $f = \{(1, a), (2, b)\}$ then this is an injective function (different inputs map to different output) but not surjective since there is no x such that $f(x) = c$.

And finally, assume $X = \{1,2\}$ $Y = \{a,b\}$ $f = \{(1,a),(2,b)\}$ then this is a total function that is both injective and surjective, i.e. a bijective function.

Def B.56. Function Composition: If $f : A \to B$ and $g : B \to C$ are functions, then we call $g \circ f : A \to C; x \longmapsto g(f(x))$ the composition of g and f (read g after f).

> Note
>
> Given $f, g : \mathbb{N} \to \mathbb{N}$, if f and g both are injective/bijective, so is $g \circ f$

Observations:

- If f is injective, then the converse relation f^{-1} is a partial function.

  For example: $X = \{1,2,3\}, Y = \{a,b,c,d\}, f : X \to Y = \{(1,a),(2,b),(3,c)\}$. f is clearly injective since different inputs map to different outputs. $\text{dom}(f^{-1}) = \text{codom}(f)$ and vice versa. $f^{-1} = \{(a,1),(b,2),(c,3)\}$. It is a partial function because for $d \in \text{dom}(f^{-1})$ there is no $f^{-1}(d)$

- If f is surjective, then the converse relation f^{-1} is a total relation.

  For example: $X = \{1,2,3\}, Y = \{a,b\}, f : X \to Y = \{(1,a),(2,b),(3,b)\}$. f is clearly surjective because both a, b are hit by some x. $f^{-1} = \{(a,1),(b,2),(b,3)\}$. It is a total function.

- IF f is bijective, call the converse relation **inverse function**, we also write it as f^{-1}.

- If f is bijective, then the inverse function f^{-1} is a total function.

- If $f : X \to Y$ is bijective, then $f \circ f^{-1} = \text{Id}_Y$.

  For example: $X = \{1,2,3\}, Y = \{a,b,c\}, f = \{(1,a),(2,b),(3,c)\}, f^{-1} = \{(a,1),(b,2),(c,3)\}$. $(f \circ f^{-1})(y) = f(f^{-1}(y))$

  - $y = a : f^{-1}(a) = 1, f(1) = a$
  - $y = b : f^{-1}(b) = 2, f(2) = b$
  - $y = c : f^{-1}(c) = 3, f(3) = c$

  $\to \quad f \circ f^{-1} = \text{Id}_Y$

Def B.57. Image and Preimage: Let $f : X \to Y$ be a function, $X' \subseteq X$ and $Y' \subseteq Y$, then we call

- $f(X') := \{y \in Y \mid \exists x \in X'.(x,y) \in f\}$ the **image** of X' under f,
- $\text{Im}(f) := f(X)$ the **image** of f, and
- $f^{-1}(Y') := \{x \in X \mid \exists y \in Y'.(x,y) \in f\}$ the **preimage** of Y' under f.

> Note
>
> Let $f : \mathbb{N} \to \mathbb{N}$ be injective and let its partial inverse be
>
> $$f^{-1} : \mathbb{N} \rightharpoonup \mathbb{N}, \qquad \text{dom}(f^{-1}) = \text{im}(f), \quad f^{-1}(y) = x \text{ iff } f(x) = y.$$
>
> Then $f^{-1} \circ f = \text{Id}_{\mathbb{N}}$ as a total function $\mathbb{N} \to \mathbb{N}$.
> *Proof.* For any $x \in \mathbb{N}$ we have $f(x) \in \text{im}(f) = \text{dom}(f^{-1})$, hence $(f^{-1} \circ f)(x) = f^{-1}(f(x)) = x$. □

For example: Define $f : \mathbb{N} \to \mathbb{N}, \quad f(x) = 2x$. Then f is injective with image $\text{im}(f) = \{0,2,4,6,\dots\}$. Its partial inverse is

$$f^{-1} : \mathbb{N} \rightharpoonup \mathbb{N}, \quad f^{-1}(y) = \tfrac{y}{2} \text{ if } y \text{ is even}, \quad \text{undefined if } y \text{ is odd}.$$

For all $x \in \mathbb{N}$ we have

$$(f^{-1} \circ f)(x) = f^{-1}(f(x)) = f^{-1}(2x) = \tfrac{2x}{2} = x,$$

hence $f^{-1} \circ f = \text{Id}_{\mathbb{N}}$. By contrast, $f \circ f^{-1}$ is only the identity on the even numbers.

Another example: Define $f : \mathbb{N} \to \mathbb{N}, \quad f(x) = x + 1$. Then f is injective with image $\text{im}(f) = \{1,2,3,\dots\}$. Its partial inverse is

$$f^{-1} : \mathbb{N} \rightharpoonup \mathbb{N}, \quad f^{-1}(y) = y - 1 \text{ if } y \geq 1, \quad \text{undefined if } y = 0.$$

For all $x \in \mathbb{N}$ we have

$$(f^{-1} \circ f)(x) = f^{-1}(f(x)) = f^{-1}(x+1) = (x+1) - 1 = x,$$

hence again $f^{-1} \circ f = \mathrm{Id}_{\mathbb{N}}$. Here $f \circ f^{-1}$ is only the identity on $\{1, 2, 3, \dots\}$.

Def B.58. Isomorphic: We call two sets A and B isomorphic iff there is a bijection $f : A \to B$.

Def B.59. Cardinality: We say that a set A is **finite** and has **cardinality** $\Gamma(A) \in \mathbb{N}$, iff there is a bijective function $f : A \to \{n \in \mathbb{N} \mid n < \Gamma(A)\}$.

Def B.60. Countably Infinite: We say that a set A is countably infinite, iff there is a bijective function $f : A \to \mathbb{N}$. The cardinality of such set is denoted by $\aleph_0$ Examples: $\mathbb{N}, \mathbb{Z}, \cdots$

Def B.61. Uncountably Infinite: We say that a set A is **uncountably infinite**, iff A is infinite and there is **no** bijective function $f : A \to \mathbb{N}$. The cardinality of the such sets is denoted by $\mathfrak{c}$ (for continuum). Examples: $\mathbb{R}, [0, 1], \mathbb{C}, \mathcal{P}(\mathbb{N})$

Def B.62. Countable: A set A is called countable, iff it is finite or countably infinite

> Note
>
> The set of real numbers $\mathbb{R}$ is not countable (there is no bijection) $\mathbb{R} \to \mathbb{N}$

Def B.63. Restriction: Let $f : A \to B$ and $C \subseteq A$, then we call the function $f \restriction_C := \{(c, b) \in f \mid c \in C\}$ the restriction of f to C.

Def B.64. Lambda-Notation: Let X be a set and E an expression depending on a variable $x \in X$. The expression $\lambda x \in X.\, E$ denotes the (partial) function f from X to Y with $f(x) = E$ for all $x \in X$.

Examples:

- the identity function is written in lambda notation: $\lambda n \in \mathbb{N}.n = \{(n, n) \mid n \in \mathbb{N}\} = \mathrm{Id}_{\mathbb{N}} = \{(0, 0), (1, 1), (2, 2), \cdots\}$
- the square function: $\lambda x \in \mathbb{N}.x^2 = \{(x, x^2) \mid x \in \mathbb{N}\} = \{(0, 0), (1, 1), (2, 4), (3, 9), \cdots\}$
- the "add" operation: $\lambda(x, y) \in \mathbb{N} \times \mathbb{N}.x + y = \{((0, 0), 0), ((1, 0), 1), \cdots\}$

Def B.65. Curried Function Type: Let X, Y, Z be sets. An *uncurried* function is written as

$$f : X \times Y \to Z, \quad f(x, y) = E(x, y).$$

The same function can equivalently be written in *curried* form as

$$f : X \to Y \to Z, \quad f(x)(y) = E(x, y).$$

Thus in the curried notation the function takes one argument at a time: for $x \in X$, the value $f(x)$ is itself a function $Y \to Z$, and applying it to $y \in Y$ yields $f(x)(y) \in Z$.

Example. The addition function can be written in uncurried or curried form:

$$\mathrm{add} : \mathbb{N} \times \mathbb{N} \to \mathbb{N}, \quad \mathrm{add}(m, n) = m + n,$$

equivalently: $\mathrm{add} : \mathbb{N} \to \mathbb{N} \to \mathbb{N}, \quad \mathrm{add}(m)(n) = m + n.$

In the curried form, for fixed $m \in \mathbb{N}$, the expression $\mathrm{add}(m) : \mathbb{N} \to \mathbb{N}$ is the function $n \mapsto m + n$, i.e. the function that adds m to its argument.

Another Example (Trinary)

$$h : \mathbb{N} \times \mathbb{N} \times \mathbb{N} \to \mathbb{N}, \quad h(m, n, o) = m + n + o.$$

Equivalently, in curried form we can write

$$h : \mathbb{N} \to \mathbb{N} \to \mathbb{N} \to \mathbb{N}, \quad h(m)(n)(o) = m + n + o.$$

In this form, $h(m)$ is a function

$$h(m) : \mathbb{N} \to \mathbb{N} \to \mathbb{N}, \quad h(m)(n)(o) = m + n + o,$$

which for fixed m returns the function "add m to the sum of two further arguments."

Def B.66. Well-Defined Function: Let X, Y be sets. A function declaration $f : X \to Y$ together with a defining expression $E(x)$ is called *well-defined* iff for every $x \in X$:

- the expression $E(x)$ is meaningful (all variables are bound and the operations are valid), and
- the result $E(x)$ lies in Y.

Equivalently: a function is well-defined if its type annotation $f : X \to Y$ is consistent with the expression that defines it.

Checking well-definedness in practice:

The general definition above says that a function declaration must be consistent with its defining expression. Typical ways in which this can fail are:

- **Type mismatch:** the declared type $X \to Y$ does not agree with the actual input/output structure of the definition.
- **Free variables:** the definition uses symbols that are not bound by the input.
- **Partiality:** the definition does not provide an output for all inputs in the domain.

To illustrate, let us fix two functions

$$f : \mathbb{N} \times \mathbb{N} \to \mathbb{N}, \qquad g : \mathbb{N} \to \mathbb{N},$$

and examine a series of possible definitions of functions h. Each case shows whether the definition is well-defined and why.

A correct curried definition

$$h : \mathbb{N} \to \mathbb{N} \to \mathbb{N}, \quad h(m)(n) = f(n, g(m)).$$

This is well-defined: for each $m \in \mathbb{N}$ we obtain $g(m) \in \mathbb{N}$, so $(n, g(m)) \in \mathbb{N} \times \mathbb{N}$ and $f(n, g(m)) \in \mathbb{N}$.

Incorrect composition

$$g \circ f : \mathbb{N} \to \mathbb{N}.$$

This is not well-defined with the given type. Since $f : \mathbb{N} \times \mathbb{N} \to \mathbb{N}$, the composition $g \circ f$ has domain $\mathbb{N} \times \mathbb{N}$, hence $g \circ f : \mathbb{N} \times \mathbb{N} \to \mathbb{N}$, not $\mathbb{N} \to \mathbb{N}$.

A correct nested definition

$$h : \mathbb{N} \to \mathbb{N} \to \mathbb{N} \to \mathbb{N}, \quad h(m)(n, o) = f(n, f(m, o)).$$

This is well-defined: $f(m, o) \in \mathbb{N}$, so $(n, f(m, o)) \in \mathbb{N} \times \mathbb{N}$, and thus $f(n, f(m, o)) \in \mathbb{N}$.

Type mismatch between curried and uncurried

$$h : \mathbb{N} \to \mathbb{N} \to \mathbb{N}, \quad h = f.$$

This is not well-defined, since $f : \mathbb{N} \times \mathbb{N} \to \mathbb{N}$ has a different type from the declared type of h. Mathematically $f(x, y)$ and $f(x)(y)$ can be identified via currying, but in type-theoretic terms they are distinct unless an explicit curry operation is applied.

Free variable problem

$$h : \mathbb{N} \to \mathbb{N}, \quad h(x) = f(x, y).$$

This is not well-defined, since y is not bound. Without fixing y in advance, the right-hand side is not a function of x alone.

Partial definition problem

$$h : \mathbb{N} \times \mathbb{N} \to \mathbb{N}, \quad h(x, 1) = x.$$

This is not a total function, because the definition only specifies values when the second argument is 1. A function of type $\mathbb{N} \times \mathbb{N} \to \mathbb{N}$ must assign a value for every pair (x, y).

Def B.67. Invertible: Let A and B be sets and $f : A \to B$ a function, then f is called invertible, iff there is a function $g : B \to A$, such that $f \circ g = \mathrm{Id}_B$. g is called the inverse function (or just inverse) of f and is written as f^{-1}.

Theorem B.3. A function $f : A \to B$ is invertible, iff it is bijective.

Proof.

1. If f is bijective, then it is invertible:
   - Assume $f : A \to B$ is bijective
   - for each $b \in B$ there exists a unique $a \in A$ such that $f(a) = b$. Define $g : B \to A$ by sending b to this unique a. Therefore:
     1. $g \circ f = \mathrm{Id}_A$, because if you start with $a \in A$, apply f, then g, you get back a.
     2. $f \circ g = \mathrm{Id}_B$, because if you start with $b \in B$, apply g, then f, you get back b.

- Hence, g is the inverse of f. So f is invertible

2. If f is invertible, then it is bijective:

Suppose there exists a function $g : B \to A$ such that $g \circ f = \mathrm{Id}_A$ and $f \circ g = \mathrm{Id}_B$. We want to show f is bijective.

1. injectivity
   (a) Assume $f(a) = f(a')$.
   (b) Apply g to both sides:
   $$a = \mathrm{Id}_A(a) = g(f(a)) = g(f(a')) = a'$$
   (c) So, $a = a'$. Hence f is injective.
2. surjectivity
   (a) Take any $b \in B$. Then
   $$f(g(b)) = (f \circ g)(b) = \mathrm{Id}_B(b) = b$$
   (b) So b is indeed an image of f. Hence f is surjective.

□

Lemma B.4. Let A and B be sets, then the set $\mathcal{R} = \mathcal{P}(A \times B)$ of all relations between A and B and the function space $A \to \mathcal{P}(B)$ are isomorphic.

Proof. For each $R \in \mathcal{R}$ and each $x \in A$ we define: $S_x^R := \{\, y \in B \mid (x, y) \in R \,\}$ (the set of all y in B that are related to x by R). Example:

$$A = \{1, 2\} \quad B = \{a, b, c\} \quad R = \{(1, a), (1, c), (2, b)\}$$

Then for $x = 1 : S_1^R = \{a, c\}$ and for $x = 2 : S_2^R = \{b\}$.

We define a the following mapping:

$$\varphi : \mathcal{R} \to (A \to \mathcal{P}(B)), \quad \varphi(R) = f_R \qquad \text{where } f_R(x) = S_x^R.$$

For our example, this would be $\varphi(R) = f_R = \{1 \mapsto \{a, c\}, 2 \mapsto \{b\}\}$

Then, for any $f \in A \to \mathcal{P}(B)$, we compute the inverse as $\varphi^{-1}(f) = \{(x, y) \mid y \in f(x)\}$. This is exactly the relation corresponding to f.

Since both directions match up perfectly, φ is a bijection, hence an isomorphism. □

Example: the $>$ relation on $\mathbb{N}$ can also be represented as the function $f_>$ that maps a number n to the set $\{n + 1, \cdots\}$, i.e. $f_> : \mathbb{N} \to \mathcal{P}(\mathbb{N})\ ;\ n \mapsto \{\, m \mid m > n \,\}$

Another example: let $A = \{x, y\}, B = \{1\}$, the cartesian product $A \times B = \{(x, 1), (y, 1)\}$. Hence, the power set (all possible relations) is:

$$\mathcal{P}(A \times B) = \{\{\phi\}, \{(x, 1)\}, \{(y, 1)\}, \{(x, 1), (y, 1)\}\}$$

Note: the cardinality of the power set of the cartesian product of two sets A, B is $\Gamma(\mathcal{P}(A \times B)) = 2^{\Gamma(A)\Gamma(B)}$.

$\mathcal{P}(B) = \{\{\phi\}, \{1\}\}$, function space $A \to \mathcal{P}(B)$ is the set of all functions $f : A \to \mathcal{P}(B)$:

$$A \to \mathcal{P}(B) = \{f_1, f_2, f_3, f_4\}$$

where:

- $f_1 = \{(x, \{\phi\}), (y, \{\phi\})\}$
- $f_2 = \{(x, \{\phi\}), (y, \{1\})\}$
- $f_3 = \{(x, \{1\}), (y, \{\phi\})\}$
- $f_4 = \{(x, (1)), (y, \{1\})\}$

Since $\Gamma(A \to \mathcal{P}(B)) = \Gamma(\mathcal{P}(A \times B))$, we can define a bijection:

- $\varphi(\{\phi\}) \mapsto f_1$
- $\varphi(\{(x, 1)\}) \mapsto f_3$
- $\varphi(\{(y, 1)\}) \mapsto f_2$

- $\varphi(\{(x,1),(y,1)\}) \mapsto f_4$

$\rightsquigarrow$ isomorphism.

Note

The bijection $\phi(R) = f$ entails the following properties of R:
- R is a total relation, iff $\phi \notin \operatorname{Im}(f)$
- R is a partial function, iff $\Gamma(S) \leq 1 \quad \forall S \in \operatorname{Im}(f)$
- $R = \phi$, iff $\operatorname{Im}(f) = \{\phi\}$
- R is reflexive, iff $x \in f(x) \quad \forall x \in A$

We have defined equivalence relations as reflexive, symmetric, and transitive relations, e.g., equality is an equivalence relation.

Let S be a set of persons, and $=_A \subseteq S^2$, such that $s =_A t$, iff s and t have the same age, then $=_A$ is an equivalence relation on S.

Let S and T be sets and $S \sim_\Gamma T$, iff there is a bijection $f : S \to T$, then $\sim_\Gamma$ is an equivalence relation. For example, let $S = \{1,2,3\} \quad T = \{a,b,c\}$. We can define a bijection $f = \{1 \mapsto a, 2 \mapsto b, 3 \mapsto c\}$, so, $S \sim_\Gamma T$ (they have the same size).

Lemma B.5. If S is a set and $R \subseteq S^2$ is an equivalence relation, then $= \subseteq R$.

Proof Sketch: because of reflexivity ($\forall x \in S, (x,x) \in R$), the equality relation $=$ is exactly $\{(x,x) \mid x \in S\}$. Therefore, $= \subseteq R$.

B.4 Equivalence Classes

Def B.68. Equivalence Classes and Quotients: Let S be a set and R be an equivalence relation on S, then for any $x \in S$ we call the set $[x]_R := \{\, y \in S \mid R(x,y) \,\}$ the **equivalence class** of x under R, and the set $S/R := \{\, [x]_R \mid x \in S \,\}$ the **quotient space** of S under R, it is often read as S *modulo* R. The element x is called the **representative** of $[x]_R \in S/R$.

Example: Let $S = \mathbb{Z}$ and R is the equivalence relation on S that describes the same parity (congruence modulo 2) $x \equiv y \pmod 2 \Leftrightarrow 2 \mid (x-y)$:

$$R = \{\, (x,y) \in \mathbb{Z} \times \mathbb{Z} \mid x \equiv y \pmod 2 \,\}$$

- $(x,y) \in R$ if the difference $x - y$ is divisable by 2.
- i.e. either both are even, or both are odd.

All odd-odd pairs:

$$\{\, (x,y) \mid x, y \in \{\cdots, -3, -1, 1, 1, 3, 5, \cdots\} \,\}$$

All even-even pairs:

$$\{\, (x,y) \mid x, y \in \{\cdots, -4, -2, 0, 2, 4, 6, \cdots\} \,\}$$

Example pairs in R: $(1,3), (5,-1), (7,7), (2,4), (0,-6), (-8,10)$ Then, we have the following two distinct classes:

- $[1]_R = \{\cdots, -3, -1, 1, 1, 3, 5, \cdots\}$
- $[2]_R = \{\cdots, -4, -2, 0, 2, 4, 6, \cdots\}$

For each $x \in \mathbb{Z}$ we have:

- the set $[1]_R$ as the equivalence class if x is odd
- the set $[2]_R$ as the equivalence class if x is even
- 1 and 2 are the representatives

And the quotient space set of $\mathbb{Z}$ under R is:

$$\mathbb{Z}/R = \{[1]_R, [2]_R\} = \{\text{odds}, \text{evens}\}$$

Def B.69. Canonical Projection: The mapping $\pi_R : S \to S/R \quad ; \quad x \mapsto [x]_R$ is called the **canonical projection** or **canonical surjection** of S to S/R.

Given our example, we define the following canonical projection:

$$\pi_R : \mathbb{Z} \to \{[1]_R, [2]_R\}, \quad x \mapsto [1]_R \text{ if odd, and } x \mapsto [2]_R \text{ if even.}$$

Def B.70. System of Representatives: Let R be an equivalence relation on S, a subset $M \subseteq S$ is called a system of representatives, iff M contains exactly one representative for each equivalence class of R.

Given our example, a system of representatives is any subset of $\mathbb{Z}$ that picks exactly one element from each equivalence class, e.g.

- $M = \{1, 2\}$ is a system of representatives
- $M = \{-3, 0\}$ is a system of representatives
- $M = \{5, -8\}$ is a system of representatives

Observation: Often a quotient space S/R behaves similarly to the original set S.

Example: $n \equiv_k m \Leftrightarrow k \mid (n-m)$ is an equivalence relation and $\pi_{\equiv_k} : \mathbb{Z}/\equiv_k \to \{\, n \mid 0 \leq n \leq (k-1) \,\}$ is bijective.

Another example, the general congruence modulo $n \equiv_k m \Leftrightarrow k \mid (n-m)$ partitions $\mathbb{Z}$ into exactly k equivalence classes (all numbers that leave the same remainder when divided by k). e.g. $k = 3$:

- $[0] = \{\cdots, -6, -3, 0, 3, 6, \cdots\}$
- $[1] = \{\cdots, -5, -2, 1, 4, 7, \cdots\}$
- $[2] = \{\cdots, -4, -1, 2, 5, 8, \cdots\}$

We have the canonical projection $\pi_{\equiv_k} : \mathbb{Z} \to \mathbb{Z}/\equiv_k, \quad n \mapsto [n]$ that sends each $n \in \mathbb{Z}$ to its equivalence class.

Notice that the quotient space $\mathbb{Z}/\equiv_k$ has exactly k classes, so, we have the following bijection:

$$\varphi : \mathbb{Z}/\equiv_k \to \{0, 1, \cdots, k-1\}, \quad [n] \mapsto \text{ the remainder of } n \bmod k.$$

If we compose the canonical projection with the bijection we get the remainder, e.g. for $k = 3$:

$$-6 \xmapsto{\pi_{\equiv_3}} [0] \xmapsto{\varphi} 0.$$

$$7 \xmapsto{\pi_{\equiv_3}} [1] \xmapsto{\varphi} 1.$$

Lemma B.6. Let $=\subseteq S \times S$ be the equality relation on a set S, then $[x]_= \in S/=$ is always a **singleton** and thus $S \sim_\Gamma S/=$.

Proof. If we take the equality relation on a set S, then for any $x \in S$, the equivalence class is:

$$[x]_= = \{\, y \in S \mid y = x \,\}$$

But the only y that satisfies $y = x$ is $y = x$ itself. So we get $[x]_= = \{x\}$. This is a **singleton** set. So every equivalence class under $=$ has exactly one element.

The quotient space is $S/= \;:= \{\, [x]_= \mid x \in S \,\}$ and each $[x]_=$ is the singleton set $\{x\}$, so the quotient space becomes $S/= \;:= \{\, \{x\} \mid x \in S \,\}$. In other words, the quotient space under the equality relation just replaces each element x with one-element set $\{x\}$.

That means the set S is in bijection with its quotient space under $=$ via the following bijective map:

$$\phi : S \to S/=, \quad x \mapsto [x]_= = \{x\}$$

Therefore, we express the bijection via same cardinality:

$$S \sim_\Gamma S/=$$

□

Fun Example: Let S be the set of all cities worldwide and $\sim_c \in S^2$ defined by $a \sim_c b$, iff a and b are in the same country. Then $\sim_c$ is an equivalence relation on S, and the quotient space $S/\sim_c$ of S under $\sim_c$ consists of the equivalence class $[x]_{\sim_c}$ where $x \in S$. The capital of a country is a good representative for an equivalence class.

For instance: let's take a subset of all worldwide cities for simplicity, consider domain of discourse:

$$S = \{\text{Berlin}, \text{Munich}, \text{Hamburg}, \text{Paris}, \text{Lyon}, \text{Marseille}, \text{Rome}, \text{Milan}\}.$$

For $a, b \in S$ define the relation:

$$a \sim_c b \iff a \text{ and } b \text{ are in the same country.}$$

This relation is:

- reflexive: any city is in the same country as itself.
- symmetric: if Berlin is in the same country as Munich, then Munich is in the same country as Berlin.
- transitive: if Berlin and Munich are in the same country, and Munich and Hamburg are in the same country, then Berlin and Hamburg are in the same country.

Hence, $\sim_c$ is an equivalence relation.

The quotient space $S/\sim_c$ is the set of equivalence classes, i.e. the partition of S into countries:

$$[\text{Berlin}]_{\sim_c} = \{\text{Berlin}, \text{Munich}, \text{Hamburg}\}, \quad [\text{Paris}]_{\sim_c} = \{\text{Paris}, \text{Lyon}, \text{Marseille}\}, \quad [\text{Rome}]_{\sim_c} = \{\text{Rome}, \text{Milan}\}.$$

So

$$S/\sim_c = \Big\{\{\text{Berlin}, \text{Munich}, \text{Hamburg}\},\ \{\text{Paris}, \text{Lyon}, \text{Marseille}\},\ \{\text{Rome}, \text{Milan}\}\Big\}.$$

The set of chosen representatives is: $\{\text{Berlin}, \text{Paris}, \text{Rome}\}$.

B.5 Algebraic Structures

Def B.71. Mathematical (Algebraic) Structure: A mathematical structure combines multiple mathematical objects (the components) into a new object. The components usually have names by which they can be referenced. Given a definition of a mathematical structure S, we say that any object that conforms to that is an instance of S.

Def B.72. Structure Signature: A structure signature combines the components, their types, and accessor names of a mathematical structure in a tabular overview.

Def B.73. Group: A group is a mathematical structure $\langle G, \circ, e, .^{-1} \rangle$ that consists of:

- a base set G of objects,
- an operation $\circ : G \times G \to G$, such that $\forall a, b \in G. a \circ b \in G$ and $\forall a, b, c \in G. (a \circ b) \circ c = a \circ (b \circ c)$,
- a unit $e \in G$, such that $\forall a \in G. e \circ a = a \circ e = a$, and
- the inverse function $.^{-1} : G \to G$, such that $\forall a \in G. a \circ a^{-1} = a^{-1} \circ a = e$

An example of a group is the set $G = \mathbb{Z}$, with the operation $+ : \mathbb{Z} \times \mathbb{Z} \to \mathbb{Z}$; then $e = 0$ and $.^{-1}$ is $\lambda x \in \mathbb{Z}. -x$. We write it as $\langle \mathbb{Z}, +, 0, - \rangle$.

Question: can we do the same for multiplication where $e = 1$? If we think about it, we would be stuck on finding the $.^{-1}$ component of the group. What can we always multiply with any integer and get $e = 1$ back? Take for example 1, we can multiply it with $\frac{1}{1}$ (great!). Take -1, we can multiply it with $\frac{1}{-1}$ (also great!). But take any other integer, e.g. 2, we must multiply it with $\frac{1}{2}$ to get 1 back, however, $\frac{1}{2} \notin \mathbb{Z}$, it is however $\in \mathbb{Q}$! Moreover, 0 never has a multiplicative inverse in any field! The multiplicative group would only make sense iff $G = \mathbb{Q} \setminus \{0\}$. We write that as $\langle \mathbb{Q} \setminus \{0\}, *, 1, a \mapsto \frac{1}{a} \rangle$.

But we need some multiplication structure that works for $\mathbb{Z}$!

Def B.74. Monoid: A monoid is a mathematical structure $\langle M, \circ, e \rangle$ that consists of:

- A base set M of objects,
- An operation $\circ : M \times M \to M$, such that for all $a, b \in M$, we have $a \circ b \in M$ (closure),
- Associativity: for all $a, b, c \in M$, $(a \circ b) \circ c = a \circ (b \circ c)$, and
- A unit element $e \in M$, such that for all $a \in M. e \circ a = a \circ e = a$.

Hence, the mathematical structure $\langle \mathbb{Z}, *, 1 \rangle$ is a monoid (a group missing an inverse function).

We saw that $\langle \mathbb{Z}, +, 0, - \rangle$ is a group under addition. And $\langle \mathbb{Z}, *, 1 \rangle$ is a monoid under multiplication. Now, we want to do everyday arithmetics with integers; we want expressions where we have additions and multiplication (e.g. $a * (b + c)$) which evaluates to $(a * b) + (a * c)$.

For that, we need a mathematical structure where both addition and multiplication live together.

Def B.75. Ring: A ring is a mathematical structure $\langle R, +, 0, -, *, 1 \rangle$ consisting of:

- A base set R.
- An addition operation $+ : R \times R \to R$ such that $\langle R, +, 0, - \rangle$ is a group (called abelian group).
- A multiplication operation $* : R \times R \to R$ such that $\langle R, *, 1 \rangle$ is a monoid.
- Distributivity: for all $a, b, c \in R, a * (b + c) = (a * b) + (a * c), (a + b) * c = (a * c) + (b * c)$

Hence, mixing components from the group $\langle \mathbb{Z}, +, 0, - \rangle$ and the monoid $\langle \mathbb{Z}, *, 1 \rangle$ leaves us with a ring $\langle \mathbb{Z}, +, 0, -, *, 1 \rangle$

Def B.76. Magma: A magma is a mathematical structure $\langle M, \circ \rangle$ consisting of:

- A base set M, and
- A binary operation $\circ : M \times M \to M$.

Example: $\langle \mathbb{Z}, - \rangle$ (integers under subtraction). $\langle \mathbb{N}, + \rangle$ (natural numbres under addition)

Def B.77. Semigroup: A semigroup (magma with associativity) is a mathematical structure $\langle S, \circ \rangle$ consisting of:

- A base set S,
- A binary opertation $\circ : S \times S \to S$, and
- Associativity: for all $a, b, c \in S, (a \circ b) \circ c = a \circ (b \circ c)$

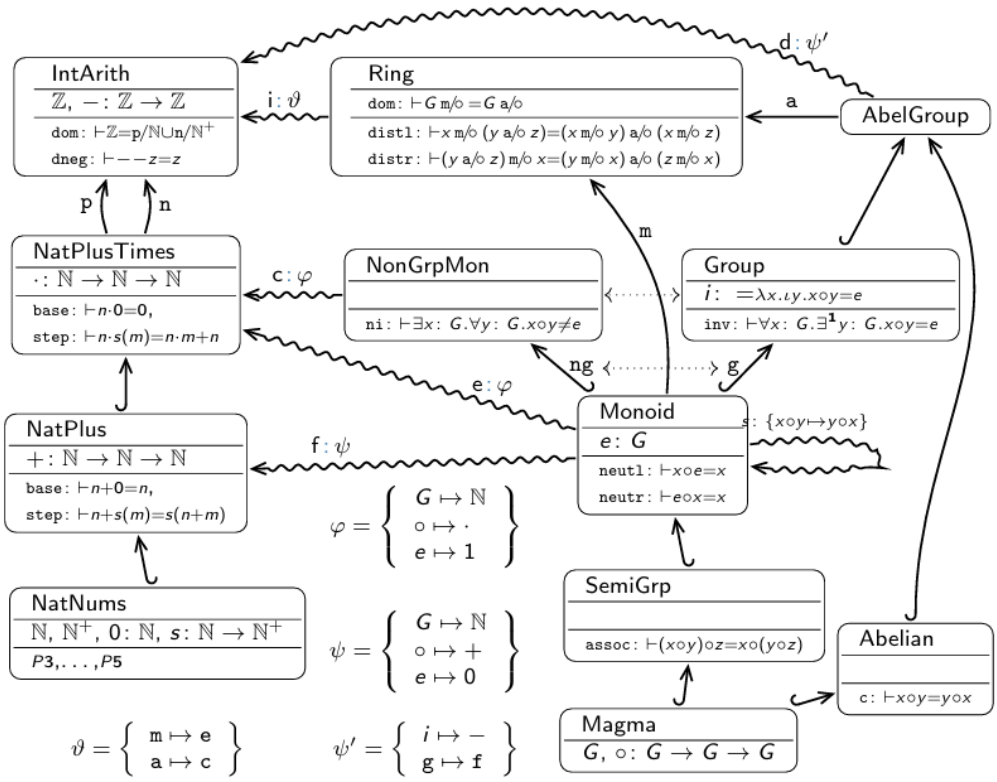
Example: $\langle \mathbb{Z}, + \rangle$ (integers under addition).

> **Note**
>
> Given collections M and S of instances of magmas and semigroups, $S \subseteq M$ as there are magmas whose operation is not associative.

Def B.78. Abelian Group: An abelian group (or commutative group) is a group $\langle G, \circ, e, .^{-1} \rangle$ such that:

$$\forall a, b \in G. a \circ b = b \circ a$$

Mathematical Structures in a Graph:



- nodes are structures with objects ($\circ : \tau$) and properties ($n :\vdash P$).
- edges represent inheritance
  - regular inheritance: $\hookrightarrow$
  - copying substructures: $\xrightarrow{n}$
  - interpretation via φ: $\overset{n}{\rightsquigarrow}$
- Examples are just interpretations ($\rightsquigarrow$) read backwards
- Any object / property / theorem can be copied along any edge

Def B.79. Field: A field is a mathematical structure

$$\langle F, +, 0, -, *, 1 \rangle$$

consisting of:

- A base set F,
- An addition operation such that $\langle F, +, 0, - \rangle$ is an abelian group,

- A multiplication operation such that
$$\left\langle F \setminus \{0\}, *, 1, a \mapsto \frac{1}{a} \right\rangle$$
is an abelian group,
- Distributivity:
$$a * (b + c) = (a * b) + (a * c)$$
and
$$(a + b) * c = (a * c) + (b * c)$$
for all $a, b, c \in F$.

Examples:

- $\mathbb{Q}$ (rational numbers) is a field. Every non-zero rational number has a multiplicative inverse in $\mathbb{Q}$. For example:
$$2^{-1} = \frac{1}{2}$$
- $\mathbb{R}$ (real numbers) is a field.
- $\mathbb{C}$ (complex numbers) is a field.
- $\mathbb{Z}$ (integers) is **not** a field, because most integers do not have multiplicative inverses in $\mathbb{Z}$. For example:
$$\frac{1}{2} \notin \mathbb{Z}$$
- The set of matrices
$$\mathbb{R}^{n \times n}$$
is generally **not** a field, because matrix multiplication is not commutative and some non-zero matrices are not invertible.

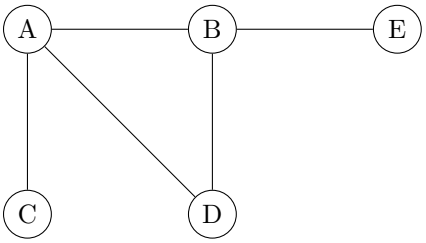
C Topics in Computer Science

C.1 Graph Theory

Def C.1. Undirected Graph: An undirected graph is a pair $\langle V, E\rangle$ such that

- V is a set of **vertices (nodes)**, and
- $E \subseteq \{\,\{v, \bar{v}\} \mid v, \bar{v} \in V \wedge v \neq \bar{v}\,\}$ is the set of its **undirected edges**.

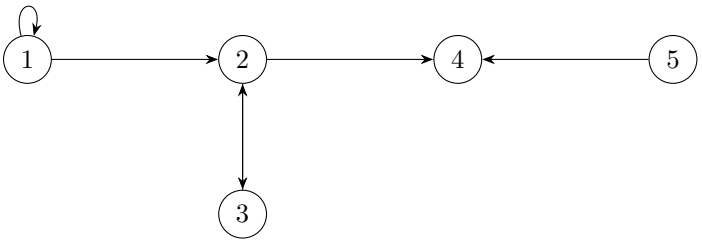
Example: An undirected graph $G_1 = \langle V_1, E_1\rangle$, where $V_1 = \{A, B, C, D, E\}$ and $E_1 = \{\{A, B\}, \{A, C\}, \{A, D\}, \{B, D\}, \{B, E\}\}$



Def C.2. Directed Graph: A directed graph (digraph) is a pair $\langle V, E\rangle$ such that

- V is a set of **vertices (nodes)**, and
- $E \subseteq V \times V$ is the set of its **directed edges**.

Example: A directed graph $G_2 = \langle V_2, E_2\rangle$, where $V_2 = \{1, 2, 3, 4, 5\}$ and $E_2 = \{(1, 1), (1, 2), (2, 3), (3, 2), (2, 4), (5, 4)\}$



Def C.3. Indegree: Given a directed graph $\langle V, E\rangle$, the indegree $\text{indeg}(v)$ of a vertex $v \in V$ is defined as $\text{indeg}(v) = \Gamma(\{\, w \mid (w, v) \in E\,\})$.

Example: $\text{indeg}(2) = 2$

Def C.4. Outdegree: Given a directed graph $\langle V, E\rangle$, the outdegree $\text{outdeg}(v)$ (branching factor) of a vertex $v \in V$ is defined as $\text{outdeg}(v) = \Gamma(\{\, w \mid (v, w) \in E\,\})$.

Example: $\text{outdeg}(5) = 1$

> Note
>
> For an undirected graph, $\text{indeg}(v) = \text{outdeg}(v)$ for all $v \in V$.

Observation: directed graphs are nothing else than relations.

Def C.5. Initial vs Terminal Node: Let $G = \langle V, E\rangle$ be a directed graph, then we call a node $v \in V$

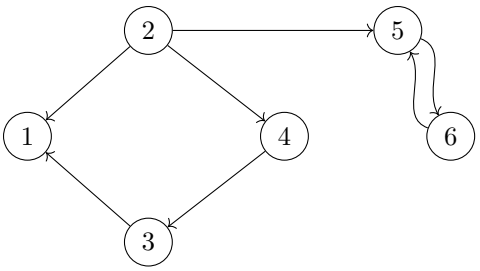
- **initial** (source of G), iff there is no $w \in V$ such that $(w, v) \in E$ (no predecessor)
- **terminal** (sink of G), iff there is no $w \in V$ such that $(v, w) \in E$ (no successor)

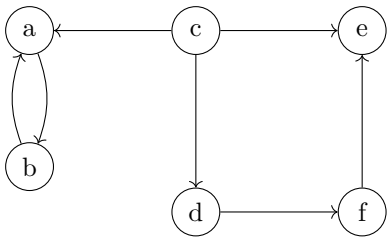
Def C.6. Graph Isomorphism: Iff we can find a bijection $\psi : V \to \bar{V}$ between two graphs $G = \langle V, E\rangle$ and $\bar{G} = \langle \bar{V}, \bar{E}\rangle$ then we call them isomorphic. The bijection $\psi : V \to \bar{V}$ is defined as $(a, b) \in E \Leftrightarrow (\psi(a), \psi(b)) \in \bar{E}$ for directed graph. And for undirected graph it is defined as $\{a, b\} \in E \Leftrightarrow \{\psi(a), \psi(b)\} \in \bar{E}$

Def C.7. Equivalent Graphs: Two graphs G and $\bar{G}$ are **equivalent** iff there is an isomorphism ψ between G and $\bar{G}$.

Example: the following two graphs are equivalent as there exists an isomorphism between them given by the bijection:

$$\psi := \{a \mapsto 5, b \mapsto 6, c \mapsto 2, d \mapsto 4, e \mapsto 1, f \mapsto 3\}$$

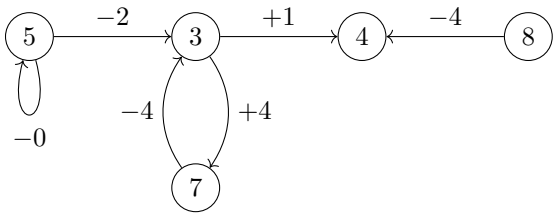




Def C.8. Labeled Graph: A labeled graph G is a quadruple $\langle V, E, L, l\rangle$ where $\langle V, E\rangle$ is a graph and $l : V \cup E \rightharpoonup L$ is a partial function into a set L of labels.

Example: $G = \langle V, E, L, l\rangle$ with $V = \{A, B, C, D, E\}$, where

- $E = \{(A, A), (A, B), (B, C), (C, B), (B, D), (E, D)\}$
- $l : V \cup E \rightharpoonup \{+, -, \phi\} \times \{1, \cdots, 9\}$ with
  - $l(A) = 5 \quad l(B) = 3 \quad l(C) = 7 \quad l(D) = 4 \quad l(E) = 8,$
  - $l((A, A)) = -0 \quad l((A, B)) = -2 \quad l((B, C)) = +4,$
  - $l((C, B)) = -4 \quad l((B, D)) = +1 \quad l((E, D)) = -4$



Def C.9. Paths in Graphs: Given a graph $G := \langle V, E\rangle$ we call a $n+1$-tuple $p = \langle v_0, \cdots, v_n\rangle \in V^{n+1}$ a **path** in G iff $(v_{i-1}, v_i) \in E$ for all $1 \leq i \leq n$ and $n > 0$.

- We say that v_i are nodes on p and that v_0 and v_n are **linked** by p.
- v_0 and v_n are called the **start** and **end** of p (write start(p) and end(p)), the other v_i are called **inner nodes** of p.
- n is called the **length** of p (write len(p)).
- We denote the set of paths in G with $\Pi(G)$.

Note

Not all v_i-s in a path are necessarily different.

For a graph $G = \langle V, E\rangle$ and a path $p = \langle v_1, \cdots, v_n\rangle \in V^{n+1}$, write

- $v \in p$, iff $v \in V$ is a vertex on the path ($\exists i. v_i = v$)
- $e \in p$, iff $e = (v, \bar{v}) \in E$ is an edge on the path ($\exists i. v_i = v \wedge v_{i+1} = \bar{v}$)

Example: Consider $G = \langle V, E\rangle$ with $V = \{a, b, c, d\}$ and $E = \{(a, b), (b, c)\}$:

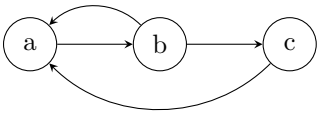
An example path would be $p = \langle a, b, c\rangle$. Here $n = 2$ and $p \in V^3 = \{(a, a, a), (a, b, d), \cdots\}$. Obviously, since $n = 2$ so we have 2 edges and 3 nodes.

- nodes a, b, c are on the path
- start(p) $= a$ and end(p) $= c$
- b is an inner node, and len(p) $= 2$
- $e = (a, b), \bar{e} = (b, c) \in p$
- $\Pi(G) = \{(a, b), (b, c), (a, b, c)\}$

Def C.10. Cyclic Graphs: Given a directed graph $G = \langle V, E \rangle$, a path p is called **cyclic** iff $\text{start}(p) = \text{end}(p)$. A cycle $\langle v_0, \cdots, v_n \rangle$ is called **simple**, iff $v_i \neq v_j$ for $1 \leq i, j \leq n$ with $i \neq j$ (all inner nodes are distinct).

Def C.11. DAG: A directed graph with no cycles is called **directed acyclic graph (DAG)**.

Example: $p = \langle a, b, c, a \rangle$ is simple cyclic path. $p = \langle a, b, a, b, a \rangle$ is a cyclic path that is not simple.



Def C.12. Node Depth: Let $G = \langle V, E \rangle$ be a directed graph, then the depth $\text{dp}(v)$ of a vertex $v \in V$ is defined to be 0, iff v is a source of G and the supremum $\sup(\{\, \text{len}(p) \mid \text{indeg}(\text{start}(p)) = 0 \wedge \text{end}(p) = v \,\})$ otherwise, i.e. the length of the longest path from a source of G to v (can be infinite).

Def C.13. Graph Depth: Given a directed graph $G = \langle V, E \rangle$, the **depth** ($\text{dp}(G)$) of G is defined as the supremum $\sup(\{\, \text{len}(p) \mid p \in \Pi(G) \,\})$, i.e. the maximal path length in G.

Def C.14. Tree: A tree is a DAG $G = \langle V, E \rangle$ such that

- There is exactly one initial node $v_r \in V$ (called the **root**)
- All nodes but the root have indegree of 1.

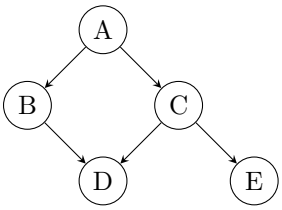
We call v the **parent** of w, iff $(v, w) \in E$ (w is a child of v). We call a node v a **leaf** of G, iff it is terminal, i.e. if it does not have children.

Lemma C.1. For any node $v \in V$ except the root v_r, there is exactly one path $p \in \Pi(G)$ with $\text{start}(p) = v_r$ and $\text{end}(p) = v$.

Def C.15. Non-Descendants: Let $\langle X, E \rangle$ be a DAG, $x \in X$, and E^* the reflexive transitive closure of E. The **non-descendants** of x are the elements of the set $\text{NonDesc}(x) := \{y \mid (x, y) \notin E^*\} \setminus \text{Parents}(x)$

Example: Consider the following DAG $G = \langle X, E \rangle$ where:

- $X = \{A, B, C, D, E\}$
- $E = \{(A, B), (A, C), (B, D), (C, D), (C, E)\}$
- $E^* = \{(A, A), (B, B), (C, C), (D, D), (E, E), (A, B), (A, C), (A, D), (A, E), (B, D), (C, D), (C, E)\}$



Let's find the **non-descendants** of node B:

- First, we find all y such that $(B, y) \in E^*$. Looking at E^*, the only pairs starting with B are (B, B) and (B, D). Thus, the reachable nodes from B are $\{B, D\}$.
- The set of nodes y where $(B, y) \notin E^*$ is everything else: $\{A, C, E\}$.
- The parents of B is the set $\text{Parents}(B) = \{A\}$.
- By the definition, we exclude the parents: $\text{NonDesc}(B) = \{A, C, E\} \setminus \{A\} = \{C, E\}$.

C.2 Complexity Theory

Def C.16. Algorithm: An algorithm is a series of instructions to control a (computation) process.

Def C.17. Computational Problem: A computational problem is a triple $\langle I, S, \sigma \rangle$, where I is a set of inputs, S is a set of solutions, and $\sigma : I \to \mathcal{P}(S)$ asigns to each input the set of valid solutions. The problem is called **computable** if there exists an algorithm $\alpha : I \to S$ such that for every $i \in I, \alpha(i) \in \sigma(i)$ and $\alpha(i)$ is defined, i.e. the algorithm always **halts**.

Def C.18. Partially Computable Problem: A computational problem $\langle I, S, \sigma \rangle$ is called **partially computable** if there exists a *partial* algorithm $\alpha : I \rightharpoonup S$ such that for every $i \in I$, if $\alpha(i)$ is defined then $\alpha(i) \in \sigma(i)$. In this case α may diverge on some inputs.

Def C.19. Resource: A resource is a measurable quantity consumed by an algorithm during execution. The most common resources are **time** (number of computation steps) and **space** (amount of memory used).

Def C.20. Complexity Theory: (Computational) complexity theory focuses on classifying computational problems according to their resource usage, and relating these classes to each other.

Def C.21. Complexity Measure: A complexity measure is a mapping $m : I \to \mathbb{N}$ that assigns to each input $i \in I$ a natural number describing the amount of some resource consumed by an algorithm on input i.

Def C.22. Input Size: The size of an input $i \in I$, denoted $|i|$, is a natural number measuring the length of the encoding of i (for instance, the number of bits required to represent i).

Def C.23. Asymptotic Analysis: Asymptotic analysis studies the growth of a complexity measure as the input size n tends to infinity. It abstracts from constant factors and lower-order terms to capture the long-term behavior of an algorithm.

Def C.24. Running Time: If an algorithm α terminates within $t(n)$ steps on every input of size n, then we say that α has **running time** $T(\alpha) := t$.

Def C.25. Time Complexity: Let $S \subseteq \mathbb{N} \to \mathbb{N}$ be a set of natural number functions. An algorithm α with running time $T(\alpha) = t$ is said to have **time complexity** in S (written $T(\alpha) \in S$, or colloquially $T(\alpha) = S$) if $t \in S$.

Def C.26. Space Complexity: Let $S \subseteq \mathbb{N} \to \mathbb{N}$ be a set of natural number functions. An algorithm α is said to have **space complexity** in S if, on every input of size n, α uses at most $s(n)$ units of memory for some $s \in S$.

Def C.27. Best-Case Complexity: The best-case complexity of an algorithm α is the function

$$T_\alpha^{\text{best}}(n) = \min\{\text{running time of } \alpha(i) \mid i \in I, |i| = n\}.$$

It describes the minimal resource usage among all inputs of size n.

Def C.28. Average-Case Complexity: The average-case complexity of an algorithm α is the expected resource usage over all inputs of size n, with respect to some probability distribution D on I. Formally,

$$T_\alpha^{\text{avg}}(n) = \mathbb{E}_{i \sim D, |i|=n}[\text{running time of } \alpha(i)].$$

Def C.29. Worst-Case Complexity: The worst-case complexity of an algorithm α is the function

$$T_\alpha^{\text{worst}}(n) = \max\{\text{running time of } \alpha(i) \mid i \in I, |i| = n\}.$$

It describes the maximal resource usage among all inputs of size n.

> **Note**
>
> We are mostly interested in worst-case complexity.

Def C.30. Big-$\mathcal{O}$ Notation: In practice, asymptotic analysis is most often applied to the worst-case complexity. Let $f, g : \mathbb{N} \to \mathbb{R}_{\geq 0}$. We say that f is **asymptotically bounded** by g, written $f \in \mathcal{O}(g)$ or $f \leq_a g$, if there exist constants $c > 0$ and $n_0 \in \mathbb{N}$ such that

$$\forall n \geq n_0 : \quad f(n) \leq c \cdot g(n).$$

Equivalently, the **Landau set** $\mathcal{O}(g)$ is the set of all such functions:

$$\mathcal{O}(g) = \{f \mid \exists c > 0, \exists n_0 \in \mathbb{N}, \forall n \geq n_0 : f(n) \leq c \cdot g(n)\}.$$

Example: Let $f(n) = 3n^2 + 2n + 7$ and $g(n) = n^2$. We want to show that $f \in \mathcal{O}(g)$.

By the definition of Big-$\mathcal{O}$, this means we must find constants $c > 0$ and $n_0 \in \mathbb{N}$ such that

$$\forall n \geq n_0 : \quad f(n) \leq c \cdot g(n).$$

Let's take $n_0 = 1$, then $\forall n \geq 1$ we have:

$$2n \leq 2n^2 \quad \text{and} \quad 7 \leq 7n^2.$$

Therefore,

$$f(n) = 3n^2 + 2n + 7 \quad \leq \quad 3n^2 + 2n^2 + 7n^2 = 12n^2.$$

Here we see that $f(n) \leq 12 \cdot g(n)$ for all $n \geq 1$. Thus the definition holds with $c = 12$ and $n_0 = 1$. We conclude that $f(n) \in \mathcal{O}(n^2)$.

Lemma C.2. Growth Ranking. For $\bar{k} > 2$ and $k > 1$ we have:

$$\mathcal{O}(1) \subset \mathcal{O}(\log_2(n)) \subset \mathcal{O}(n) \subset \mathcal{O}(n^2) \subset \mathcal{O}(n^{\bar{k}}) \subset \mathcal{O}(k^n)$$

The following sets are often used for S in $T(\alpha)$:

Landau set	class name	rank	Landau set	class name	rank
$\mathcal{O}(1)$	constant	1	$\mathcal{O}(n^2)$	quadratic	4
$\mathcal{O}(\log_2 n)$	logarithmic	2	$\mathcal{O}(n^k)$	polynomial	5
$\mathcal{O}(n)$	linear	3	$\mathcal{O}(k^n)$	exponential	6

Table 6: Common complexity classes

Computing with Landau sets is quite simple:

- $\mathcal{O}(c \cdot f) = \mathcal{O}(f)$ for any constant $c \in \mathbb{N}$ (*drop constant factors*)
- $\mathcal{O}(f) \subseteq \mathcal{O}(g) \Rightarrow \mathcal{O}(f + g) = \mathcal{O}(g)$ (*drop low-complexity summands*)
- $\mathcal{O}(f \cdot g) = \mathcal{O}(f) \cdot \mathcal{O}(g)$ (*distribute over products*)

Example: $\mathcal{O}(4n^3 + 3n + 7^{1000n}) = \mathcal{O}(2^n)$ (*exponential*)

Note

$\mathcal{O}(n) \subset \mathcal{O}(n \cdot log_2(n)) \subset \mathcal{O}(n^2)$ (*linearithmic*)

Note

Updated Growth Ranking & Common Classes
To account for all algorithmic patterns discussed (such as nested loops with logarithmic guards), we refine the growth hierarchy. For $a > 2$ and $b > 1$, the asymptotic ranking is :

$$\mathcal{O}(1) \subset \mathcal{O}(\log_2 n) \subset \mathcal{O}(n) \subset \mathcal{O}(\mathbf{n \log_2 n}) \subset \mathcal{O}(n^2) \subset \mathcal{O}(n^a) \subset \mathcal{O}(b^n)$$

Determining the Time/Space Complexity of Algorithms

Assume we have

- Γ: a mapping that tells us the complexity of variables.
- α: a program fragment.
- $T_\Gamma(\alpha)$: the time complexity of α under Γ.
- $C_\Gamma(\alpha)$: the context update, i.e. what new cost α introduces.

We use induction on program strcuture:

1. Constant

If $\alpha = \delta$ (where δ is a literal, like 5 or `true`), then $T_\Gamma(\alpha) \in \mathcal{O}(1)$

2. Variable

If $\alpha = v$ and $v \in \text{dom}(\Gamma)$, then $T_\Gamma(\alpha) \in \mathcal{O}(\Gamma(v))$

3. Function Call

If $\alpha = \varphi(\psi)$, with $T_\Gamma(\alpha) \in \mathcal{O}(f)$, and $T_{\Gamma \cup C_\Gamma(\varphi) \in \mathcal{O}(g)}$, then $T_\Gamma(\alpha) \in \mathcal{O}(f \circ g)$

4. Assignment

If $\alpha = v := \varphi$, with $T_\Gamma(\varphi) \in S$, then $T_\Gamma(\alpha) \in S$, $C_\Gamma(\alpha) = \Gamma \cup (v, S)$

5. Composition (sequence)

If $\alpha = \varphi; \psi$, with $T_\Gamma(\varphi) \in P$, and $T_{\Gamma \cup C_\Gamma(\varphi)}(\psi) \in Q$, then $T_\Gamma(\alpha) \in \max\{P, Q\}$

6. Branching

If $\alpha =$ `if` γ `then` φ `else` ψ `end`, with $T_\Gamma(\gamma) \in C$, $T_{\Gamma \cup C_\Gamma(\gamma)}(\varphi) \in P$, and $T_{\Gamma \cup C_\Gamma(\gamma)}(\psi) \in Q$, then $T_\Gamma(\alpha) \in \max\{C, P, Q\}$

7. Loop

If $\alpha =$ `while` γ `do` φ `end`, with $T_\Gamma(\gamma) \in \mathcal{O}(f)$, and $T_{\Gamma \cup C_\Gamma(\gamma)}(\varphi) \in \mathcal{O}(g)$, then $T_\Gamma(\alpha) \in \mathcal{O}(f(n) \cdot g(n))$

> Note
>
> By default, the overall time complextiy is $T(\alpha) = T_\phi(\alpha)$

Sequence Example

```
x := readArray(n)   -- O(n)
y := sum(x)         -- O(n)
```

$\max(\mathcal{O}(n), \mathcal{O}(n)) = \mathcal{O}(n)$

Conditional Example

```
if isSorted(x) then     -- O(n)
    binarySearch(x,k)   -- O(log n)
else
    linearSearch(x,k)   -- O(n)
end
```

$\max(\mathcal{O}(n), \mathcal{O}(\log(n), \mathcal{O}(n))) = \mathcal{O}(n)$

While loop example

```
i := 1
while i <= n do
    i := i + 1
end
```

Guard (loop condition check) runs $\mathcal{O}(n)$ times, body is $\mathcal{O}(1)$. Total $\mathcal{O}(n \cdot 1) = \mathcal{O}(n)$

Function Call Example

Suppose `buildArray(n)` $\in \mathcal{O}(n^2)$ returns an array of size n^2 and `sort(m)` $\in \mathcal{O}(m \cdot \log(m))$

```
sort(buildArray(n))
```

$\mathcal{O}((n^2) \cdot \log(n^2)) = \mathcal{O}(n^2 \cdot \log(n))$

Nested For Loop Example

```
for i = 1 to n do       -- O(n)
    for j = 1 to n do   -- O(n)
        x := x + 1      -- O(1)
```

Inner loop: $\mathcal{O}(n \cdot 1)$, outer loop: $\mathcal{O}(n \cdot n \cdot 1)$, total: $\mathcal{O}(n^2)$

Nested (one logarthmic loop)

```
for i = 1 to n do
    k := 1
    while k < n do
        k := 2 * k
```

Inner loop starts with $k = 1$, each iteration k doubles and the loop stops when $k \geq n$. How many times can we double before reaching n? Solve $2^t \geq n \rightsquigarrow t = \lceil \log_2(n) \rceil$. So the inner loop runs $\mathcal{O}(\log(n))$ iterations. Each iteration is $\mathcal{O}(1)$, therefore total $\mathcal{O}(\log(n))$.

Outer loop runs exactly n times with each time the cost is $\mathcal{O}(\log(n))$. Total cost is $n \cdot \mathcal{O}(\log(n)) = \mathcal{O}(n \cdot \log(n))$

> Note
>
> Take home message: not every time you see two nested loops you assume $\mathcal{O}(n^2)$

Here is another example: assume `<<body>>` $\in \mathcal{O}(1)$:

```
i, j := 0
while i <= n-1 do
    i := i+1
    while j <= 10 do
        j := j+1
        <<body>>
    end
end
```

Outer Loop: i increments by 1 each time untill $i > n - 1$ we stop, so it runs n times.

Inner Loop: the condition is $j \leq 10$, but note that j is initialized once before the loop and not reset for each i! That means, first outer iteration $j = 0$, runs until $j = 11$, i.e. 11 iterations. After that, $j = 11$ always and the condition $j \leq 10$ is false, so the inner loop never runs again. Inner loop total cost is $\mathcal{O}(11) = \mathcal{O}(1)$

Therefore, all in all we have linear time ($\mathcal{O}(n)$).

C.3 Formal Languages and Grammars

Def C.31. Alphabet: An alphabet is a finite set; we call each element $a \in A$ a **character**, and an n-tuple $s \in A^n$ a **string** (of *length* n over A).

> Note
>
> We often write a string $\langle c_1, \cdots, c_n \rangle$ as "$c_1 \cdots c_n$", for instance `"abc"` for $\langle a, b, c \rangle$

Example: Let $A = \{0, 1\}$ be an alphabet

- `"0", "1"` are strings of length 1 ($A^1 = \{0, 1\}$)
- `"00","01","10","11"` are strings of length 2 in (A^2)
- `"001"` $\in A^3$

Def C.32. Empty String: Let A be an alphabet, then $A^0 = \{\langle\rangle\}$, where $\langle\rangle$ is the unique 0-tuple. We consider $\langle\rangle$ as the string of length 0 and call it the **empty string** and denote it with ϵ.

> Note
>
> $\{1, 2, 3\} = \{3, 2, 1\}$ BUT $\langle 1, 2, 3 \rangle \neq \langle 3, 2, 1 \rangle$ (sets vs strings)

Def C.33. String Length: Given a string s, we denote its length with $|s|$.

Def C.34. String Concatenation: The concatenation $\text{conc}(s, t)$ of two strings $s = \langle s_1, \cdots, s_n \rangle \in A^n$ and $t = \langle t_1, \cdots, t_m \rangle \in A^m$ is defined as $s + t$ or simply `st`

Example: conc("text", "book") = "text" + "book" = "textbook".

Def C.35. Kleene Plus and Kleene Star: Let A be an alphabet, then we define the sets:

- $A^+ := \bigcup_{i \in \mathbb{N}^+} A^i$ (*nonempty strings*), and
- $A^* := A^+ \cup \{\epsilon\}$ (*strings*).

Def C.36. Formal Language: A set $L \subseteq A^*$ is called a *formal language* over A.

Example: If $A = \{a, b\}$ then:

$$A^* = \bigcup_{n=0}^{\infty} A^n$$

where $A^0 = \epsilon$, $A^1 = \{a, b\}$, $A^2 = \{a, b, ab, ba\}$, and so on ...

$$A^* = \{\epsilon, a, b, aa, ab, ba, bb, aaa, aab, aba, abb, \cdots\}$$
$$A^+ = \{a, b, aa, ab, ba, bb, aaa, aab, aba, abb, \cdots\}$$

A formal language might be $L = \{a, b, aab, bbb, \cdots\}$

Def C.37. Repeating Chars: We use $c^{[n]}$ for the string that consists of the character `c` repeated n times.

Examples:

- Let $A = \{a, b\}$, $a^{[5]} =$ `"aaaaa"`.
- The set $M := \left\{ \text{ba}^{[n]} \mid n \in \mathbb{N} \right\}$ of strings that start with character b followd by an arbitrary numbers of a's is a formal language over $A = \{a, b\}$

> Note
>
> A formal language might be infinite and even undecidable even if the alphabet A is finite. For example, let $A = \{a, b\}$ then the formal language $L = \{a, ab, bba\}$ is finite but the formal language $L = \{ a^{[n]} \mid n \in \mathbb{N} \}$ is infinite.

Since we cannot list all strings in a formal language L because there are infinitely many, we need a **finite description** that can generate all strings in L. We need **Grammars**.

Def C.38. Phrase Structure Grammar: A phrase structure grammar (*type 0 grammar, unrestricted grammar, grammar*) is a tuple $\langle N, \Sigma, P, S \rangle$ where

- N is a finite set of **nonterminal symbols**,
- Σ is a finite set of **terminal symbols**. (members of $N \cup \Sigma$ are called symbols).
- P is a finite set of **production rules**: pairs $p := h \to b$ (also written as $h \Rightarrow b$), where $h \in (\Sigma \cup N)^* N (\Sigma \cup N)^*$ and $b \in (\Sigma \cup N)^*$. The string h is called the **head** of p and b the **body**.
- $S \in N$ is a distinguished symbol called the **start symbol** (also **sentence symbol**).

The sets N and Σ are assumed to be disjoint. Any word $w \in \Sigma^*$ is called a **terminal word**.

> Note
>
> Production rules map strings with at least one nonterminal to arbitrary other strings

Notation: If we have n production rule $h \to b_i$ sharing a head, we often write $h \to b_1 \mid \cdots \mid b_n$ instead.

Example: A simple grammar for english sentences:

$$\begin{aligned} \text{S} &::= \text{NP Vi} \\ \text{NP} &::= \text{Article N} \\ \text{Article} &::= \text{the} \mid \text{a} \mid \text{an} \\ \text{N} &::= \text{dog} \mid \text{teacher} \mid \cdots \\ \text{Vi} &::= \text{sleeps} \mid \text{smells} \mid \cdots \end{aligned}$$

Here S is the start symbol, NP, Article, N, and Vi are nonterminals, the, a, dog, smells, ... are terminals. Let's generate a valid sentence from the grammar:

```
NP           Vi
Article N    Vi
the     N    Vi
the    dog   Vi
the    dog sleeps
```

Def C.39. Lexical: A production rule whose head is a single nonterminal and whose body consists of a single terminal is called **lexical** or a **lexical insertion rule**.

Example: Both of the following rules are lexical

$$\begin{aligned} \text{Noun} &\to \text{"dog"} \\ \text{Verb} &\to \text{"run"} \end{aligned}$$

Def C.40. Lexicon of a Grammar: The subset of lexical rules of a grammar G is called the **lexicon** of G

Def C.41. Vocabulary: The set of body symbols are called the vocabulary (or the alphabet).

Def C.42. Lexical Categories of a Grammar: The nonterminals that appear in the heads of lexical rules are called lexical categories of the grammar.

Def C.43. Structural Rules: The non lexicon production rules are called structural.

Def C.44. Phrasal categories: The nonterminals in the heads of production rules that expand into other nonterminals are called phrasal (syntactic) categories.

Def C.45. G-Derivation: Given a phrase structure grammar $G := \{N, \Sigma, P, S\}$, we say G **derives** $t \in (\Sigma \cup N)^*$ from $s \in (\Sigma \cup N)^*$ in one step, iff there is a production rule $p \in P$ with $p = h \to b$ and there are $u, v \in (\Sigma \cup N)^*$, such that $s = uhv$ and $t = ubv$. We write $s \to_G^p t$ (or $s \to_G t$ if p is clear from the context) and use $\to_G^*$ for the reflexive transitive closure of $\to_G$. We call $s \to_G^* t$ a G-derivation of t from s.

Let's unravel that. Take the following grammar:

- $N = \{S\}$
- $\Sigma = \{a, b\}$
- $P = \{S \to \text{aSb}, S \to \text{ab}\}$
- $S = S$

The following derivations holds:

- $S \to^{p}_{G}$ aSb where $p : S \to$ aSb (*one step*)
- aSb $\to^{q}_{G}$ aabb where $q : S \to$ ab (*one step*)
- $S \to^{S\to\text{aSb}}_{G}$ aSb $\to^{S\to\text{ab}}_{G}$ aabb (*multiple steps*). So, $S \to^{*}_{G}$ aabb

Def C.46. Sentential Form: Given a phrase structure grammar $G := \{N, \Sigma, P, S\}$, we say that $s \in (\Sigma \cup N)^*$ is a **sentential form** of G, iff $S \to^{*}_{G} s$

Def C.47. Sentence: A sentential form that does not contain nonterminals is called a sentence of G, we also say that G **accepts** s. We say that G **rejects** s, iff it is not a sentence of G.

Examples

1. "Article teacher Vi" is a sentential form

$$\begin{aligned} S &\to_G \text{NP Vi} \\ &\to_G \text{Article N Vi} \\ &\to_G \text{Article teacher Vi} \end{aligned}$$

2. "the teacher sleeps" is a sentence

$$\begin{aligned} S &\to^{*}_{G} \text{Article teacher Vi} \\ &\to_G \text{the teacher Vi} \\ &\to_G \text{the teacher sleeps} \end{aligned}$$

Def C.48. Language Generation: The language $L(G)$ of G is the set of its sentences. We say that $L(G)$ is **generated** by G.

Def C.49. Equivalent Grammars: We call two grammars **equivalent**, iff they have the same languages.

Def C.50. Universal Grammar: A grammar G is said to be **universal** if $L(G) = \Sigma^*$

Def C.51. Syntactic Analysis: Syntactic analysis (parsing / syntax analysis) is the process of analyzing a string of symbols, either in a formal or a natural language by means of a grammar.

> **Note**
>
> The shape of the grammar determines the size of its language

Def C.52. Context-Sensitive: We call a grammar context-sensitive (or type 1), if the bodies of production rules have no less symbols than the heads.

Example: The language $\{a^{[n]}b^{[n]}c^{[n]}\}$ is accepted by

$$\begin{aligned} S &\to \text{a b c} \mid A \\ A &\to \text{a } A\ B \text{ c} \mid \text{a b c} \\ \text{c } B &\to B \text{ c} \\ \text{b } B &\to \text{b b} \end{aligned}$$

Def C.53. Context-Free: We call a grammar context-free (or type 2), iff the heads have exactly one symbol.

Example: The language $\{a^{[n]}b^{[n]}\}$ is accepted by $S \to$ a S b $\mid \epsilon$

Def C.54. Regular: We call a grammar regular (or type 3, or REG), if additionally the bodies are empty or consist of a nonterminal, optionally followed by a terminal symbol.

Example: The language $\{a^{[n]}\}$ is accepted by $S \to S$ a $\mid \epsilon$

> **Note**
>
> By extension, a formal language L is called *context-sensitive / context-free / regular*, iff it is the language of a respective grammar. Context-free grammars are sometimes `CFGs` and context-free languages `CFLs`.

Observation: Natural languages are probably context sensitive but parsable in real time!